

CITYU STUDY NOTES · A4 PRINT EDITION

# CS5222

## 计算机网络 · 课堂讲义与练习

沿老师课程学习，按需补基础，再做题与复习。

中文串讲 · English terminology · 双语题答 · 原课定位

| 文档                  | 内容                                                           | 页码    |
|---------------------|--------------------------------------------------------------|-------|
| Chapter01           | Chapter 1 / Introduction                                     | 2—18  |
| Chapter02           | Chapter 2 / Application Layer                                | 19—34 |
| Chapter03           | Chapter 3 / Transport Layer (当前 part 1—2)                    | 35—43 |
| Tutorial01          | Tutorial 1 / Switching, circuits and sharing                 | 44—50 |
| Tutorial02          | Tutorial 2 / Delay and traceroute                            | 51—57 |
| Tutorial03          | Tutorial 3 / HTTP and DNS                                    | 58—64 |
| Tutorial04          | Tutorial 4 / File Distribution, BitTorrent and DASH          | 65—68 |
| Tutorial05          | Tutorial 5 / Demultiplexing, Checksums and Reliable Transfer | 69—73 |
| Assignment01        | Assignment 1 / Chapters 1 and 2                              | 74—84 |
| ResearchReportGuide | Research Report Guide                                        | 85—88 |

**打印：**A4，实际大小 / 100%，每张一页。建议双面、长边翻转；正文与表格可黑白打印；颜色编码的权重图、分类图建议彩印。左侧20 mm、右侧17 mm供装订和批注。页码就是PDF页序，可按本表选择页码范围。

**使用：**每份Lecture/Chapter、Tutorial、Assignment保持独立起页；答案与原图已经全部展开。先遮住答案自己做，再核对过程。基础册独立保存，避免每讲重复打印。讲义中的电子链接仍保留；打印标注给出相应分册的专题页码；若链接指向整篇，则给文档起页。

**范围：**2026–10–09修订；当前已下载材料。优先级为有依据的学习建议，非教师公布的考试清单。原题、原结果、补充推导及本轮实测依正文标注区分；没有用历史材料替代未下载内容。

# Chapter 1 · Introduction | 一条消息怎样穿过互联网

课程目录 · Tutorial 1 [本册 p.44] · Tutorial 2 [本册 p.51] · Assignment 1 [本册 p.74] · 下一章 [本册 p.19]

你在宿舍点开课程网页，屏幕很快显示内容。中间不是一条从电脑直达学校服务器的专属电线，而是许多设备、链路和网络共同协作。本章先建立这幅全景：谁负责收发、数据怎样共享道路、为什么会等待或丢失、复杂系统如何分工。后面每章会放大其中一部分。

依据当前 完整PPT（78页）及课堂Q&A（6页）。保留老师Chapter命名与顺序。原PPT说明其改编自 Kurose/Ross 教材课件、由 Man Hon Cheung 修改；原图版权属于原作者。补充算例在正文中标注。

**基础复习（可跳过）：**先自查 $1500\text{ B} \div 2\text{ Mbps}$ 会不会算；若卡住读单位与时间轴 [基础册 p.37]。多人同时活跃的概率看组合概率 [基础册 p.28]。

本页目录 / Contents

- Chapter 1 · Introduction | 一条消息怎样穿过互联网 [本册 p.2]
  - 1. What is the Internet? | 先分清“设备视角”和“服务视角” [本册 p.3]
  - 2. What is a protocol? | 格式、顺序、动作缺一不可 [本册 p.4]
  - 3. Network edge and access | 接入网把你带到第一个入口 [本册 p.4]
  - 4. Circuit versus packet switching | 预留座位，还是来了再排队 [本册 p.6]
    - 共享带来的容量收益，有什么条件？ [本册 p.8]
  - 5. Forwarding, routing and a network of networks | 路口动作与整条路线 [本册 p.9]
  - 6. Delay and loss | 慢在哪一段，公式才选得对 [本册 p.10]
  - 7. Real delays and throughput | 测得的RTT不是某一条线的长度 [本册 p.12]
  - 8. Layers and encapsulation | 把一件复杂的事拆成清楚的职责 [本册 p.13]
  - 9. Security and history | 知道威胁，也知道哪些数字属于当年 [本册 p.15]
  - 10. Chapter 1 Q&A | 直接对照老师的五道课堂题 [本册 p.15]
  - 11. 回顾、任务与来源索引 [本册 p.18]

| 主题                          | 优先级  | 难度与卡点     | 掌握要求；依据                                                     |
|-----------------------------|------|-----------|-------------------------------------------------------------|
| Internet、protocol、edge/core | 核心必会 | 低至中；设备和功能 | 用英文解释与举例；Chapter 1完整课件，第4–12张幻灯片、Q&A                        |
| 交换、复用、流水线                   | 核心必会 | 中；容量与时序   | 画图、手算、说明假设；Chapter 1完整课件，第20–32张幻灯片、Tutorial 1、Assignment 1 |
| delay/loss/throughput       | 核心必会 | 中；四类时延和瓶颈 | 带单位计算与解释观察；Chapter 1完整课件，第43–55张幻灯片、Tutorial 2、Assignment 1 |

| 主题          | 优先级  | 难度与卡点       | 掌握要求；依据                                   |
|-------------|------|-------------|-------------------------------------------|
| 协议分层与封装     | 核心必会 | 中；服务方向/数据路径 | 跟踪消息跨层变化；Chapter 1完整课件，第56—65张幻灯片、Q&A     |
| 接入、介质、ISP结构 | 常规掌握 | 中；共享与独占     | 比较基本机制；Chapter 1完整课件，第11—19、33—42张幻灯片     |
| 安全概念与历史     | 常规掌握 | 低；定义/时代背景   | 识别威胁、解释演进；Chapter 1完整课件，第66—78张幻灯片；当前课件内容 |

优先级与难度独立。当前Tutorial、Assignment和课堂题是直接依据；历史复习笔记只作辅助，不按单次出现宣称香港本学期高频或QE范围。

## 1. What is the Internet? | 先分清“设备视角”和“服务视角”

先从一条网页请求会经过什么来认识互联网。你的电脑和学校服务器是主机，也叫端系统（host / end system）；铜缆、光纤或无线链路（link）让信号在设备之间传播；交换机和路由器负责把数据转到下一段。互联网服务提供商（Internet Service Provider, ISP）把这些网络连接起来。这就是老师所说的“设备组成视角”（nuts and bolts）。

# What's the Internet: “nuts and bolts” view



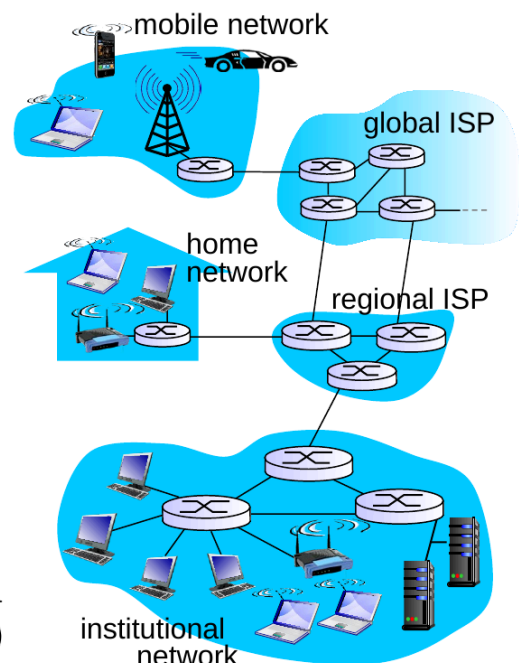
millions of connected computing devices:  
*hosts = end systems*  
 running *network apps*



❖ *communication links*  
 ■ fiber, copper, radio, satellite



❖ *Packet switches*: forward packets (chunks of data)  
 ■ *routers* and *switches*



Introduction 1-4

看图从home network沿连接走到regional/global ISP，再到别的网络。手机、PC、服务器属于应用所在端系统；交换机和路由器服务中间转发。原Chapter 1完整课件，第5张幻灯片的联网相框、冰箱等例子说明“host不等于台式电脑”。

服务视角（Chapter 1完整课件，第7张幻灯片）则问应用能获得什么：程序通过接口请求把数据送到远端程序，好比寄件时按规则写地址、交给邮政系统。应用不必亲自安排沿途每一个路口；但选用的传输服务决定它获得哪些保证。RFC（Request for Comments）是互联网技术文档系列；IETF（Internet Engineering Task Force，互联网工程任务组）制定许多互联网标准（Chapter 1完整课件，第6张幻灯片）；并非每一份RFC都是正式标准。

**English takeaway:** The Internet connects networks of hosts and packet switches, and provides communication services to applications through interfaces.

来源：Chapter 1完整课件，第4张幻灯片。

## 2. What is a protocol? | 格式、顺序、动作缺一不可

---

两个人问时间，要先提出问题，再给出回答；听不清时还可能重问。网络中的协议（protocol）同样需要约定交换消息的**格式、顺序，以及发送/收到消息时的动作**。只有“说英文”不够，还要知道先问还是先答、没收到怎么办。网络协议把这类约定变成机器可执行规则。来源：Chapter 1完整课件，第8—9张幻灯片。

课堂图先TCP连接请求/响应，再HTTP请求/文件。它是教学时序，不意味着所有互联网应用都先用TCP；下一章会比较TCP与UDP。发送方与接收方遵守同一套约定，才能把字节解释成相同的内容。

**自查 / Check:** 协议是否只规定报文长什么样？ / Does a protocol specify only the format of a message?

**答案 / Answer**

不，还规定消息顺序和在发送/接收等事件发生时执行的动作。 / No; it also specifies message ordering and actions taken on relevant events.

## 3. Network edge and access | 接入网把你带到第一个入口

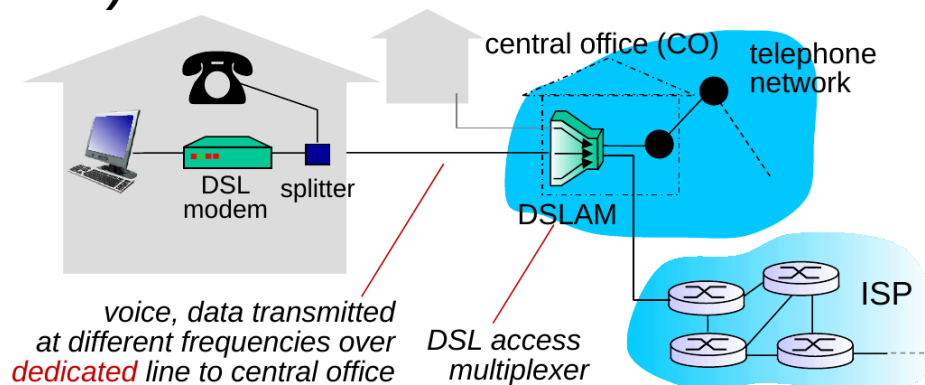
---

接入网把家里或学校的端系统连到互联网的第一个入口，常见方式有家庭接入、学校/公司接入、移动接入。每次比较都问两件事：速率多少（bit/s）？资源是共享还是专用？仅看到“带宽很大”还不能推断每个用户都能持续独占它。

以家里的电话线接入为例：打电话和上网共用一根线，却占用不同频段，像在同一条路上划出不同车道。这种方式叫数字用户线路（Digital Subscriber Line, DSL）。运营商一端用DSLAM设备汇集许多用户的线路，再接入ISP网络。家到局端的这一小段可以专用，后面的道路仍可能与别人共享。

家里常见的一只“路由器盒子”也可能同时做几份工作：modem把数据转换为线路上的信号，路由/NAT负责转发和地址转换，防火墙按规则过滤流量，Wi-Fi接入点（access point, AP）提供无线连接。装在同一个盒子里，不等于这些功能相同。

# Home access net: digital subscriber line (DSL)



- ❖ **DSL modem**: takes digital data & translates it to high frequency tones for transmission over telephone wires.
- ❖ use *existing* **telephone line** by frequency division multiplexing
  - voice over DSL phone line goes to telephone net
  - data over DSL phone line goes to Internet
- ❖ **DSL access multiplexer (DSLAM)**: Translate analog signals from many such houses into digital format.

Introduction 1-13

企业以太网把主机连到交换机，再由机构路由器连ISP。无线LAN经AP共享无线介质，蜂窝接入经运营商基站。课件列出的54Mbps、3G/4G等速率是原版本的示例，不作为2026年的产品上限或购买指标；标称速率也不同于应用可得吞吐。

| 介质 (Chapter 1完整课件, 第17—19张幻灯片) | 信号怎样走    | 要理解的差别                      |
|--------------------------------|----------|-----------------------------|
| Twisted pair 双绞线               | 铜导线中的电信号 | 成本、距离、布线与干扰                 |
| Coaxial cable 同轴               | 同轴导体     | 可承载多个频带；HFC结合光纤与同轴          |
| Fiber 光纤                       | 光信号      | 高容量、抗电磁干扰；“每个光脉冲一个bit”是入门简化 |
| Radio 无线                       | 电磁波传播    | 反射、遮挡、干扰，共享频谱               |
| Satellite 卫星链路                 | 经卫星中继    | 传播距离与轨道决定时延；不能将一个示例时延套到所有轨道 |

**English:** Access capacity, sharing, and physical propagation are separate considerations. A dedicated access segment does not imply an end-to-end dedicated path.

来源：Chapter 1完整课件，第11—19张幻灯片。

## 4. Circuit versus packet switching | 预留座位，还是来了再排队

Circuit switching (电路交换) 先预留路径上的资源，即使使用者暂时不发送，别人通常也不能借用这份预留。Chapter 1完整课件，第22张幻灯片原图每条边有4条电路，一次连接在经过的每段占一份。频分复用 (frequency-division multiplexing, FDM) 让各连接使用不同频带；时分复用 (time-division multiplexing, TDM) 则轮流分配重复出现的时隙。同一用户在不同链路使用哪一个编号电路不必相同。

### Circuit switching

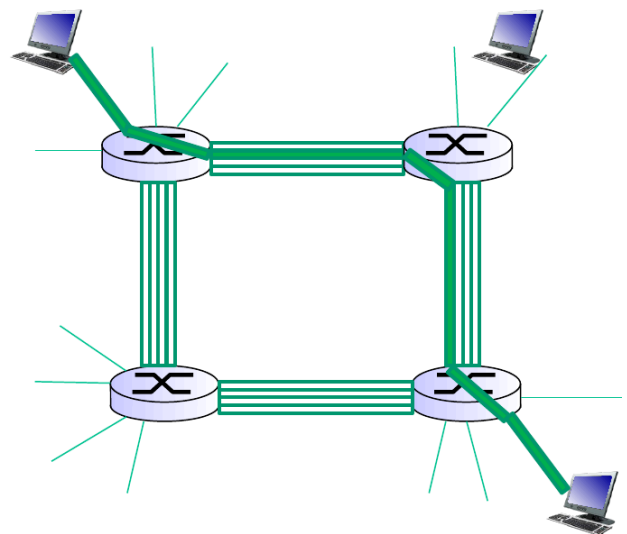
**Circuit switching:** end-end resources are **reserved** for communication between source & dest:

In diagram, each link has four circuits.

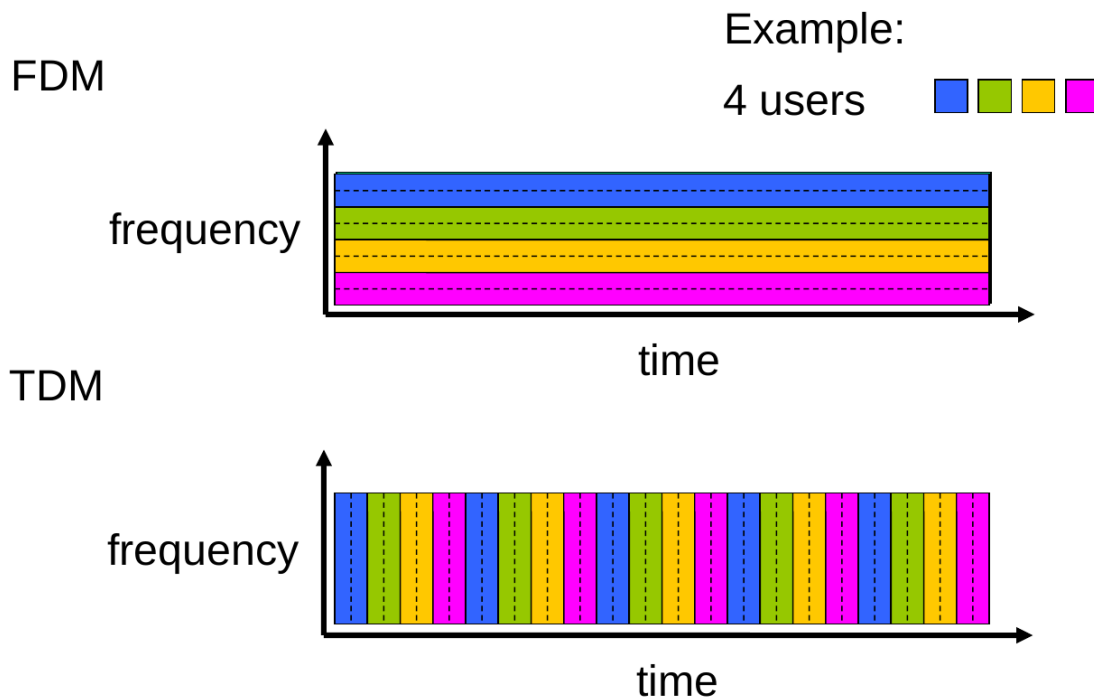
call gets 2<sup>nd</sup> circuit in top link and 1<sup>st</sup> circuit in right link.

circuit segment idle if not used by call (*no sharing*)

Commonly used in traditional telephone networks



# Circuit switching: FDM versus TDM

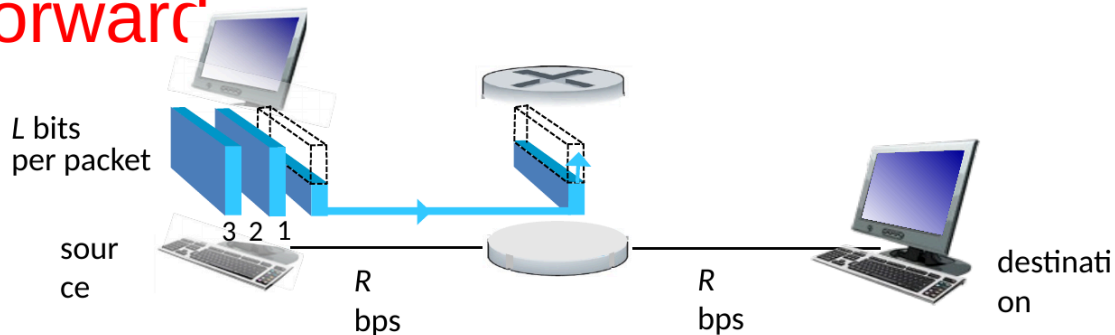


Introduction 1-23

Packet switching (分组交换) 把应用消息拆成包，按需使用链路。一包发送时可用链路发送速率 $R$ ，竞争者可能排队；这不表示每个并发用户都永久获得 $R$ 。像食堂公共座位提高利用率，但午饭同时来的人多了还是要等，不能只记“共享比较好”。

**Store-and-forward (存储转发)** 要求收完整个包再向下一跳发送。包长 $L$  bit、速率 $R$  bit/s，一跳发送耗时 $L/R$ 。在忽略传播、排队、处理的Chapter 1完整课件，第26张幻灯片模型中，一个包过两条链路耗时 $2L/R$ 。注意“一跳”不自动包含整条路径。

# Packet-switching: store-and-forward



takes  $L/R$  seconds to  
transmit (push out)  $L$ -bit  
packet into link at  $R$  bps

*store and forward*: entire  
packet must arrive at router  
before it can be transmitted

on next link  
end-to-end delay =  $2L/R$  (assuming  
zero propagation delay) } more on delay shortly ...

Introduction 1-26

三个包过两跳不是  $6L/R$ ，因为后一包可在前一包进入第二链路时占用第一链路。完整时序见基础时间轴〔基础册 p.40〕与 Tutorial1 Q1〔本册 p.45〕。等长包、等速率、连续发送且没有额外时延时， $P$  个包、 $N$  条链路总耗时  $(P + N - 1)L/R$ 。

## 共享带来的容量收益，有什么条件？

原例：1Mbps 链路，每用户活跃时 100kbps、活跃概率 0.1。电路交换最多固定支持 10 人；35 人做分组共享时，假设用户独立，活跃人数  $K$  服从 Binomial(35, 0.1)。

$$P(K > 10) = \sum_{k=11}^{35} \binom{35}{k} 0.1^k 0.9^{35-k} \approx 0.0004243. \quad (1.1)$$

原页写“less than .0004”不够精确；独立模型重算约 0.0004243，约 0.04243%，是略大于 0.0004。Chapter 1 完整课件，第 30 张幻灯片只有 Binomial 标题，补课给出从列举到公式的台阶。

这说明超额需求较少，不是保证零排队、零丢包或人人始终满速；短时到达突发、包调度和缓冲也影响等待。用户相关性改变时需要重新建模。分组交换适合突发需求，电路预留则提供更可预测资源，选择取决于服务需求。

来源：Chapter 1 完整课件，第 29 张幻灯片。

## 5. Forwarding, routing and a network of networks | 路口动作与整条路线

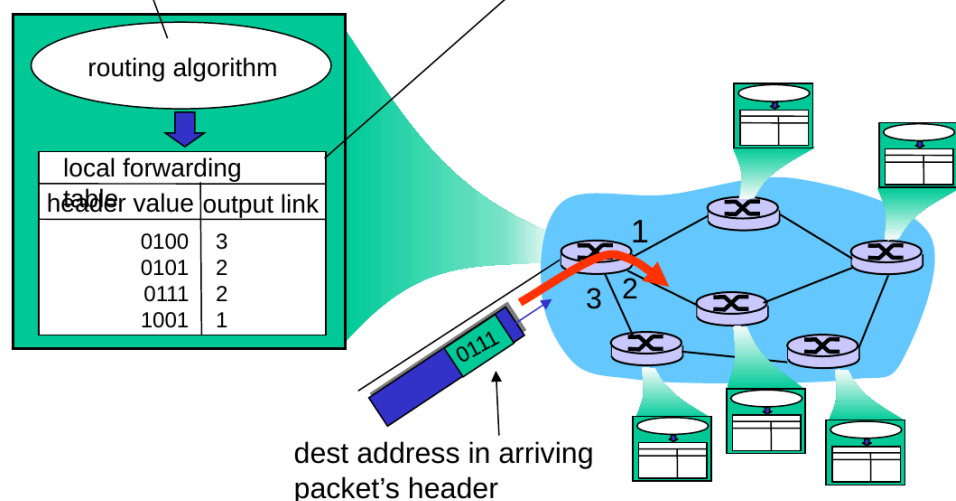
课堂的forwarding是当前设备查表，把到来的包从合适出口送走；routing是确定路径并形成所需转发信息的过程。导航规划与开到路口选出口的类比有帮助，但真实路由可能分布式更新，不必有一个中央导航员。

### Two key network-core functions

*routing*: determines source-destination route taken by packets

- *routing algorithms*

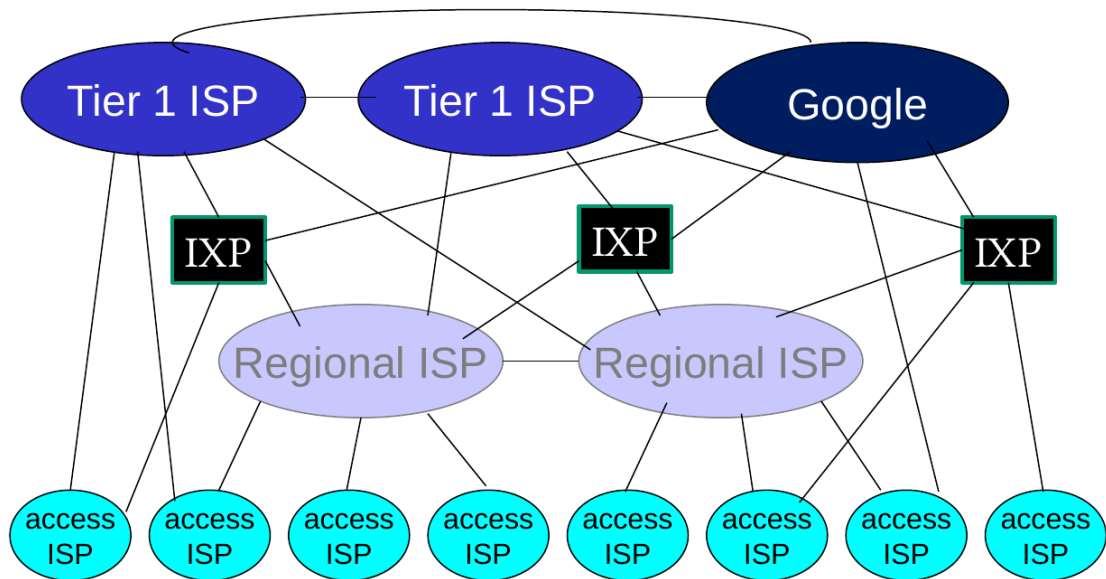
*forwarding*: move packets from router's input to appropriate router output



Network Layer 4-28

不同网络还需要相互连接。一步步构造互联网：若 $N$ 个接入ISP两两直连，需 $N(N-1)/2$ 条连接，扩展困难；接入ISP可以向上游购买传输服务（transit），让上游替它把流量送往更多网络；也可以与另一网络直接互联（peering）。互联网交换点（Internet Exchange Point, IXP）提供集中互联的位置。区域网络（regional network）连接一片地区，内容提供商网络（content-provider network）连接自己的服务节点。内容提供商将服务放近用户、建立自有网络，会减少对某些上游路径的依赖，从而可能缩短传输路径。原公司的例名保留为图中时代背景。

# Internet structure: **network of networks!**



Small # of well-connected large networks

**"tier-1" commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage

**content provider network** (e.g, Google): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs Introduction 1-42

**English:** Forwarding is a local per-packet action; routing determines paths. The Internet consists of interconnected networks with both technical and economic relationships.

来源：Chapter 1完整课件，第28张幻灯片；第33—42张幻灯片。

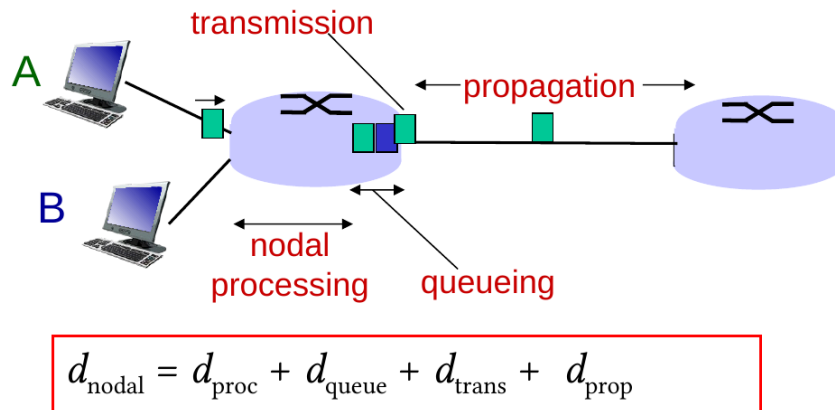
## 6. Delay and loss | 慢在哪一段，公式才选得对

区分四类时延：

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}. \quad (1.2)$$

| 项            | 发生什么        | 计算/影响                 |
|--------------|-------------|-----------------------|
| Processing   | 检错、读头、决定出口  | 题中给定或忽略               |
| Queueing     | 等前面的包使用出口   | 取决于到达与队列状态            |
| Transmission | 将整包推入链路     | $L/R$ ; L bit、R bit/s |
| Propagation  | 已进入介质的信号往前走 | $d/s$ ; d m、s m/s     |

## Four sources of packet delay



$d_{\text{proc}}$ : nodal processing

- check bit errors
- determine output link
- typically < msec

$d_{\text{queue}}$ : queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Introduction 1-45

发送时延看一整包需要多久进入链路，传播时延看已经出发的信号多久走到另一端。下面用老师的车队例子，把这两个计时起点分开。

课堂的车队原例：10辆车，每辆收费12秒，最后一辆过第一个收费亭需120秒；100km以100km/h行驶需1小时，全部在第二亭前到齐需62分钟。Chapter 1完整课件，第48张幻灯片改为每车1分钟、车速1000km/h；首车1分钟后出发，再走6分钟， $t=7$ 分钟到第二亭，早于第10辆 $t=10$ 分钟离开第一亭。两题都从第一辆开始服务计时，没让第二亭再处理一遍。

队列的traffic intensity为  $\rho = La/R$ ， $a$ 为平均包到达率 (packets/s)， $L$ 为本模型固定包长。 $\rho$ 接近1时在常见随机排队模型中等待显著增长； $\rho > 1$ 长期输入超过服务能力，无限缓冲理想模型不稳定，现实有限缓冲会丢包。仅平均 $\rho < 1$ 不能保证任意流量下没有大突发或所有等待性质都良好，课件曲线是模型直觉。

把四项接成一次完整计时（教学示范）：某节点从开始处理一个500 B的包计时。处理1 ms、排队2 ms，随后以2 Mbps发送；链路长1000 km，传播速度 $2 \times 10^8$  m/s。忽略其他开销。

发送部分先把500 B换成4000 bit，得到 $4000/(2 \times 10^6) = 2$  ms；传播部分把1000 km换成 $10^6$  m，得到5 ms。从开始处理到末位到达下一节点，总共 $1+2+2+5=10$  ms。首位到达时包可能还没收齐，所以这里明确计到末位。

**独立变式 / Transfer:** 另一个包1000 B,  $R=4$  Mbps,  $d=3000$  km,  $s=2\times 10^8$  m/s, 处理1 ms、排队3 ms。从开始处理到整包到达下一节点需多久? / A 1000-byte packet has a 4-Mbps link, 3000-km distance, propagation speed  $2\times 10^8$  m/s, 1-ms processing and 3-ms queueing. Find the time from processing start to complete reception at the next node.

**答案 / Answer**

发送2ms、传播15ms, 合计1+3+2+15=21ms。 / Transmission is 2 ms, propagation is 15 ms; total delay is 21 ms.

来源: Chapter 1完整课件, 第44–49张幻灯片; 第47张幻灯片。

## 7. Real delays and throughput | 测得的RTT不是某一条线的长度

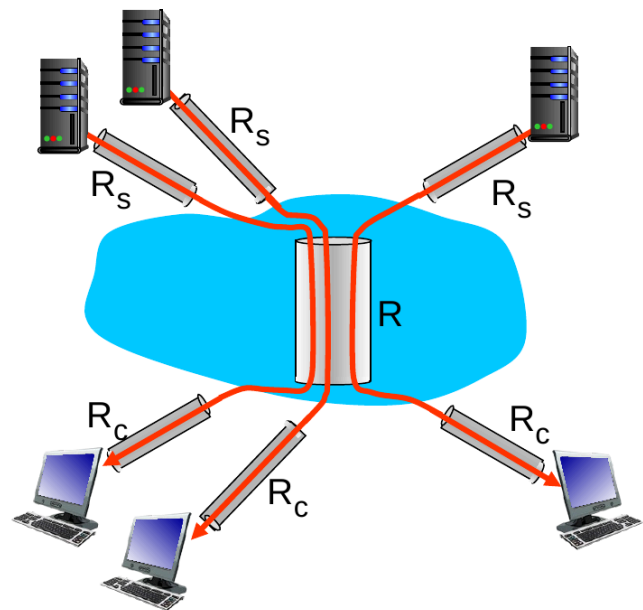
用traceroute逐渐增加TTL探查沿途响应, 每行列出应答地址与往返时间RTT。后跳比前跳小并不矛盾: 探测发生时间、排队、回程路径和应答处理可能不同; 不能简单逐行相减当作单条链路时延。星号表示规定时间内未收到响应, 不足以单独证明那台路由器坏了或一定被防火墙挡住。带日期的本机观察放在Tutorial2 Q5 [本册 p.56]。

课堂的丢包来自缓冲满等机制; 包是否重传、由谁重传取决于协议, 不是所有丢包都被网络自动恢复。吞吐throughput是单位时间实际交付多少bit, 容量capacity是链路能力; 应用有效载荷的goodput还会扣去协议与重传等开销。

简单持续流、无竞争时, 多链路瓶颈近似  $\min(R_s, R_c)$ ; 10条连接公平分享骨干R且其他条件简化时, 每连接为  $\min(R_s, R_c, R/10)$ 。公平分享是题设, 不是每个真实网络自动保证的定理。

# Throughput: Internet scenario

per-connection end-end  
throughput:  
 $\min(R_c, R_s, R/10)$   
in practice:  $R_c$  or  $R_s$  is  
often bottleneck



10 connections (fairly) share  
backbone bottleneck link  $R$  bits/sec

Introduction 1-55

**自测 / Check:** 接入10Mbps、服务器20Mbps、5条流公平共享30Mbps骨干，每流瓶颈吞吐？ /  
Each flow has 10-Mbps access and a 20-Mbps server link; five flows fairly share a 30-Mbps backbone. Find the per-flow bottleneck rate.

**答案 / Answer**

$\min(10, 20, 30/5) = 6\text{Mbps}$ 。 / 6 Mbps under the stated sharing model. 容量不要相加成60Mbps。

来源：Chapter 1完整课件，第50—51张幻灯片；第52张幻灯片。

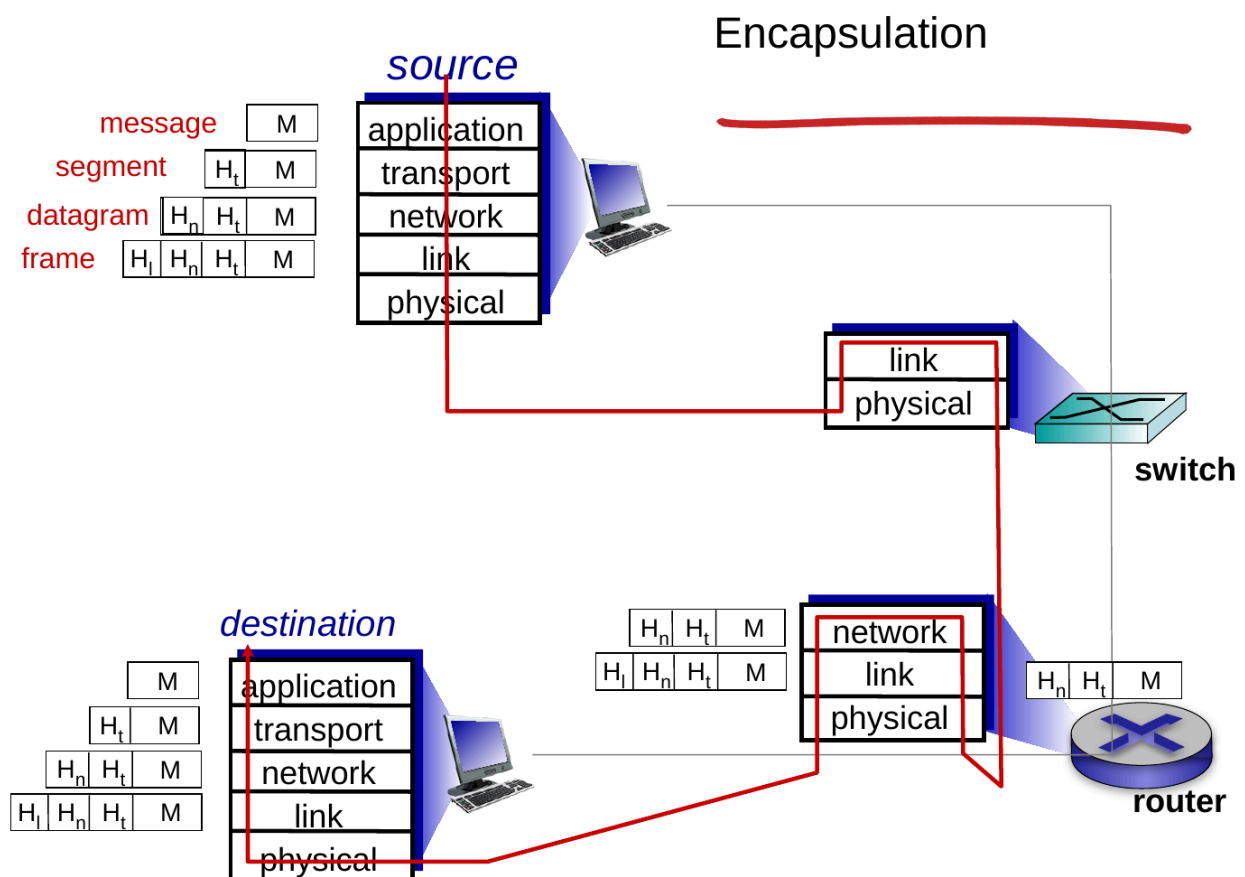
## 8. Layers and encapsulation | 把一件复杂的事拆成清楚的职责

老师用航空旅行解释分工：买票、托运行李、登机、起飞、航线各有职责。某一层使用下层服务，同时向上层提供服务；合理接口让内部变化不必牵动所有部分。网络分层借助清楚的责任和接口组织复杂任务。来源：Chapter 1完整课件，第58—60张幻灯片。

| Internet五层（从上向下） | 核心职责       | 课堂例子           |
|------------------|------------|----------------|
| Application      | 应用间协议与内容含义 | HTTP、SMTP、FTP  |
| Transport        | 进程到进程传输    | TCP、UDP；保证不同   |
| Network          | 主机间数据报与路由  | IP             |
| Link             | 相邻节点间传输    | Ethernet、Wi-Fi |
| Physical         | 介质上的比特信号   | 铜线/光/无线信号      |

从底向上编号时transport是第4层。OSI另列presentation/session，Internet应用仍可能需要编码、加密、会话等功能，只是不独立命名为这两层；“没有这一层”不等于“不需要这个功能”。

封装（encapsulation）将上一层信息作为本层payload，加上本层头部，某些链路协议还加尾部。例如浏览器的HTTP请求先是一段应用消息（message），TCP给它加传输头成为报文段（segment），IP再加网络头成为数据报（datagram），链路层包成帧（frame），最后变成介质上的比特信号。接收端按相反顺序解释并移去各层信息。UDP在传输层通常也称datagram，读这个词时要确认所在层。



Introduction 1-64

沿原图看：端系统走完整协议栈；典型转发路由器处理到网络层，以太网交换机主要处理链路层。不要由这个教学模型推断所有现实设备绝不检查更高层。每跳链路帧可更换，应用消息不因此被每个路由器当成网页程序执行。

**English:** Each layer offers services to the layer above and uses services below. Encapsulation adds the information needed by the current layer.

## 9. Security and history | 知道威胁，也知道哪些数字属于当年

介绍恶意软件、DoS、嗅探与地址伪造。病毒常依附宿主内容并需执行，蠕虫可自动传播；并非“被动收到任意包就必然感染”，通常还需漏洞或执行机制。Botnet由受控设备组成，DDoS是分布式资源耗尽；packet sniffing能看到可访问的流量，但加密与网络结构限制能读到的明文。伪造源IP不等于已经能接收发回那个源地址的响应。

按时间讲分组交换研究、ARPANET、互联思想、TCP/IP与DNS、Web商业化、移动与云。应记住问题如何推动结构演变：让不同网络自治互联、尽力而为、分散控制；“stateless routers”在这里指不为每个端到端连接预留电路状态，不是路由器没有路由表或任何状态。

保留原页历史顺序，同时澄清两处可核查的简写：Chapter 1完整课件，第74张幻灯片将ALOHAnet标为1970卫星网络；夏威夷大学记录其岛际packet-radio服务于1971年6月开始，卫星连接属于后续发展。UH项目史 Chapter 1完整课件，第76张幻灯片的Mosaic年份应注意版本语境，NCSA记录其最早普及阶段在1993年。NCSA历史 两处查阅于2026-09-23。

课堂的2016设备数量、早期社交网络用户数只作为课件当时背景，不称当前规模；“instantaneous”是宣传式简写，跨网络通信仍有非零时延。

来源：Chapter 1完整课件，第67—71张幻灯片；第73—77张幻灯片；第77张幻灯片。

## 10. Chapter 1 Q&A | 直接对照老师的五道课堂题

以下为原题条件的双语整理，答案为依据本章的推导，原Q&A文件没有附独立教师答案页。原图可在末尾展开。

1. Which is not an edge device: PC, smartphone, server, router? / 哪个不属于本章端系统意义的网络边缘设备？答D, router；这里edge按运行应用的end systems定义，不否认现实术语也有“edge router”。 / D, a router, using the chapter's end-system definition of edge.
2. Three L-bit packets cross two R-bit/s store-and-forward links. Ignore propagation, queueing and processing. Find completion time. / 3个等长包过2条等速存储转发链路，忽略其他时延，多久？答B,  $4L/R$ ；流水线时间表见§4。 / B,  $4L/R$ , because the links can work on different packets concurrently.
3. Which delay is affected by congestion in the chapter's model? / 四项时延中哪项随拥塞变化？答B, queueing；其他给定包长/速率/距离条件不变。真实设备处理负载也可能变化，但原题考四类简化模型。 / B, queueing delay in the stated model.
4. A 100Mbps modem connects a phone through a 54Mbps router, upgraded to 1Gbps. What is ideal throughput after upgrading? / 100Mbps外网口，54Mbps路由器换1Gbps后理想瓶颈？答A,  $\min(100, 1000) = 100\text{Mbps}$ ；还假设手机及其他段无更低瓶颈。 / A, 100 Mbps under the simplified bottleneck assumptions.

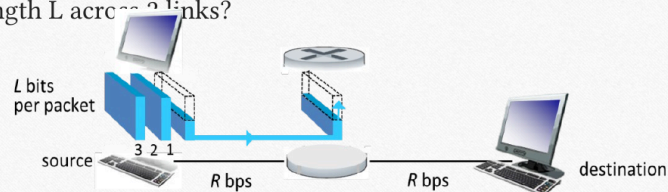
5. Which transport-layer statements are true? / 传输层哪项正确? A总可靠: 错, UDP不保证; B是第4层: 对 (底层起数); C为下方网络层提供服务: 错, 服务向上; D给segment加头再上送 application: 错, 发送方向给应用消息加传输头后向下。 / Only B is correct under bottom-up layer numbering.

#### 原Q&A页图 / Original question slides

- Which of this is **not** a network edge device?
  - A) PC
  - B) Smartphones
  - C) Servers
  - D) Router

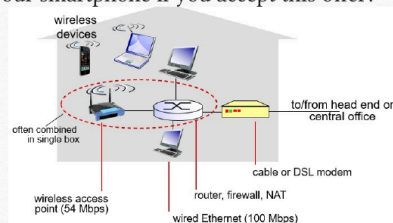
- (Store-and-forward transmission) Ignore queuing, propagation delay, and processing delay. What will be the end-to-end delay for sending 3 packets each of length  $L$  across 2 links?

- A.  $3L/R$
- B.  $4L/R$
- C.  $5L/R$
- D.  $6L/R$



- Which of the following packet delay(s) is/are affected by network congestion?
- A) nodal processing delay
- B) queueing delay
- C) transmission delay
- D) propagation delay

- Suppose you are using a modem with speed 100 Mbps for your wireless router with speed 54 Mbps. They are connecting your smartphone to the Internet. During contract renewal, PCCW offers to give you a new Wi-Fi router of 1 Gbps to replace the old one. What is the end-to-end throughput of your smartphone if you accept this offer?
- A) 100 Mbps
- B) 54 Mbps
- C) 154 Mbps
- D) 1 Gbps



- What of the following statement(s) is/are true about the transport layer?
- A) It is always providing a reliable process-process data transfer.
- B) It is in Layer 4 of the Internet protocol stack.
- C) It is providing services for the network layer below.
- D) In the encapsulation, it adds a header to the segment and pass it to the application layer above.

## 11. 回顾、任务与来源索引

顺着主线复述：端系统产生消息 → 接入网进入互联网 → 分组在链路及交换设备间前进 → 共享导致排队/丢失，路径容量决定瓶颈 → 分层协议规定每一部分的格式与动作。接着做Tutorial1〔本册 p.44〕的交换与容量题、Tutorial2〔本册 p.51〕的时延与测量，再做Assignment1 P6/P31〔本册 p.74〕。

英文独立表达：Explain why increasing a link's rate does not reduce its propagation delay. / 解释为什么增加速率不减少传播时延。要点： $L/R$ 改变， $d/s$ 由距离与传播速度决定。/ Link rate changes transmission time; propagation depends on distance and signal speed.

首轮可选 N015（分组交换）、N025（四类时延）、N033（流量强度）、N072（流水线），再在Markji按卡号复习。微课入口：NET02。

| 原完整PPT页码 | 本文位置   | 覆盖内容                     |
|----------|--------|--------------------------|
| 1—3      | 开篇     | 来源、目标、路线；目录页不重复计主题       |
| 4—9      | §1—2   | 设备/服务/API/RFC、协议与原对话     |
| 10—19    | §3     | 接入、DSL、家庭/企业/无线、物理介质     |
| 20—28    | §4—5   | 电路、FDM/TDM、分组、存储转发、转发/路由 |
| 29—32    | §4     | 原35用户例、二项补足、交换比较         |
| 33—42    | §5     | ISP逐步构造、peering/IXP/内容网络 |
| 43—49    | §6     | 四类时延、两道车队题、rho           |
| 50—55    | §7     | traceroute、丢包、吞吐、共享瓶颈    |
| 56—65    | §8     | 航空类比、五层/OSI、封装与名称        |
| 66—71    | §9     | 安全目标与四类威胁                |
| 72—78    | §9、§11 | 演进、时代数字、总结               |
| Q&A 1—6  | §10    | 标题及全部5道题                 |

part1的49页对应完整1—49；part2的29页对应完整50—78，页码重编不算新增教学内容。原图在相关段落展示，计算步骤见正文。

## CS5222-Chapter02

# Chapter 2 · Application Layer | 让两端的程序真正说上话

课程目录 · 上一章 [本册 p.2] · Tutorial 3 [本册 p.58] · Assignment 1 [本册 p.74] · Research Report [本册 p.85]

上一章知道消息可以穿过网络，这一章把镜头移到两端：浏览器怎样开口要网页，邮件怎样送到邮箱，名字怎样换成地址，多个用户怎样互相传文件，视频为什么可以边下边播。最后亲手让两个本机程序交换一句话。

依据当前 完整Chapter2 (107页)，保持 应用原则 → HTTP → 邮件 → DNS → P2P → 视频/CDN → socket 的实际正文顺序。早期目录列有FTP，但当前完整文件未提供独立FTP教学段，后半目录为视频/CDN。课件来源为Kurose/Ross、Man Hon Cheung修改，原图版权保留。

**可选基础：** 报文与字节 [基础册 p.39]、时间轴 [基础册 p.40]。运行Python先读Lecture1的函数与文件 [CS5489册 p.8]，按需要复习即可。

## 本页目录 / Contents

- Chapter 2 · Application Layer | 让两端的程序真正说上话 [本册 p.19]
  - 1. Applications, processes and sockets | 先找到真正对话的双方 [本册 p.20]
  - 2. Service requirements | “可靠”不等于“准时” [本册 p.21]
  - 3. Web and HTTP | 拿到HTML不等于拿齐整页 [本册 p.21]
    - 连接可以复用，请求是否并行还要另问 [本册 p.21]
    - 读报文：把每一行与问题对应 [本册 p.22]
  - 4. Cookies and caches | 识别会话，与复用内容是两件事 [本册 p.23]
  - 5. Electronic mail and SMTP | 把“送到邮箱”和“打开邮箱”拆开 [本册 p.24]
  - 6. POP3, IMAP and webmail | 下载之后要不要留在服务器？ [本册 p.25]
  - 7. DNS services and hierarchy | 通讯录不是只放在一台电脑上 [本册 p.25]
  - 8. Recursive versus iterative DNS | 是“替我办完”，还是“告诉我下一站” [本册 p.26]
  - 9. DNS records and messages | 名字、值、类型、有效时间要一起读 [本册 p.27]
  - 10. P2P file distribution | 新加入的人也可以带来上传能力 [本册 p.27]
  - 11. BitTorrent | 先找邻居，再交换稀缺点 [本册 p.28]
  - 12. Video, DASH and CDN | 边下边播，需要的不只是平均带宽 [本册 p.29]
  - 13. Socket programming | 把课堂的大写转换例子真正跑通 [本册 p.29]
  - 14. Back to the course | 用一条请求链把本章连起来 [本册 p.31]
  - 课堂 Q&A：按原题判断并说明条件 [本册 p.31]
    - 1. Transport service | 为应用选择传输服务 [本册 p.31]
    - 2. HTTP timing | 从HTML到两张图片 [本册 p.32]
    - 3. Mail addressing | 目标主机与服务端口 [本册 p.32]
    - 4. DNS records | 一个名字怎样对应多台服务器 [本册 p.32]
    - 5. BitTorrent | 找到谁、向谁下载 [本册 p.32]
    - 6. Socket services | 接口提供了什么 [本册 p.33]

- 15. Source coverage and cautions | 每一页有去处 [本册 p.33]

| 主题                   | 优先级  | 难度与卡点        | 掌握要求与依据                                         |
|----------------------|------|--------------|-------------------------------------------------|
| 架构、进程、socket、TCP/UDP | 核心必会 | 中；对象/服务边界    | 解释并运行通信；Chapter 2完整课件，第4—17、94—105张幻灯片          |
| HTTP消息与时延            | 核心必会 | 中；连接、流水线、对象数 | 解读完整请求、画RTT；Chapter 2完整课件，第19—37张幻灯片、Tutorial 3 |
| SMTP、POP3、IMAP       | 核心必会 | 中；投递和访问分开    | 写出完整事务与状态；Chapter 2完整课件，第39—51张幻灯片、Assignment 1 |
| DNS、递归/迭代、缓存         | 核心必会 | 中；谁向谁问       | 跟踪消息与算RTT；Chapter 2完整课件，第53—69张幻灯片、Tutorial 3   |
| P2P、BitTorrent       | 常规掌握 | 中；资源下界与实际性能  | 推导三项下界，解释分块；Chapter 2完整课件，第71—80张幻灯片            |
| 视频、DASH、CDN          | 常规掌握 | 中；码率/吞吐/缓冲   | 解释机制、算小例；Chapter 2完整课件，第82—92张幻灯片               |

优先级依据当前课件与配套任务；考试范围以教师公布的要求为准。

## 1. Applications, processes and sockets | 先找到真正对话的双方

浏览器、邮件、游戏、会议和视频程序通常运行在端系统上，开发一个网络应用不需要重新编写沿途每台路由器。进程（process）是运行中的程序；一个主机能同时有多个进程，因此只写主机IP不够，还需端口等传输端点信息。

Client/server架构中，一方提供相对稳定可达的服务，客户端发起请求；P2P中节点既请求也提供资源。纯P2P不依赖永远在线的中心数据服务器，但实际系统可有tracker、登录或协调服务，不能由“使用P2P”推断完全没有服务器。客户端/服务器也可以是某一次交互的角色，不是某台机器终身的职业。

假设浏览器要向服务器索取网页。浏览器把要发出的字节交给操作系统提供的接口，这个接口叫socket（套接字）；操作系统再通过传输协议发送。IP地址帮助找到目标主机，端口号帮助找到主机上的相应服务。这样，访问网页和收邮件就能在同一台电脑上同时进行。

对照课件时留意：32-bit地址指IPv4；第10张幻灯片的 `connect` 是客户端主动连接，附近的“accept incoming”注释与该代码不符。

**自查 / Check:** 同一IP上有Web和邮件服务，只给IP能否指出收消息的进程？ / Can an IP address alone distinguish a web process from a mail process on the same host?

### 答案 / Answer

不够，还要传输协议与端口等端点信息。 / No; transport protocol and port information are needed. 课堂HTTP 80、SMTP 25是相应传统服务示例，不是全部现代配置。

来源：Chapter 2完整课件，第3—4张幻灯片。

## 2. Service requirements | “可靠”不等于“准时”

---

应用协议规定消息类型、语法、语义和交互规则。开放标准便于互操作；专有协议由特定实体控制，并不意味着一定没有任何公开文档。选择传输服务前，先问应用需要什么：完整性、及时性、持续吞吐、安全性。

文件少一段不能算完成，实时通话晚到的音节可能已没有价值。TCP提供可靠、有序字节流，并有流控（保护接收端）与拥塞控制（适应网络）；它不承诺固定时限、最小吞吐，也不自动加密。UDP提供数据报接口，没有这些内建可靠性/顺序保证；应用可在其上实现所需机制。TCP可靠性也不是“网络永久断开仍能保证交付”，失败会暴露给应用。

用SSL称呼安全层，是旧术语；今天应区分TLS与旧SSL。教学要点是加密、完整性和端点认证需要额外机制，不能将“用TCP”当作已经安全。服务表里的码率与应用例名是课堂时代背景，核心是需求比较，不是当前产品参数清单。

**English:** Reliability, ordering, latency, throughput and security are different service properties. UDP's minimal service lets applications choose additional mechanisms, but does not remove responsibility for congestion and reliability design.

来源：Chapter 2完整课件，第12张幻灯片；第17张幻灯片。

## 3. Web and HTTP | 拿到HTML不等于拿齐整页

---

分base HTML和referenced objects：HTML告诉浏览器还需要哪些图片、脚本等，每个对象有URL。URL不只是主机名，还包含路径、可能的协议方案、端口等。本章和Tutorial使用传统HTTP/1.x over TCP模型；不能把“HTTP必然使用TCP”推广到HTTP/3，后者基于QUIC。RFC9114

浏览器是HTTP客户端，Web服务器应请求返回资源（Chapter 2完整课件，第20张幻灯片）。HTTP无状态是说核心请求语义不要求服务器记住先前请求，不等于TCP连接没有状态，也不等于网站不能存购物车；cookie等机制随后会补上会话关联。

### 连接可以复用，请求是否并行还要另问

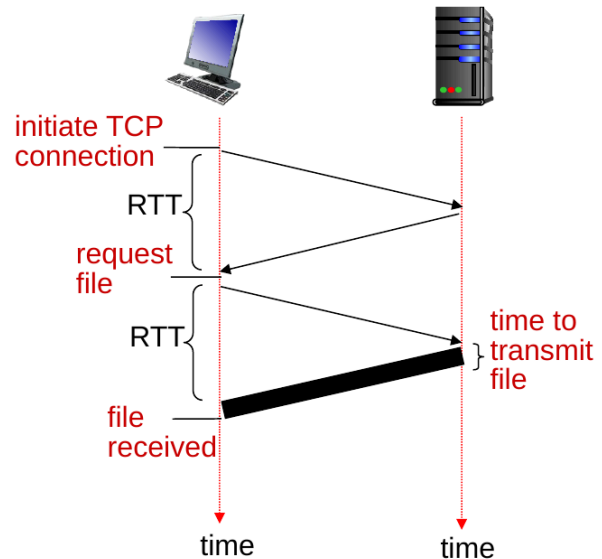
Non-persistent HTTP：每个对象新建TCP连接。简化为握手1RTT、请求到首个响应1RTT，再加对象发送时间；忽略DNS等其他部分时  $2RTT + L/R$ 。它不是包含TLS、慢启动、服务处理的全现实上网公式。

# Non-persistent HTTP: response time

**Round-trip time (RTT):** time for a small packet to travel from client to server and back

**HTTP response time:**

1. one RTT to **initiate TCP connection**
  2. one RTT for **HTTP request** and first few bytes of **HTTP response** to return
  3. **file transmission** time
- ❖ (Overall) non-persistent HTTP response time =  $2\text{RTT} + \text{file transmission time}$



Application Layer 2-26

Persistent HTTP: 复用连接，后续对象省去重新握手。**持久连接不等于流水线 (pipelining)**；无流水线时每次等上一请求响应后才发下一次。多个并行TCP连接又是第三个概念，可让不同对象同时等待，但占更多资源，吞吐也受共享瓶颈限制。

补充小例：base HTML加2个小对象，都在同服务器，忽略发送时间和DNS。串行非持久共  $3 \times 2 = 6\text{RTT}$ ；单个持久无流水线是1握手+3请求=4RTT；拿到HTML后一次发两个后续请求的理想流水线模型为3RTT。先问清协议/浏览器约定，才有唯一算式。原Chapter 2完整课件，第27张幻灯片画的是顺序复用，Tutorial3c据此读。

**变式 / Transfer:** 同一服务器上的HTML还引用4个很小的对象。初始无连接，先收到HTML，忽略DNS、发送、TLS、丢包和处理时延。串行非持久与单条无流水线持久连接各需多少RTT？ / A base HTML file references four small objects on the same server. Start without connections and fetch HTML first. Ignore DNS, transmission, TLS, loss and processing. Compare serial non-persistent HTTP with one non-pipelined persistent connection.

**答 / Answer:** 非持久有5次“握手+请求”，共10 RTT；持久只握手一次，加5次请求，共6 RTT。 / Ten RTTs versus six RTTs. 复用连接省的是握手，请求本身仍需往返。

**读报文：把每一行与问题对应**

请求行是method、request target、version；头字段包括Host、User-Agent、Accept等；CRLF分行，空行结束头部。GET通常索取表示；HEAD只请求相应响应头；POST提交内容供资源处理，不能仅凭方法名保证请求是否“安全”或是否一定有某种body。

# HTTP request message

- ❖ two types of HTTP messages: *request, response*
- ❖ **HTTP request message:**
  - ASCII (human-readable format)

The diagram shows an HTTP request message in ASCII format. Annotations on the left point to specific parts of the message:

- request line (GET, POST, HEAD commands):** Points to the first line: `GET /index.html HTTP/1.1\r\n`
- header lines:** Points to the subsequent lines: `Host: www-net.cs.umass.edu\r\n`, `User-Agent: Firefox/3.6.10\r\n`, `Accept: text/html,application/xhtml+xml\r\n`, `Accept-Language: en-us,en;q=0.5\r\n`, `Accept-Encoding: gzip,deflate\r\n`, `Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n`, `Keep-Alive: 115\r\n`, and `Connection: keep-alive\r\n`
- carriage return, line feed at start of line indicates end of header lines:** Points to the final `\r\n` at the end of the header block.
- carriage return character** and **line-feed character:** Arrows point to the `\r` and `\n` characters in the first line.

The full message is:

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

\* Check out the online interactive exercises for more examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive/](http://gaia.cs.umass.edu/kurose_ross/interactive/)

Application Layer 2-30

原HTTP/1.1文本适合观察ASCII结构；不要据此认为所有HTTP版本在传输中都相同明文文本布局。Tutorial3将用真实题干逐项判断，IP地址未出现时应明确说无法仅从这些头字段得到，不能从Host误认浏览器地址。

**English:** A page may require multiple object requests. Connection persistence, pipelining and parallel connections are separate choices; name the timing model before counting RTTs.

来源：Chapter 2完整课件，第19张幻灯片；第30—31张幻灯片。

## 4. Cookies and caches | 识别会话，与复用内容是两件事

课堂的Susan购物例子：网站生成标识，响应Set-Cookie交给浏览器，浏览器后续发Cookie，服务器用它找到后端会话/购物车。四部分是响应头、浏览器保存、服务端记录、后续请求头。Cookie不是自动等于用户名密码，也不保证一个ID就足以安全认证；它也带来跨请求关联与隐私问题。

课堂的cache对浏览器是服务器，对origin又是客户端。命中且可用时直接回应，未命中向源获取，减少响应时延与接入流量。需要考虑新鲜度、验证及不应共享的个人内容；不是“见到任意响应就永久缓存”。这和cookie保存会话身份作用不同。

**变式 / Transfer:** 一个代理向源服务器请求未命中的对象时，它是client还是server？对原浏览器呢？  
/ What roles does a proxy play toward the origin and toward the browser?

**答案 / Answer**

对源是client，对浏览器是server。 / It is a client to the origin and a server to the browser. 角色按具体交互定义。

来源：Chapter 2完整课件，第32—35张幻灯片；第36—37张幻灯片。

## 5. Electronic mail and SMTP | 把“送到邮箱”和“打开邮箱”拆开

Alice的邮件客户端（user agent）先把信交给发送邮件服务器，放入outgoing queue；服务器经SMTP投递到Bob的邮件服务器，进入mailbox；Bob再用客户端访问。发件/收件服务器可能在不同交互中分别担当client/server。

SMTP通过命令/响应完成greeting、邮件传送、结束。课堂Chapter 2完整课件，第42张幻灯片原对话顺序是 HELO → MAIL FROM → RCPT TO → DATA → 邮件内容 → 单独一行. → QUIT。只有服务器给出354后才发DATA内容；250代表该阶段成功，但不是“Bob已阅读”。同一持久TCP连接可投递多封邮件。

### Sample SMTP interaction

```

S: 220 hamburger.edu (server's host name)
C: HELO crepes.fr (client's host name)
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr> (beginning of message)
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup? (two-line message)
C: How about pickles?
C: . (end of message)
S: 250 Message accepted for delivery
C: QUIT (all messages have been sent)
S: 221 hamburger.edu closing connection
  
```

将HTTP称pull、SMTP称push，是典型课堂使用方式；HTTP也支持客户端上传，别把类比读成协议完全禁止别的方向。邮件内容头From/To/Subject与SMTP信封命令作用不同；头部与正文之间为空行。Chapter 2完整课件，第41/45张幻灯片的7-bit ASCII是基础历史模型，附件与国际化需编码/扩展，不能说现代电子邮件只能写英文。

**English:** SMTP transfers mail to a receiving server. Message headers are distinct from SMTP envelope commands, and server acceptance does not mean the recipient has read the message.

来源：Chapter 2完整课件，第39—43张幻灯片；第44张幻灯片。

## 6. POP3, IMAP and webmail | 下载之后要不要留在服务器？

邮件到达服务器后，收件人还要访问邮箱。POP3的三阶段是Authorization、Transaction、Update。客户端认证后用LIST看大小、RETR取内容、DELE标记删除，成功QUIT进入Update提交删除。**RETR本身不会自动删除**，DELE也不是立刻不可撤销地移除；在Transaction中RSET可清除删除标记，非正常断开不进入正常Update流程。RFC1939 §5-6

Download-and-delete与download-and-keep由客户端是否标记删除并成功结束会话等行为形成，不是“POP3只能删”。跨会话不维护IMAP那样的文件夹/已读同步状态，不等于服务器不保存邮箱或不能提供持久UIDL识别；客户端可记住已下载UID来避免重复下载。

IMAP把邮件和文件夹等状态留在服务器，支持多设备一致访问。Webmail中浏览器与网站通常使用HTTP(S)，而邮件服务器之间仍可SMTP；不要因为你在网页按发送，就断言浏览器直接向所有收件服务器开SMTP连接。完整两封邮件事务见Assignment1 P17〔本册 p.80〕。

来源：Chapter 2完整课件，第46—51张幻灯片。

## 7. DNS services and hierarchy | 通讯录不是只放在一台电脑上

课堂的Domain Name System既是分布式数据库，也是应用层协议。它解决主机名到地址、别名、邮件服务器定位与负载分布等问题。一个名字可以返回多个地址，同一台服务器也可服务多个名字。

以查询 `www.example.com` 为例，本机通常把问题交给一个递归解析器（recursive resolver，课件称 local DNS）。如果没有可用缓存，它先问根服务器（root），得知该向哪些.com顶级域服务器询问；再问顶级域（top-level domain，TLD）服务器，找到负责example.com的权威服务器（authoritative server）；最后向权威服务器取得所需记录。每一步可能只给出“下一站问谁”，这叫委派（delegation）。

root、TLD和权威服务器形成管理层次；本地解析器负责替你沿这条线查询及缓存结果。一个权威服务器负责的一片记录叫区域（zone）。域名树的一部分还可以交给其他服务器管理，因此zone的边界不一定与某个domain的全部下级名称重合。

把root写成“自己联系权威并取回最终映射”不代表通常迭代解析的根服务行为；按随后Chapter 2完整课件，第60—61张幻灯片的常见图，递归解析器接收referral后自己继续查询。RFC1034 §4.3 原页根服务器实例数量是当时快照，不作为当前数量。不要把根服务标识数、实例数、物理机器数混成同一

个概念。

来源：Chapter 2完整课件，第53—59张幻灯片；第57张幻灯片。

## 8. Recursive versus iterative DNS | 是“替我办完”，还是“告诉我下一站”

请求主机向本地resolver发递归请求：“请给最终答案或错误”。resolver向root、TLD、权威发送代请求：“若不知道，告诉我下一站”。收到最终答案再返还主机。Chapter 2完整课件，第62张幻灯片展示全递归作为另一种可能模式，不能默认每一层都替下一层递归。

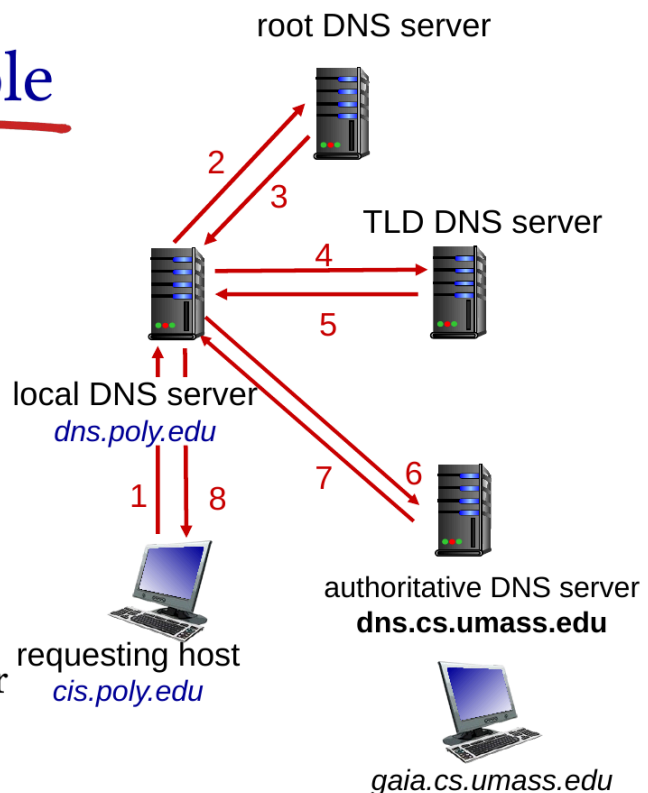
### DNS name resolution example

#### Iterative query:

- ❖ contacted server replies with name of server to contact
- ❖ “I don’t know this name, but ask this server”
- ❖ E.g., (2,3), (4,5), (6,7)

#### Typical query pattern:

- ❖ query from the requesting host to the local DNS server is **recursive**
- ❖ remaining queries are **iterative**.



Application Layer 2-61

看图先跟数字1—8，不要把每根箭头当一个RTT：一次往返有去和回。无缓存、顺序访问root/TLD/权威时，若host↔local为RTT<sub>L</sub>，每个local↔远端为RTT<sub>r</sub>，总RTT<sub>L</sub>+3RTT<sub>r</sub>。命中local缓存、无需再验证且忽略处理时，只RTT<sub>L</sub>。原图也用于Tutorial3。若RTT<sub>L</sub>=2 ms、RTT<sub>r</sub>=10 ms，无缓存时为2+3×10=32 ms；本地命中时为2 ms。

**独立题 / Check：** 仍按图中顺序，忽略处理与发送时延。RTT<sub>L</sub>改为3 ms、RTT<sub>r</sub>为15 ms，无缓存与本地有效缓存命中分别多久？ / Under the same sequential DNS flow and negligible processing/serialization, use RTT<sub>L</sub>=3 ms and RTT<sub>r</sub>=15 ms. Find the uncached and valid local-cache-hit times.

**答 / Answer：** 3+3×15=48 ms与3 ms。 / 48 ms and 3 ms.

缓存有TTL (time to live, 有效存活时间), 回答可以非权威但仍来自缓存的权威资料链。过期需要重新查询; 记录变化不会自动让所有旧缓存同时消失。DNS TTL是缓存时间概念, 和IP TTL的跳数控制不是同一字段。

## 9. DNS records and messages | 名字、值、类型、有效时间要一起读

| 类型    | name→value     | 用途                        |
|-------|----------------|---------------------------|
| A     | 主机名→IPv4地址     | 地址映射; IPv6另用AAAA (补充区分)   |
| NS    | 区域/域名→权威服务器主机名 | 委派; 不是直接给全部终端IP           |
| CNAME | 别名→规范名         | 后续仍需得到地址记录                |
| MX    | 邮件域→邮件交换服务器名   | 实际还含优先级字段; 不是发往Web服务器的A记录 |

课堂的查询/响应共用结构: ID匹配事务; flags说明请求/回答、是否希望递归、是否支持递归、是否权威等; counts说明后续questions、answers、authority、additional的数量。附加区可以携带相关地址, 减少后续查询, 但不是任意附加记录都自动可信。

新公司例子: 向注册商登记域和权威服务器, 父区设置委派NS, 必要时有glue地址; 在自己权威区添加www的地址记录。课件同页混用networkuptopia/networkutopia且www与apex不一致, 是拼写/名称示意问题; 真正配置时它们是不同的DNS名字, 必须统一, 不能指望解析器猜出来。

课堂的DDoS、缓存投毒与反射放大用来理解可靠性/信任边界。原“Not successful to date”描述的是旧课件编写时的情况。缓存降低某些上游访问压力, 但不能证明DNS不可能遭攻击。

**English:** Recursive resolution delegates completion; iterative resolution returns referrals. DNS records have distinct types and TTLs; a non-authoritative cached answer is not automatically an incorrect answer.

来源: Chapter 2完整课件, 第66–67张幻灯片; 第68张幻灯片; 第69张幻灯片。

## 10. P2P file distribution | 新加入的人也可以带来上传能力

给理想模型: 服务器有F bit文件, 需分发到N个peer, 服务器上传 $u_s$  bit/s, peer i上传 $u_i$ 、下载 $d_i$ , 核心网络不额外限速,  $d_{min} = \min_i d_i$ 。

Client-server的服务器共需上传NF bit, 最慢客户端下载F bit:

$$D_{cs} \geq \max \left( \frac{NF}{u_s}, \frac{F}{d_{min}} \right). \quad (2.1)$$

服务器不必物理“顺序开完一个连接才开下一个”,  $NF/u_s$ 来自总上传工作量, 哪怕并行也绕不过这个容量下界。

P2P中的节点可以互助上传，但最初至少要有一份文件从源头出去；每个节点仍须下载F，所有节点总共需NF的接收量：

$$D_{p2p} \geq \max \left( \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_i u_i} \right). \quad (2.2)$$

图和公式是理想容量下界，实际调度、块稀缺、离线、协议开销会增加耗时。Chapter 2完整课件，第75张幻灯片“always less”应读为在理想能力比较中不劣且有时更好，可能相等，不能保证任意实际P2P更快。

**回到Chapter 2完整课件，第75张幻灯片原例：** 每个peer上传u，F/u=1小时，服务器u<sub>s</sub>=10u，所有下载速率至少u<sub>s</sub>。于是CS下界为N/10小时；P2P下界为max(0.1, N/(10+N))小时。N=10时为1小时对0.5小时，N=100时为10小时对约0.909小时；N=1时两者均0.1小时。用户增加也带来上传资源，这解释了原图的扩展性趋势。

**完整补算：** F=100Mbit、N=4、u<sub>s</sub>=10Mbps、每个d<sub>i</sub>=5Mbps、u<sub>i</sub>=2Mbps。CS下界max(40,20)=40s；P2P下界max(10,20,400/18)=22.22s。分母总上传18Mbps，不把下载速率也当上传资源加进去。

**变式 / Transfer:** 同条件但各peer上传0，P2P下界？ / Set every peer upload rate to zero; keep all other parameters.

**答案 / Answer**

max(10,20,40)=40s，与CS相同。 / 40 s, equal to the client-server bound. 这是“P2P一定严格更快”的反例。

来源：Chapter 2完整课件，第71—75张幻灯片。

## 11. BitTorrent | 先找邻居，再交换稀缺块

一个torrent是交换同一文件分块的节点集合，tracker帮助发现参与者，不必亲自发送全部文件。peer加入时无块，逐渐下载并向别人上传；成员离开/加入称churn。

请求块采用rarest-first，优先扩散目前邻居中少见的块，降低最后只缺同一块的风险。发送方面，课堂用“偏向近期给自己较高上传速率的四个伙伴，每10秒重评、每30秒随机尝试一个新伙伴”的tit-for-tat/optimistic unchoking例子。这里的4/10/30属于课件所述算法版本，客户端实现可能采用不同参数。随机试新伙伴避免只锁在既有关系里。

原图中的 256Kb chunks 表示一个分块大小的示例（Chapter 2完整课件，第76张幻灯片）；实际分块长度随torrent选择，b/B也需辨别；理解单位见基础页，不要求把这个旧示例当标准固定块长。

来源：Chapter 2完整课件，第76—80张幻灯片。

## 12. Video, DASH and CDN | 边下边播，需要的不只是平均带宽

解释视频由连续图像组成；空间冗余可压缩同一帧重复信息，时间冗余利用帧间变化。不同编码码率对应不同质量/数据量，但码率不是用户网络吞吐。“4Mbps视频”指播放每秒需要约4Mbit编码数据，而不是网络必然提供4Mbps。

从单个HTTP文件到Dynamic Adaptive Streaming over HTTP (DASH)：服务器把视频切成时间块，同一块有多个码率版本，manifest给出访问位置；客户端估计吞吐与缓冲状况，决定何时请求、用哪档、从哪里取。不能只用刚测到的瞬时峰值去选最高清，否则可能很快耗尽缓冲。

补算：4秒片段、编码2Mbps，块大小8Mbit；可用吞吐4Mbps时下载2秒，若同时正常播放，缓冲时长净增加约2秒。若只有1Mbps则下载8秒，需要足够初始缓冲，否则会停顿。这里忽略请求/变码率等开销，按固定编码率计算；实际块大小可以变化。

**自己算 / Try：** 一个6秒视频块按3 Mbps编码，可用吞吐6 Mbps，初始缓冲充足且边下边播。求大小、下载时间和缓冲时长净增量。 / A six-second chunk is encoded at 3 Mbps and downloaded at 6 Mbps. With enough initial buffer and simultaneous playback, find its size, download time and net buffer gain.

**答 / Answer：** 18 Mbit、3秒、净增6-3=3秒，忽略请求和解码开销。 / 18 Mbit, 3 seconds to download, and 3 seconds of net buffer gain, ignoring request and decoding overhead.

内容分发网络（content delivery network, CDN）把副本放在多个地理/网络位置，降低单站压力与长路径问题。Enter deep靠近接入网络、地点多；bring home集中于较少大型IXP附近集群。选择“近”往往看网络路径/负载，不只地图公里数。DASH控制表示与请求，CDN提供分发位置，二者可配合但不是同义词。原用户数、流量占比和部署地点数均为原页时期的案例。

**English:** Encoding rate describes media demand, while throughput describes delivery capacity. DASH adapts chunk requests; CDNs distribute replicas across serving locations.

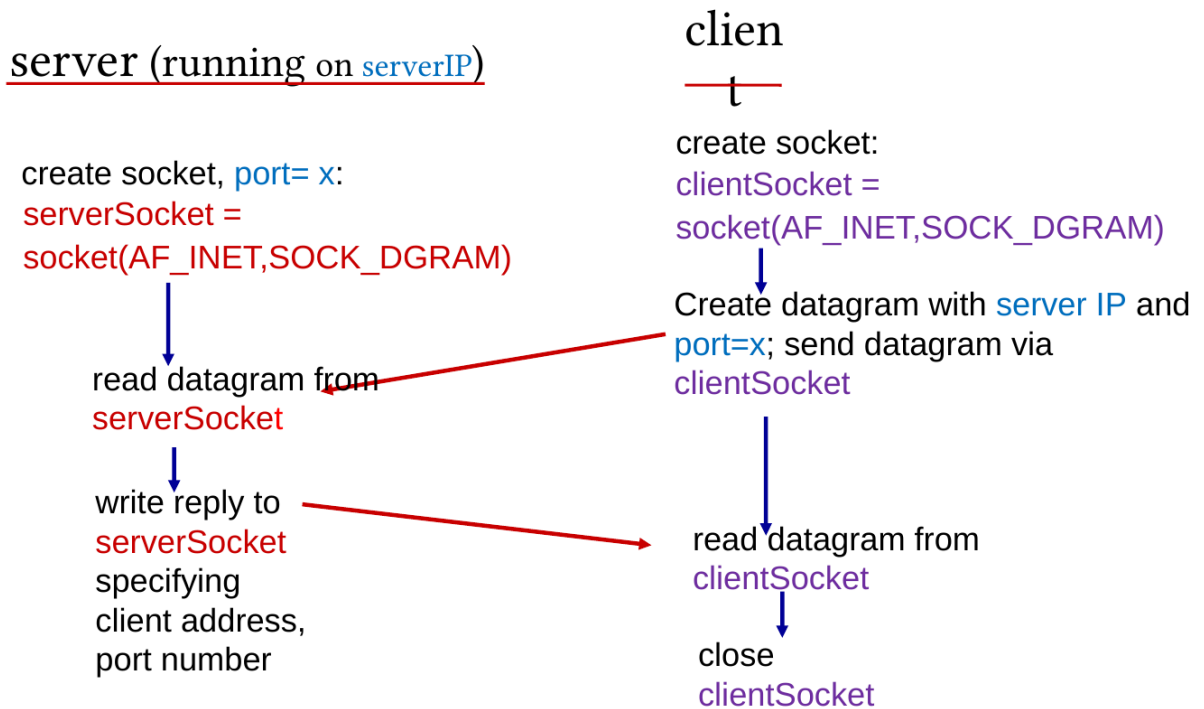
来源：Chapter 2完整课件，第82—84张幻灯片；第85—88张幻灯片；第89—92张幻灯片。

## 13. Socket programming | 把课堂的大写转换例子真正跑通

课堂的原任务：client读一句文字 → server转大写 → 回给client。当前原代码含Python2 raw\_input/print写法，以及from socket import \*与socket.AF\_INET混用。配套socket\_demo.py 使用Python3、模块导入、显式UTF-8编码和超时，只绑定本机127.0.0.1。

UDP路径：socket(AF\_INET, SOCK\_DGRAM) → 服务端bind → client.sendto(data, address) → server.recvfrom得到bytes和client地址 → sendto回复 → client.recvfrom。它不做连接握手；收到回包不证明所有未来数据报都可靠。

# Client/server socket interaction: UDP



Application 2-97

TCP路径：server创建监听socket、bind、listen；client创建socket再connect；server.accept返回新的连接socket；双方用该连接收发；结束关闭连接socket，监听socket可继续接新连接。Chapter 2完整课件，第102张幻灯片用源端口帮助区分连接，完整连接识别还涉及双方地址/端口与协议，不能只看一个源端口。

```

# 在本目录运行。两个终端；服务端只处理一次请求便退出。
python labs/socket_demo.py --transport udp --role server --port 12000
python labs/socket_demo.py --transport udp --role client --port 12000 --message 'hello network'
# TCP只需把两边的udp同时改成tcp。
python labs/socket_demo.py --self-test
  
```

TCP没有消息边界：本例规定每条文本以换行结束，sendall发送全部bytes，接收循环直到换行，不能假设一次recv(1024)一定收到整句。UDP保留报文边界，但要保证例题报文在接收缓冲范围内。

本机通信示范的执行记录见 socket-self-test.json，包括UDP、TCP和超过一次读取大小的TCP消息。它验证本机接口与消息分帧，不测量跨互联网可靠性或时延。

**独立任务 / Try:** 将客户端文字改成course practice, 两种协议各运行一次; 再解释TCP为什么有监听与连接两个socket。 / Run both transports with this message, then explain the two server-side TCP sockets.

#### 预期与解释 / Expected result

输出COURSE PRACTICE。监听socket接受新连接, 连接socket处理当前客户端的数据, 职责不同。  
/ The reply is COURSE PRACTICE; the listening socket accepts connections while the connected socket exchanges this client's data.

来源: Chapter 2完整课件, 第94–105张幻灯片。

## 14. Back to the course | 用一条请求链把本章连起来

打开网页时: DNS得到服务地址 → socket取得所选传输服务 → HTTP表达所需资源 → 响应中的cookie可关联状态、cache可复用内容。邮件用SMTP投递、POP3/IMAP/HTTP访问; 大文件与视频则进一步考虑peer上传能力、分块、编码与分发位置。共同的问题始终是: **双方是谁、消息怎么写、何时发送、需要哪些服务、状态由谁保存。**

接着做Tutorial3〔本册 p.58〕的报文/DNS/HTTP题和Assignment1〔本册 p.80〕的POP3与Zoom分析。首轮可选 N100 (连接复用)、N118 (Webmail路径)、N124 (DNS查询模式)、N233 (HTTP时序), 在Markji按卡号定位。相关微课NET05继续可用。

英文自测: Explain why persistent HTTP does not necessarily imply pipelining. / 为什么持久HTTP不等于流水线? 关键答: 复用的是连接, 流水线另决定是否在上一响应完成前发后续请求。 / Persistence reuses a connection; pipelining additionally overlaps outstanding requests.

## 课堂 Q&A: 按原题判断并说明条件

Q&A\_cha2学生版第2–7张幻灯片有六题, 未附教师解答。下面保留完整选项, 答案根据本章机制整理。

### 1. Transport service | 为应用选择传输服务

**EN:** Which services should use UDP? A) File transfer. B) Video streaming. C) Email. D) Online games.

**中文:** 哪些服务适合使用UDP? A文件传输; B视频流; C邮件; D在线游戏。

**答 / Answer:** 课堂对时延敏感服务的简化模型选B、D; 实际协议还要结合应用设计, 例如视频也可能使用HTTP/TCP。 / B and D under the classroom model of latency-sensitive services; real applications may choose other transports, including HTTP/TCP for video.

## 2. HTTP timing | 从HTML到两张图片

**EN:** A browser obtains an HTML page and two very small JPEG images from the same server using non-persistent HTTP. What is the total response time? A) 2 RTTs. B) 3 RTTs. C) 6 RTTs. D) 8 RTTs.

**中文:** 浏览器从同一服务器获取HTML和两张很小的JPEG, 采用非持久HTTP, 总响应时间是A 2、B 3、C 6还是D 8个RTT?

**答 / Answer:** 按课件顺序获取、忽略DNS与对象传输时延的约定, 三个对象各需2RTT, 共6RTT, 选C。原题未明确并行性; 并行连接时答案要重算。 / C: six RTTs for serial requests, ignoring DNS and object transmission time. Parallel connections would change the timing.

## 3. Mail addressing | 目标主机与服务端口

**EN:** What addressing information does a client process need to communicate with the mail server at abc.com? A) Its IP address, usually obtained through DNS. B) Its IP address, which the client usually already knows. C) Its port number, usually obtained through DNS. D) Port number 25.

**中文:** 客户进程联系abc.com的邮件服务器, 需要哪些地址信息? A通常经DNS得到IP; B客户端通常已知道IP; C通常经DNS得到端口; D端口25。

**答 / Answer:** 按本题SMTP投递情境选A、D: IP通过DNS确定, 端口25来自协议与配置。它不代表所有邮件客户端操作都用25; 提交、收信另有服务。 / A and D for SMTP delivery: resolve the server's IP address and use port 25. Submission and mailbox access are different services.

## 4. DNS records | 一个名字怎样对应多台服务器

**EN:** Which DNS record type can enable load balancing in this question? A) A. B) NS. C) CNAME. D) MX.

**中文:** 哪类DNS记录可实现本题所指的负载分配? A A记录; B NS; C CNAME; D MX。

**答 / Answer:** 选A。同一名字可返回多个IPv4地址, 由客户端或返回顺序影响选择; 它本身不保证各服务器负载精确相同。 / A: multiple addresses can be returned for one name, although this alone does not ensure perfectly balanced load.

## 5. BitTorrent | 找到谁、向谁下载

**EN:** Which statement about BitTorrent is true? A) It uses a client-server architecture. B) A user needs the IP addresses of peers. C) Missing chunks with the largest size are requested first. D) A user always communicates with the same peers throughout the download.

**中文:** 哪句正确? A采用client-server架构; B需要peer的IP地址; C优先请求最大的缺失块; D下载全程始终联系同一批peer。

**答 / Answer:** 选B。BitTorrent的数据分发是P2P，常用稀缺块优先，连接的peer可以变化；发现peer可以借助tracker等机制。 / B. Data distribution is peer-to-peer, chunk selection commonly uses rarest-first, and peers may change during a download.

## 6. Socket services | 接口提供了什么

**EN:** Which statements are true? A) TCP guarantees that bytes arrive in order. B) UDP performs a handshake before sending data. C) With UDP, the sender explicitly supplies the destination IP address and port for each packet. D) Socket programming selects the network-layer protocol.

**中文:** 哪些说法正确? A TCP提供可靠且有序的字节传输; B UDP发数据前握手; C UDP发送时逐包给目的IP和端口; D socket编程选择网络层协议。

**答 / Answer:** 课件基本服务与sendto接口模型选A、C。TCP向应用交付有序字节，但连接失败时不能保证最终成功; C描述本章未连接UDP的发送接口。D不能理解为应用通过本章接口任意指定沿途网络协议。 / A and C in the taught service/API model. TCP delivers ordered bytes but can report connection failure; C describes the unconnected UDP sendto interface used here.

来源: Chapter 2课堂Q&A, 第2—7张幻灯片, 英文按原题整理。

**迁移 / Transfer:** 把Chapter 2课堂Q&A, 第3张幻灯片改为HTML后可并行取两张图, 非持久连接且理想无竞争, 总需几RTT? / After receiving the HTML, fetch its two small images through parallel non-persistent connections. Ignore DNS and transmission time, assume no contention, and express the total in RTTs.

### 答案 / Answer

HTML两RTT, 图片同一批两RTT, 共4RTT; 这是补充变式, 不替换原题顺序获取约定。 / Four RTTs under the stated idealized parallel model.

Tutorial4 [本册 p.65] 继续练本章P2P/DASH; Tutorial5 [本册 p.69] 属于下一章传输层。Q&A页面并入本章覆盖索引: Chapter 2课堂Q&A, 第1张幻灯片为封面, Chapter 2课堂Q&A, 第2—7张幻灯片对应以上六题。

## 15. Source coverage and cautions | 每一页有去处

| 完整PPT页 | 对应正文 | 核对重点                             |
|--------|------|----------------------------------|
| 1—3    | 开篇   | 来源、目标、应用例                        |
| 4—12   | §1—2 | 架构、进程、socket、寻址、语法语义             |
| 13—17  | §2   | 服务需求、TCP/UDP、旧SSL表述              |
| 18—22  | §3   | 目录、对象、HTTP/TCP和无状态               |
| 23—29  | §3   | 非持久/持久/并行、RTT约定                  |
| 30—31  | §3   | 完整请求与字段                          |
| 32—37  | §4   | cookie四部件、proxy/cache            |
| 38—45  | §5   | 目录、邮件结构、SMTP原事务、信封/内容            |
| 46—51  | §6   | POP3三阶段、keep/delete、IMAP、webmail |
| 52—59  | §7   | 目录、DNS服务、层级与local resolver       |

| 完整PPT页  | 对应正文   | 核对重点                                     |
|---------|--------|------------------------------------------|
| 60—63   | §8     | 递归/迭代的两种图、TTL/cache                      |
| 64—69   | §9     | RR、消息、注册/委派、攻击概念                         |
| 70—75   | §10    | 目录、P2P架构、分发下界与原例                         |
| 76—80   | §11    | tracker、块、churn、rarest-first、tit-for-tat |
| 81—88   | §12    | 目录、视频编码、HTTP流媒体、DASH                     |
| 89—92   | §12    | CDN两种布局、ISP/IXP回接                        |
| 93—105  | §13    | 目录、任务、UDP/TCP流程与全部原代码含义                  |
| 106—107 | §14—15 | 协议共同主题、控制/数据、状态、可靠性、边缘复杂性                |

part1对应完整1—22，part2对应23—65，part3对应66—107；内容核对忽略明确的页脚重编号。课件中的旧RFC、客户端与数字按原例保留；协议版本差异在正文就近说明。FTP独立正文和后续未下载章节列为待补材料。

## Chapter 3 · Transport Layer（当前 part 1–2）

### 数据到了电脑，还要交对进程、补齐缺口

课程目录 · 上一章：应用层〔本册 p.19〕 · Tutorial 5〔本册 p.69〕

同一台电脑可以同时开浏览器、邮件和游戏。网络把数据送到这台电脑之后，还要知道交给哪一个程序。即使地址写对，途中也可能损坏、丢失或重复；收到半段文件不能直接说“差不多齐了”。这一章从进程寻址开始，逐步增加可靠传输需要的机制。

依据当前part1的44页和part2的26页。两份课件的幻灯片序号各自从1开始。目前整理至TCP的RTT与超时；流控、连接管理及拥塞控制待后续材料补充。

**基础补课（可跳过）：** bit、byte与速率〔基础册 p.37〕、二进制〔基础册 p.38〕、时间轴〔基础册 p.40〕。阅读时留意“第几个包”和“第几个字节”的区别，§9会具体比较。

| 主题           | 优先级     | 难度与卡点       | 掌握要求及依据                                                    |
|--------------|---------|-------------|------------------------------------------------------------|
| 传输服务、复用/解复用  | 核心必会    | 中，主机/进程与端点  | 读UDP/TCP标识与回程端口；Chapter 3 part 1，第4–15张幻灯片、Tutorial 5 Q1–2 |
| UDP与校验和      | 核心必会    | 中，回卷进位      | 手算、解释检测边界；Chapter 3 part 1，第17–20张幻灯片、Tutorial 5 Q3        |
| rdt1.0–3.0   | 核心必会    | 高，双方状态与坏ACK | 逐事件解释、画乱序反例；Chapter 3 part 1，第22–44张幻灯片、Tutorial 5 Q4      |
| 停等与流水线       | 核心必会    | 中，忙碌时间/周期   | 计算利用率与吞吐；Chapter 3 part 2，第1–6张幻灯片                         |
| GBN与SR       | 核心必会    | 高，ACK与窗口    | 画一次丢包后的行为；Chapter 3 part 2，第7–15张幻灯片                       |
| TCP序号与RTO    | 核心必会    | 中，字节与估计量    | 算下一序号、累计ACK、超时；Chapter 3 part 2，第17–26张幻灯片                 |
| 完整握手/流控/拥塞控制 | 后续待当前材料 | 不以“尚缺”推断不考  | 暂留历史预习，不补成教师已讲                                             |

#### 本页目录 / Contents

- Chapter 3 · Transport Layer（当前 part 1–2）〔本册 p.35〕
  - 数据到了电脑，还要交对进程、补齐缺口〔本册 p.35〕
  - 1. Transport service | IP找到主机，传输层找到应用端点〔本册 p.36〕
  - 2. Multiplexing and demultiplexing | 同一栋楼有许多门〔本册 p.36〕
  - 3. UDP | 少做承诺，也少保存状态〔本册 p.37〕
  - 4. Internet checksum | 算得对不等于能发现一切错误〔本册 p.37〕
  - 5. rdt1.0到rdt2.2 | 每多一种故障，就多一项必要机制〔本册 p.38〕

- 6. rdt3.0 | 坏消息会来，丢掉的消息可能一直不来〔本册 p.38〕
- 7. Stop-and-wait performance | 忙了8微秒，等了30毫秒〔本册 p.39〕
- 8. GBN与Selective Repeat | 后面的包到了，要不要先收好？〔本册 p.40〕
- 9. TCP序号 | 从“第几个包”切换到“下一个字节”〔本册 p.41〕
- 10. RTT与重传超时 | 等多久才有理由怀疑包丢了？〔本册 p.42〕
- 11. 回到老师的题目〔本册 p.43〕
- 来源覆盖索引〔本册 p.43〕

## 1. Transport service | IP找到主机，传输层找到应用端点

先区分网络层的主机间逻辑通信与传输层的进程间逻辑通信。以发送一段应用数据为例：应用层称它为message（消息）；传输层封装后常称segment（报文段）；再封装进网络层的datagram（数据报），通过链路层frame（帧）送到下一站。各层名称强调不同的封装，日常说packet时则需要看上下文。

TCP提供连接上的可靠有序字节流以及流控、拥塞控制；UDP提供较简单的数据报服务。可靠不表示网络永久中断也能永远交付，不表示有固定到达期限，也不等于加密。

**自测 / Check：**一段数据已经到达正确IP地址，是否已确定交给哪一个浏览器连接？ / Does arrival at the correct host IP identify a specific browser connection?

### 答案 / Answer

没有。还需传输协议与端口等信息；TCP连接区分还涉及双方IP和端口。 / No. Transport endpoint information is still needed; established TCP connections use both address-port pairs.

来源：Chapter 3 part 1，第4—7张幻灯片。

## 2. Multiplexing and demultiplexing | 同一栋楼有许多门

发送端复用，把多个socket的数据交给网络；接收端解复用，看首部把数据送到相应socket。端口是传输端点标识的一部分，要与IP地址和协议一起理解。

先看一个普通、未连接的UDP接收socket：目的IP与目的端口标识接收socket，不同来源的数据报可以到同一个UDP socket。Chapter 3 part 1，第13张幻灯片的已建立TCP连接按四元组区分：源IP、源端口、目的IP、目的端口。因此服务器端同一个80端口可同时服务多个连接，只要四元组不同。

**跟做 / Worked：** C:7532→B:80、C:26145→B:80、A:26145→B:80是三条不同TCP连接。回程分别 B:80→C:7532、B:80→C:26145、B:80→A:26145。后两者客户端端口相同，客户端IP仍区分连接。 / Reverse both address-port pairs for the return direction.

**实现补注：** 这里比较的是课堂的普通UDP接收与已建立TCP连接。监听socket、已接受连接及connected UDP的接口细节，可在实际编程时进一步区分。

**变式 / Transfer:** 两客户端在不同IP上都使用端口50000, 连接同一个服务器443端口, 是否冲突?  
/ Two clients with different IP addresses both use source port 50000 to connect to the same server IP and port 443. Do the established TCP connections conflict?

**答案 / Answer**

四元组不同, 不因客户端端口数值相同而冲突。 / No: their source IP addresses distinguish the four-tuples.

来源: Chapter 3 part 1, 第11张幻灯片。

### 3. UDP | 少做承诺, 也少保存状态

UDP不先进行TCP式握手, 也不内建重传、排序和拥塞控制。应用可以在其上增加所需功能。需要及时性的应用可能愿意舍弃过时数据, 但“视频”这个标签不能唯一确定它使用UDP, HTTP自适应视频就是不同情境。

UDP首部四个16bit字段为source port、destination port、length、checksum, 合计8byte。length包含UDP首部与载荷, 不包含外层IP首部。100byte数据的UDP length是108byte。

**English takeaway:** UDP preserves datagram boundaries but does not provide TCP's built-in reliable ordered byte-stream service. Its length field counts the UDP header and payload.

来源: Chapter 3 part 1, 第17—18张幻灯片。

### 4. Internet checksum | 算得对不等于能发现一切错误

校验和采用16bit反码加法。先把校验字段置0, 把保护的数据组成定长字, 逐字相加; 超出16bit的进位卷回最低位, 再按位取反。接收端把含checksum的字也相加, 正确结果应为全1。

**跟做 / Worked:**  $0xFFFF + 0x0001 = 0x10000$ 。保留低16位0, 最高进位1卷回, 得到1; 取反得到  $0xFFFE$ 。接收端相加卷回后为  $0xFFFF$ 。普通“溢出直接丢掉”会得到另一个错误答案。

Tutorial5 [本册 p.71] 改用8bit以便手算, 这是题目简化, 不能把真实UDP首部缩成8bit。单比特翻转可被此校验发现; 两个适当位置的变化可能相互抵消。检验通过不证明数据来源可信, 也不等于密码学完整性认证。

**自测 / Check:** 8bit简化模型中,  $11111111 + 00000001$  的反码校验和是多少? / In a toy 8-bit model, compute the one's-complement checksum for words 11111111 and 00000001, including end-around carry.

**答案 / Answer**

回卷后00000001, 再取反11111110。 / End-around carry yields 00000001, then complement to 11111110.

来源: Chapter 3 part 1, 第19—20张幻灯片。

## 5. rdt1.0到rdt2.2 | 每多一种故障，就多一项必要机制

先看发送和接收各自负责的动作：应用通过rdt\_send把数据交给发送协议；协议调用底层不可靠通道；接收方在合适时机deliver\_data。有限状态机（FSM）记录“现在等什么”，箭头上方是触发事件/条件，下方是执行动作。不能把“没有动作”的转移理解为“这个事件不存在”。

rdt1.0假设底层不损坏、不丢失。发送就交下去，接收就交上来，不需要凭空增加ACK。随着信道假设变化，机制才有理由出现。

rdt2.0加入比特损坏：checksum检测，ACK表示收到正确数据，NAK表示重发请求。但ACK/NAK本身也可能损坏。此时发送者不知道数据到底是否已交给应用；直接重发可能导致同一条数据交付两次。

rdt2.1给数据加0/1序号。接收方记录期望编号，重复包仍可回复反馈，但不能重复交付。双方状态增加，是因为必须记住这一轮在等哪个序号；“看到包”与“向上交付新数据”是两件事。

rdt2.2去掉NAK，用带编号的ACK表达接收进度：遇到损坏/非期望包，重发最近正确包的ACK。发送方据ACK的编号判断是否完成当前发送。这不等于接收方对坏数据作成功确认。

**English takeaway:** Checksums detect certain damage, acknowledgments report receiver state, and sequence numbers distinguish new data from retransmissions. Each mechanism addresses a different ambiguity.

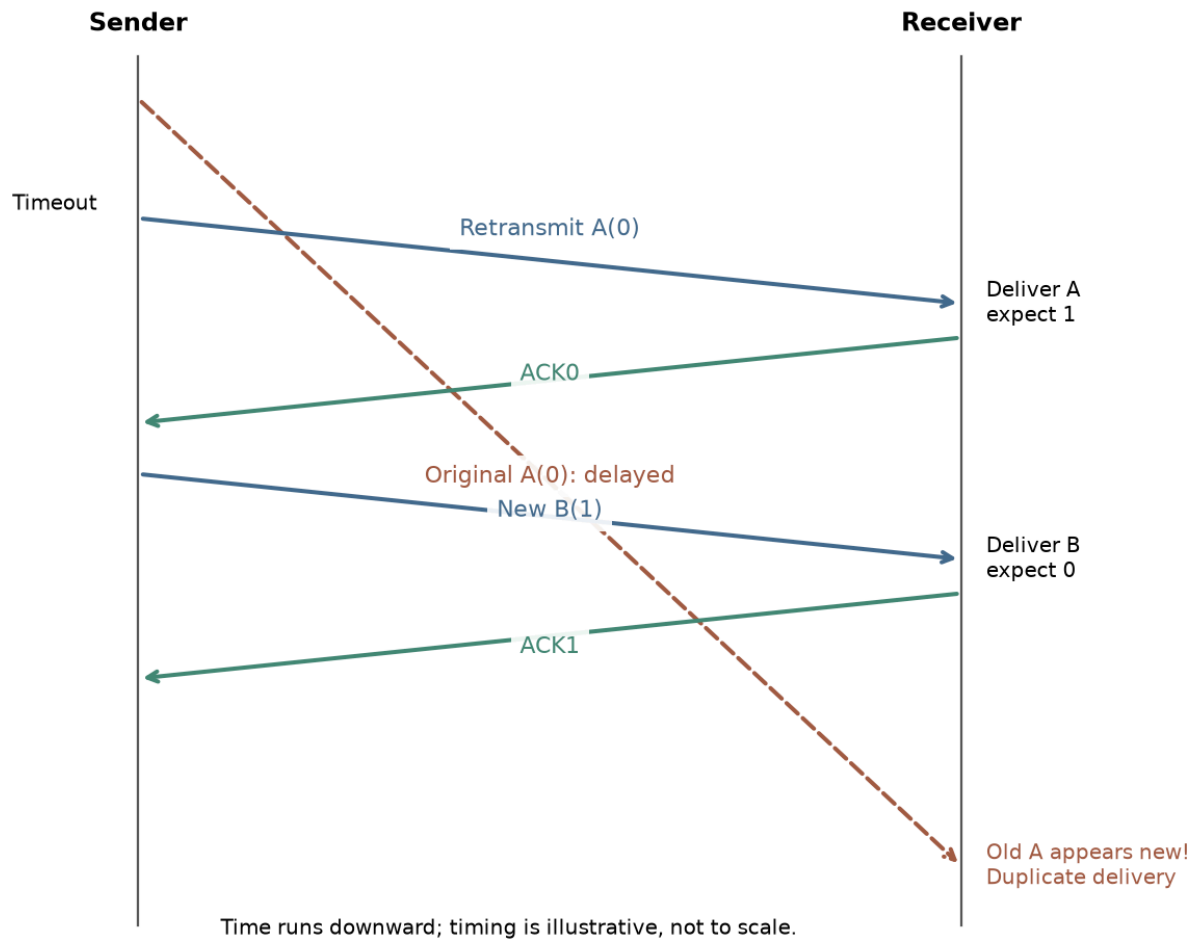
来源：Chapter 3 part 1，第22—27张幻灯片。

## 6. rdt3.0 | 坏消息会来，丢掉的消息可能一直不来

现在让模型再面对一种故障：数据或ACK可能丢失。没有反馈时不能永远等，因此发送方启动timer，超时重发当前包。收到当前包的正确ACK后停timer并进入下一编号；损坏或错误编号的ACK不能错误推进。

| 发生什么    | 发送者看见什么  | 必要行为              |
|---------|----------|-------------------|
| 数据包丢失   | 没有对应ACK  | 超时重传              |
| ACK丢失   | 也没有对应ACK | 超时重传；接收端辨重复，不重复交付 |
| ACK只是很慢 | 可能提前超时   | 正确处理重复包和旧ACK      |

一次只允许一个未确认包在途，称stop-and-wait（停等）。例如数据0已交付、ACK0丢了：发送者超时重发0；接收者此时期待1，认出它是重复包，只补发ACK0，不再次交付。这把定时器、序号和ACK三项连成了一次恢复过程。0/1足以在原可靠性模型中区分相邻轮次，但若旧副本可任意延迟到编号重用之后，接收方可能把它当新包。这个边界正是Tutorial5 Q4要求画图说明的，不能把“序号只有两种就够”背成无条件结论。



图为补充推导，时间向下。原0包被延迟，其重传先到；随后1包也完成。接收方又等0时，最早那份旧0才到。内容已经过期，但编号恰好符合期望，因此可能被重复交付。

来源：Chapter 3 part 1，第40—44张幻灯片。

## 7. Stop-and-wait performance | 忙了8微秒，等了30毫秒

看课堂的停等传输例子： $R=1\text{Gbps}$ ， $L=8000\text{bit}$ ，单程传播 $15\text{ms}$ ， $RTT=30\text{ms}$ ，忽略ACK传输和处理。发送时间 $L/R = 8\mu\text{s} = 0.008\text{ms}$ 。一轮从开始发包到ACK返回，耗 $RTT + L/R$ 。

$$U = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} \approx 0.000266596. \quad (3.1)$$

发送端忙碌比例约0.02666%，有效吞吐 $RU \approx 266596\text{bit/s}$ ，即0.2666Mbps。链路名义上1Gbps，却大部分时间在等。这是利用率，不是丢包率。

流水线允许ACK回来前连续发N个包。理想无损模型的利用率为

$$U_N = \min \left( 1, \frac{NL/R}{RTT + L/R} \right). \quad (3.2)$$

窗口越大，可填补等待；但不能超过100%。本例达到满利用需 $N \geq 3751$ 。这是忽略其他瓶颈与ACK成本的模型结果，不是给真实TCP直接配置3751就必然满速。

**迁移 / Transfer:** 同样RTT与包长, R改为100Mbps, 停等吞吐是否变为原来的十分之一? /  
Recompute after reducing R to 100 Mbps.

**答案 / Answer**

$L/R = 0.08ms$ ,  $U = 0.08/30.08$ , 吞吐约0.2660Mbps, 仍主要受等待限制。 / About 0.2660 Mbps; stop-and-wait remains dominated by the RTT.

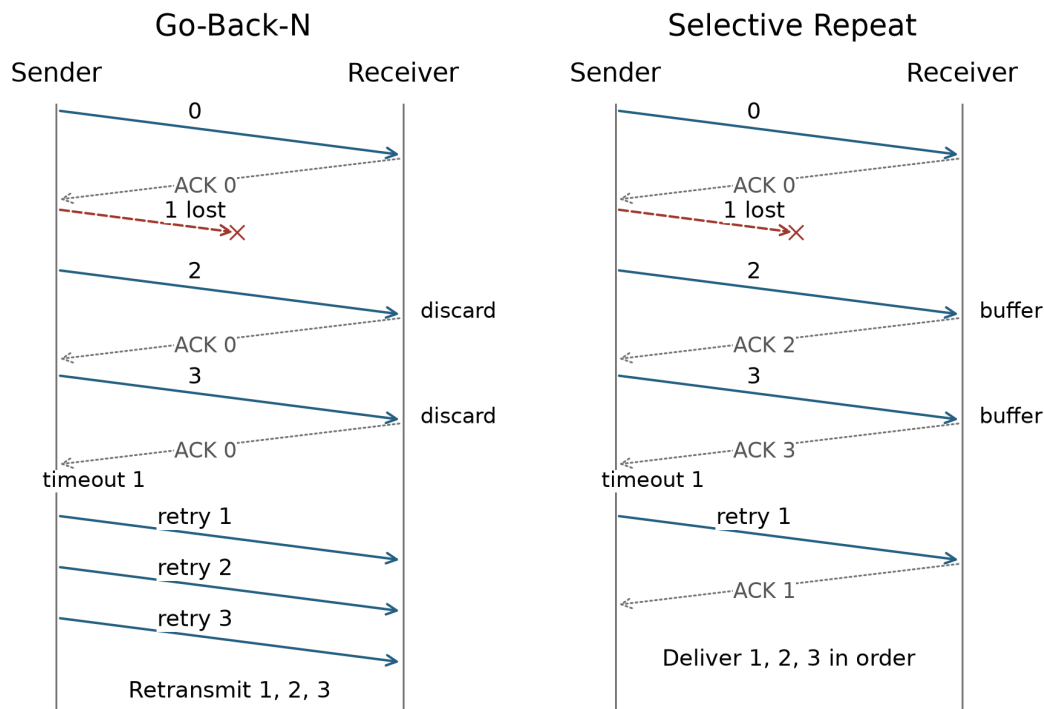
来源: Chapter 3 part 2, 第1–4张幻灯片。

## 8. GBN与Selective Repeat | 后面的包到了, 要不要先收好?

流水线传输有两种值得比较的做法。Go-Back-N (GBN, 回退N步) 发送者可有N个未确认包, base表示最早未确认序号, nextseqnum表示下一新包序号。累计ACK(n)表示直到n都已收齐。基本模型只给最早未确认包设timer, 超时从base重传所有已发未确认包; 接收方丢弃乱序包, 重复确认最后连续收到的包。

Selective Repeat (SR, 选择重传) 分别确认正确收到的包, 缓存窗口内乱序包, 每个未确认包有自己的超时逻辑。某包超时只重传该包; 应用仍需按序交付, 不能因为缓存里有后段就跳过前段缺口。

**同一场景 / Worked:** 窗口足以容纳四包, 序号不回绕; 依次发0、1、2、3, 只丢数据1, 没有其他丢失或重排。 / Both protocols have sufficient window space, with no sequence-number wraparound. Send packets 0–3; only packet 1 is lost, and all other data and ACKs arrive without reordering. GBN收到0后ACK0, 收到2和3均丢弃并ACK0, 超时通常重传1、2、3。SR可缓存2和3并分别ACK2/ACK3, 等1补上后连续交付1、2、3。这是包序号, 尚不是TCP字节ACK。



Event order runs downward; not a measured RTT scale

教学时序图：两边都只发送0~3这一批，窗口足够大，只有数据1丢失，ACK均成功，未发生序号回绕。纵向表示事件顺序，不按真实RTT比例。GBN丢弃2和3后重传1~3；SR先存好2和3，只补1。图中省略了GBN重传后的正常累计ACK。

**独立题 / Check:** 第一批发0-7, 只有数据6丢失, 无重排、ACK无丢失, 窗口足够发完这批, 序号不回绕。在任何重传开始前, GBN与SR接收端分别发哪些ACK? / In the first batch, send packets 0-7 with a sufficiently large window and no sequence-number wrap. Only data packet 6 is lost; there is no reordering or ACK loss. List each receiver's ACKs before any retransmission.

答案 / Answer

GBN: 0,1,2,3,4,5,5。SR: 0,1,2,3,4,5,7。总共7个，因为丢失包没到达，不产生ACK。 / Seven ACKs in each case; their final value differs.

序号空间必须足够，避免旧包与新包混淆；一般SR窗口限制的完整论证属于补充，可回现有微课。当前Chapter 3 part 2，第13–15张幻灯片侧重窗口、缓存与逐包确认。

来源：Chapter 3 part 2, 第5—15张幻灯片。

## 9. TCP序号 | 从“第几个包”切换到“下一个字节”

TCP首部记录了通信和控制所需的信息：源/目的端口、序号、确认号、接收窗口、校验和、控制标志等。MSS是每段可携带的数据上限，不包含所有协议首部。

TCP segment序号是该段第一个数据字节的位置；累计ACK是下一期望字节。收到0—535后，ACK为536，而不是535。后一段先到不能让连续前缀跨过缺口；基本讨论也不能把“是否缓存乱序”与“是否交给应用”混为一谈。

**跟做 / Worked:** seq=1000且含500byte, 覆盖1000—1499; 若此前均收齐, ACK1500。收到的不是“1000号包”, 不能简单加1。

在课堂的Telnet回显例子中, 两端各有自己的字节序列; 回显字符属于反方向的数据, 一个段可以同时携带本方向数据和对反方向的ACK。不要用一个共享计数器给整个双向连接编号。

**English takeaway:** A TCP sequence number locates the first payload byte; a cumulative ACK identifies the next expected byte. Each direction has its own byte sequence.

来源: Chapter 3 part 2, 第17—23张幻灯片。

## 10. RTT与重传超时 | 等多久才有理由怀疑包丢了?

超时设置需要权衡: 过短会误重传, 过长会拖慢真正丢包的恢复。SampleRTT是一次测得的往返时间, EstimatedRTT是平滑估计:

$$E_{new} = (1 - \alpha)E_{old} + \alpha S, \quad \alpha = 0.125 \text{ 为课件常用值.} \quad (3.3)$$

S是本次样本, E与S都以相同时间单位计。历史样本的影响指数衰减。波动用DevRTT近似:  $D_{new} = (1 - \beta)D_{old} + \beta|S - E|$ , 课件常用 $\beta=0.25$ ; 超时 $RTO = E + 4D$ 。D不是“又一份平均RTT”, 而是安全余量相关的波动量。

把两项都算一次 (教学例): 旧E=100 ms、旧D=10 ms, 本次S=140 ms, 取alpha=0.125、beta=0.25。本例约定先更新E, 再把新E代入D的式子。 / Start with E=100 ms, D=10 ms and S=140 ms, with alpha=0.125 and beta=0.25. In this teaching calculation, update E first and use the new E in the deviation update.

| 步骤      | 代入                                    | 结果       |
|---------|---------------------------------------|----------|
| 平滑RTT   | $0.875 \times 100 + 0.125 \times 140$ | 105 ms   |
| 与新估计的偏差 | $\text{abs}(140 - 105)$               | 35 ms    |
| 平滑波动    | $0.75 \times 10 + 0.25 \times 35$     | 16.25 ms |
| 本例超时    | $105 + 4 \times 16.25$                | 170 ms   |

如果题目规定D固定为10 ms, 则是 $105 + 4 \times 10 = 145$  ms; 若用旧E更新D, 又会得到另一值。做题时要把采用的更新顺序写明, 不能把不同约定的结果直接比较。

**变式 / Transfer:** 从同样的旧E=100 ms、旧D=10 ms重新开始, S改为80 ms, 仍先更新E再更新D。求E、D、RTO。 / Restart from E=100 ms and D=10 ms, but use S=80 ms. Keep the same coefficients and update order. Find the new E, D and RTO.

**答 / Answer:** E=97.5 ms; 偏差17.5 ms, D=11.875 ms; RTO=145 ms。 / E=97.5 ms, D=11.875 ms, RTO=145 ms.

超时样本还涉及重传后无法确定ACK对应哪次发送的歧义，原Chapter 3 part 2，第24张幻灯片因此强调不把重传段直接用于SampleRTT。真实TCP的最小超时与退避规则需另参协议规范；本节只做课件给定模型，不把本例算出的毫秒数当所有系统应采用的配置值。

来源：Chapter 3 part 2，第24—26张幻灯片。

## 11. 回到老师的题目

Tutorial5〔本册 p.69〕前两题检查端点、方向和解复用；Q3检查反码加法及漏检；Q4要求你给可靠性结论找模型反例。先完成这些，再复习N150、N154、N158、N167。

英文自测：Why can different clients share server port 80? What ambiguity do sequence numbers solve? Why is a timer needed? Why can a delayed duplicate defeat alternating-bit assumptions? How do packet ACKs differ from TCP ACKs?

最低要点 / Key points：完整端点不同；分清新数据与重传；丢失时无反馈；旧编号可能被重用；TCP确认下一个字节。 / Different endpoint tuples; distinguish new data from retransmissions; loss may produce no feedback; old numbers can be reused; TCP acknowledges the next byte.

## 来源覆盖索引

| 原页                                                   | 讲义位置                 |
|------------------------------------------------------|----------------------|
| Chapter 3 part 1, 第1—3、8、16、21张幻灯片 章节范围与目录           | 开头、各节顺序；后续目录不计为已下载正文 |
| Chapter 3 part 1, 第4—7张幻灯片 传输服务、层间关系                 | §1                   |
| Chapter 3 part 1, 第9—15张幻灯片 复用、UDP/TCP解复用与图例         | §2                   |
| Chapter 3 part 1, 第17—20张幻灯片 UDP与checksum            | §3—4                 |
| Chapter 3 part 1, 第22—27张幻灯片 可靠传输接口与FSM              | §5                   |
| Chapter 3 part 1, 第28—39张幻灯片 rdt1.0、2.0、2.1、2.2      | §5                   |
| Chapter 3 part 1, 第40—44张幻灯片 rdt3.0与四种时序情境           | §6                   |
| Chapter 3 part 2, 第1—4张幻灯片 停等与流水线利用率                 | §7                   |
| Chapter 3 part 2, 第5—15张幻灯片 GBN/SR、窗口、状态与丢包          | §8                   |
| Chapter 3 part 2, 第16张幻灯片 目录                         | 当前范围提示               |
| Chapter 3 part 2, 第17—23张幻灯片 TCP概览、首部、seq/ACK、Telnet | §9                   |
| Chapter 3 part 2, 第24—26张幻灯片 RTT与timeout             | §10                  |

原页序保留，示意图与数值算例标为补充；后续part未提供的内容可在历史预习中选读。当前范围见开头说明。

# Tutorial 1 · Switching, circuits and sharing | 把链路模型算清楚

Chapter 1 [本册 p.6] · 课程目录 · 下一份Tutorial [本册 p.51]

按当前 原题PDF (3页) 的Q1–4逐题讲解，并对照 教师解答PPT (10页)。下文“教师结论”指该文件；推演、时间表和变式由整理者补充。所有选择题的原选项在原页图中，正文给出推理步骤。

**可选先修：** bit/byte与速率 [基础册 p.37]、流水线时间轴 [基础册 p.40]、组合概率 [基础册 p.28]。

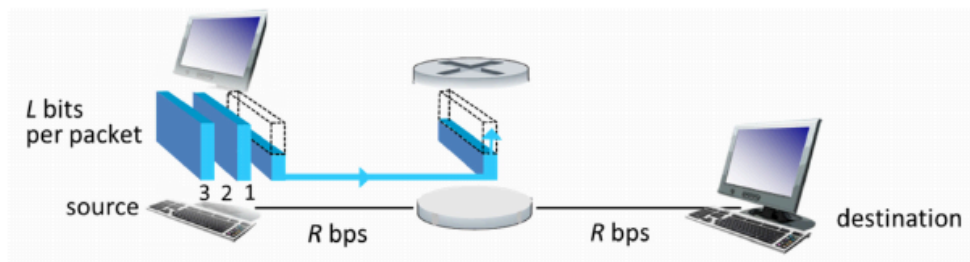
| 原题 | 优先级  | 难度/卡点       | 本题检验；依据                             |
|----|------|-------------|-------------------------------------|
| Q1 | 核心必会 | 中；不同包可以重叠   | 画存储转发时间轴；Tutorial 1教师解答，第3–4张幻灯片    |
| Q2 | 核心必会 | 中；每次连接占用哪些边 | 给出可行路由与容量上界；Tutorial 1教师解答，第6–7张幻灯片 |
| Q3 | 核心必会 | 低；单电路速率     | 单位换算与建立时延；Tutorial 1教师解答，第9张幻灯片     |
| Q4 | 核心必会 | 中；独立与组合数    | 容量和人数分布；Tutorial 1教师解答，第10张幻灯片      |

学习优先级依据当前原题与教师解答。

## Q1. Store-and-forward | 先等整包到，再转发

Tutorial 1, CS5222, 2026-27

- As we introduced in the lecture, the packet switching uses **store-and-forward transmission** at the inputs to the links. That means the entire packet must arrive at router before it can be transmitted onto the next link. (Ignore queuing, propagation delay, and processing delay)



*An illustrative example of Packet-switching: store-and-forward*

- Suppose there is exactly one packet switch between a sending host and a receiving host. The transmission rates between the sending host and the switch and between the switch and the receiving host are  $R_1$  and  $R_2$ , respectively. With store-and-forward switching, what is the total end-to-end delay to send a packet of length  $L$ ?

- $L / R_1$
- $L / R_2$
- $L / R_1 + L / R_2$
- $2L / R_1 + 2L / R_2$

- Assuming the transmission rate to be constant at  $R$ , what will be the end-to-end delay for sending  $P=3$  packets each of length  $L$  across  $N=2$  links (i.e., the packets need to pass two links to reach the destination as shown in the figure above)?

- $L / R$
- $2L / R$
- $3L / R$
- $4L / R$

- Consider the circuit switch network in Figure below. Suppose that there are 4 circuits on each link. Label the four switches A, B, C and D, going in the clockwise direction.

**完整条件 / Conditions:** 一个源主机、一个分组交换机、一个目的主机；两条链路速率为 $R_1$ 、 $R_2$ ，包长 $L$  bit。存储转发，忽略排队、传播和处理时延。b问改为相同速率 $R$ ， $P=3$ 个 $L$ -bit包跨 $N=2$ 条链路。  
 / One switch connects a sender and receiver through links  $R_1$  and  $R_2$ . Packets contain  $L$  bits. Use store-and-forward and ignore queueing, propagation and processing. In part b, three equal packets cross two equal-rate  $R$  links.

**a的推演：**  $t=0$ 开始发送； $t=L/R_1$ 时整包抵达交换机；交换机此时才开始第二段，又花 $L/R_2$ 。因此教师结论为

$$d = L/R_1 + L/R_2 \quad (\text{选C}) . \quad (\text{T1.1})$$

不是  $L/(R_1 + R_2)$ ，因为这两条链路是先后经过，不是并行拼成一条更快链路。

**b的推演：** 令 $T=L/R$ ，把每段链路当成一次只能发送一包的工位：

| 时段    | 源→交换机 | 交换机→目的 |
|-------|-------|--------|
| 0到T   | 包1    | 等待整包1  |
| T到2T  | 包2    | 包1     |
| 2T到3T | 包3    | 包2     |
| 3T到4T | 无     | 包3     |

最后整包到达时间 $4T=4L/R$ ，选D；与教师解答第4页  $(P+1)L/R$  一致。一般等速N跳为  $(P+N-1)L/R$ ，但必须保留本题全部简化条件。

**English answer:** (a)  $L/R_1+L/R_2$ . (b)  $4L/R$ . Store-and-forward constrains each packet, but separate links can transmit different packets simultaneously.

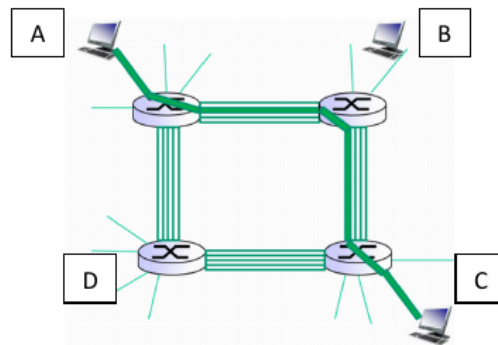
**变式 / Transfer:** 5个包、3条同速链路，同样忽略其他时延，多久？ / Five packets cross three equal-rate links under the same assumptions. Find completion time.

**答案 / Answer**

$7L/R$ 。第一包 $3L/R$ ，之后4个每隔 $L/R$ 到。 /  $7L/R$ : three intervals for the first packet, then four further intervals.

## Q2. Circuit capacity | 数连接，也要数它消耗了几条边

Tutorial 1, CS5222, 2026-27



a. What is the maximum number of simultaneous connections that can be in progress at any one time in this network?

- A) 4
- B) 8
- C) 16
- D) 32

b. Suppose that all connections are between switches A and C. What is the maximum number of simultaneous connections that can be in progress?

- A) 4
- B) 8
- C) 16
- D) 32

c. Suppose we want to make four connections between switches A and C, and another four connections between switches B and D. Can we route these calls through the four links to accommodate all eight connections?

- A) No
- B) Yes

3. Suppose we want to send a file of 160,000 bits from Host A to Host B over a circuit-switched network. Suppose that all links in the network use FDM with 12 frequency bands of the same bandwidth. Assume that the 12 frequency bands together can support a total bit rate of 1.536 Mbps. Also suppose that it takes 600 msec to establish an end-to-end circuit before Host A can

**条件 / Conditions:** A、B、C、D顺时针围成四边环，每条边4条电路，一次连接在路径每条边占一条电路。 / Four switches A,B,C,D form a clockwise ring; each link supports four circuits. A connection uses one circuit on every traversed link.

**a 最大任意连接数 / Maximum arbitrary simultaneous connections:** 让每条边只承载相邻两点之间的连接，每边4个、4条边，总16，选C。上界也为16，因为全网总共16份“边电路”且每连接至少占一份，所以这个构造达到上界。

**b 全部A到C / All connections from A to C:** 每个至少两跳，可分别走A—B—C与A—D—C，各4，总8，选B。A出发仅8份电路，给出同样的上界；不是 $16 \div 4$ ，因为每连接没有绕完整一圈。

**c 四个A—C加四个B—D / Four A—C plus four B—D:** 可行，选B (Yes)。按教师方案：A—C中2个经B、2个经D；B—D中2个经A、2个经C。

| 路径    | 连接数 | 占用的边  |
|-------|-----|-------|
| A—B—C | 2   | AB、BC |
| A—D—C | 2   | AD、DC |
| B—A—D | 2   | BA、AD |
| B—C—D | 2   | BC、CD |

逐边相加 $AB=4$ 、 $BC=4$ 、 $CD=4$ 、 $DA=4$ ，不超过容量。不要只说“总共8小于16，所以可以”，那忽略了路径可能挤在同一条边。

**English answer:** (a)16; (b)8; (c)Yes, split each opposite-pair demand evenly between its two two-hop paths. Every link then carries exactly four circuits.

**变式 / Transfer:** 每边只有3条电路，全部A—C最多多少？ / With three circuits per link, what is the maximum for A—C connections only?

**答案 / Answer**

6，两条边不重叠路径各3。 / Six, with three on each disjoint two-hop path.

### Q3. FDM and setup | 总速率不等于你的那一份

**条件 / Conditions:** 文件160000 bit；电路网络用12个等宽频带，总速率1.536 Mbps，建立端到端电路需600 ms，随后才发文件。求总时间。 / A 160000-bit file uses one of twelve equal-rate FDM circuits sharing 1.536 Mbps. Circuit setup takes 600 ms before transmission. Find total time.

每电路速率  $1.536 \times 10^6 / 12 = 128000 \text{ bit/s}$ 。发送用  $160000 / 128000 = 1.25 \text{ s}$ ，加setup 0.6s，共1.85s，选A。教师解答第9页相同。题目仅计建立和发送时延。

**English answer:** Each circuit supplies 128 kbps; 1.25 s transmission plus 0.60 s setup gives 1.85 s.

常见错法：直接除总速率1.536Mbps；把600ms当600s；把160000bit再乘8（本来就已经是bit）。

**变式 / Transfer:** 同电路发送320000bit、setup不变，总时间？ / Double the file size to 320000 bits without changing setup or circuit rate.

**答案 / Answer**

2.5+0.6=3.1s。 / 3.1 s. 文件翻倍只让发送部分翻倍，不把建立时延翻倍。

## Q4. Statistical multiplexing | 算人数前，先说是否独立

### Tutorial 1, CS5222, 2026-27

begin to transmit the file. How long does it take to send the file?

- A) 1.85 sec
- B) 1.95 sec
- C) 2.05 sec
- D) 2.15 sec

4. Suppose users share a 3 Mbps link. Also suppose each user requires 150 kbps when transmitting, but each user transmits only 10 percent of the time.

a. When circuit switching is used, how many users can be supported?

- A) 10
- B) 20
- C) 30
- D) 40

b. For the remainder of this problem, suppose packet switching is used. Find the probability that a given user is transmitting.

- A) 0.1
- B) 0.2
- C) 0.3
- D) 0.4

c. Suppose there are 120 users. Find the probability that at any given time, exactly  $n$  users are transmitting simultaneously.

- A)  $\binom{120}{n}(0.1)^{120-n}(0.9)^n$
- B)  $\binom{120}{n}(0.9)^{120-n}(0.1)^n$
- C)  $(0.9)^n$
- D)  $(0.1)^n$

**条件 / Conditions:** 共享3Mbps；每人活跃时需150kbps，每人10%时间活跃。a用电路交换；b/c用分组交换，c共有120用户。 / Users share 3 Mbps, each requiring 150 kbps when active, with activity fraction 10%. Part a uses circuits; parts b/c use packet switching, with 120 users in c.

a 每用户预留150kbps，可容纳  $3 \times 10^6 / (150 \times 10^3) = 20$ 人，选B。不把它再除活跃比例，电路资源已经预留。

b 随机时刻某用户活跃概率0.1，选A。把长期活跃比例作为该随机时刻概率，是题目的稳态抽样约定。

c 在用户独立、每人同为 $p=0.1$ 的模型下， $K \sim \text{Binomial}(120, 0.1)$ ，恰 $n$ 人活跃概率

$$P(K = n) = \binom{120}{n} (0.1)^n (0.9)^{120-n}, \quad n = 0, 1, \dots, 120. \quad (\text{T1.2})$$

对应原选项B（其因子顺序先写0.9）。组合数选出哪些人在发；0.1乘 $n$ 次、0.9乘其余 $120-n$ 次。原题没有清楚写独立，教师二项解答隐含这个假设，不能遗漏。

**English answer:** (a)20 users; (b)0.1; (c) $\text{choose}(120, n) \cdot 0.1^n \cdot 0.9^{120-n}$ , assuming independent user activity.

平均活跃人数 $Np=12$ 并不意味着始终12人；容量20也不保证任何时刻都够用。补充：超额事件为 $K>20$ ，应该求从21起的尾概率，不是只算 $K=20$ 或 $K=21$ 。超额需求可用同一个人数分布继续计算。

**变式 / Transfer:** 3个独立用户、 $p=0.1$ ，容量可同时容纳2人，超额概率？ / Three independent users have  $p=0.1$  each; capacity supports two. Find excess-demand probability.

#### 答案 / Answer

三人全活跃， $0.1^3=0.001$ 。 / All three must be active, giving 0.001.

## 回查与巩固

所有数值与环路逐边占用由 network\_examples.py 复算。原题PDF p1=Q1/Q2引入，p2=Q2/Q3引入，p3=Q3继续/Q4；Tutorial 1教师解答，第1–2张幻灯片背景、3–4Q1、5电路图、6–7Q2、8FDM/TDM、9Q3、10Q4均已对应。

复习使用现有Tutorial1卡，首选N071/N072（存储转发）、N076（FDM）、N077（人数与容量），以及Chapter1〔本册 p.15〕相关题。学习完应能用英文解释 *why*，不只记住C/D/B这些选项字母。

---

CS5222–Tutorial02

## Tutorial 2 · Delay and traceroute | 先定义计时事件，再按顺序相加

Chapter 1 [本册 p.10] · 课程目录 · Tutorial 1 [本册 p.44] · Assignment 1 [本册 p.74]

依据当前 原题PDF (3页) 与 教师解答PPT (13页)，按Q1–5讲解。难点是计时边界与转发模型；优先级均为核心必会，Q1/Q3中等偏难，Q2/Q4中等，Q5需实际观察与解释。优先级依据当前 Tutorial。

**基础复习 (可跳过)：** 单位 [基础册 p.37]、首位与末位时间轴 [基础册 p.40]。所有计算约定  $\text{Mbps}=10^6\text{bit/s}$ 、 $\text{kbps}=10^3\text{bit/s}$ 。

## Q1. VoIP | 语音先攒成一包，还要算这段等待

Tutorial 2, CS5222, Sem A, 2026-27

1. In this problem, we consider sending real-time voice from Host A to Host B over a packet-switched network (VoIP). Host A converts analog voice to a digital 128 kbps bit stream on the fly. Host A then groups the bits into 64-byte packets. There is one link between Host A and B; its transmission rate is 4 Mbps and its propagation delay is 8 msec. As soon as Host A gathers a packet, it sends it to Host B. As soon as Host B receives an entire packet, it converts the packet's bits to an analog signal. How much time elapses from the time a bit is created (from the original analog signal at Host A) until the bit is decoded (as part of the analog signal at Host B)? (note: 128 kbps =  $128 \times 10^3$  bps, 4Mbps =  $4 \times 10^6$  bps)

- A) 2.08 msec  
B) 6.08 msec  
C) 9.128 msec  
D) 12.128 msec

2. a) Consider a packet of length  $L$  which begins at end system A and travels over three links to a destination end system. These three links are connected by two packet switches. Let  $d_i$ ,  $s_i$ , and  $R_i$  denote the length, propagation speed, and the transmission rate of link  $i$ , for  $i = 1, 2, 3$ . The packet switch delays each packet by  $d_{proc}$ . Assuming no queuing delays, in terms of  $d_i$ ,  $s_i$ , and  $R_i$ , ( $i = 1, 2, 3$ ), and  $L$ , what is the total end-to-end delay for the packet?

- A)  $L/(R_1 + R_2 + R_3) + d_1/s_1 + d_2/s_2 + d_3/s_3 + 2d_{proc}$   
B)  $L/R_1 + L/R_2 + L/R_3 + d_1/s_1 + d_2/s_2 + d_3/s_3 + 2d_{proc}$   
C)  $LR_1 + LR_2 + LR_3 + d_1s_1 + d_2s_2 + d_3s_3 + 2d_{proc}$   
D)  $LR_1 + LR_2 + LR_3 + d_1s_1 + d_2s_2 + d_3s_3$

b) Suppose now packet is 1,500 bytes, the propagation speed on all three links is  $2.5 \times 10^8$  m/s, the transmission rates of all three links are 2 Mbps, the packet switch processing delay is 3 msec, the length of the first link is 5,000 km, the length of the second link is 4,000 km, and the length of the last link is 1,000 km. For these values, what is the end-to-end delay? (Note that  $M = 10^6$ ,  $k = 10^3$ )

- A) 24 msec  
B) 64 msec  
C) 86 msec  
D) 96 msec

**完整条件 / Conditions:** A以128kbps实时生成语音比特，每64byte成一包，收集完成立即发送；链路4Mbps、传播8ms，B收完整包后还原模拟语音。问生成一个bit到它被解码的时延。 / A generates voice at 128 kbps, groups it into 64-byte packets and immediately transmits each completed

packet over a 4-Mbps link with 8-ms propagation. B decodes after receiving the complete packet. Find generation-to-decoding delay.

先按包首位及连续流体近似计算：包长512bit；收集一包  $512/128000 = 4\text{ms}$ ；把包送上链路  $512/4000000 = 0.128\text{ms}$ ；传播8ms。首位从生成到可开始播放的延迟为 **12.128ms**，选D，与Tutorial 2教师解答，第5张幻灯片结论一致。

这里“任意一位”还要区分读取与播放这两个时刻。一个包内越晚产生的bit等待成包越短；若问“该bit到达整包缓冲、可被读取”的延迟，数值随位次变化，约从12.128ms降到8.128ms。若接收端在包到齐后，按与源相同128kbps的节奏顺序恢复/播放语音，后面的bit在接收端也会多等相同位次时间，因此每个bit的生成到播放时延相同，都是12.128ms。教师“all bits”的说法应连着这个播放约定读。

设第j位相对首位晚产生  $\tau_j$ ，包到达时刻  $T=12.128\text{ms}$ ；可读取延迟为  $T-\tau_j$ ，顺序播放时刻为  $T+\tau_j$ ，播放延迟则  $(T+\tau_j)-\tau_j=T$ 。因此，答题前要确定问的是可读取时刻还是播放时刻。

**English answer:** Under the instructor's packetization and source-rate sequential playout model, the delay is  $4+0.128+8=12.128\text{ ms}$  for every bit. Complete-packet availability alone gives a position-dependent delay.

**变式 / Transfer:** 包改128byte，其他条件不变，首位/同速播放时延？ / Double packet size to 128 bytes, keeping all rates and propagation unchanged.

**答案 / Answer**

$8+0.256+8=16.256\text{ms}$ 。 / 16.256 ms. 较大包减少每份数据的头部比例，但收集等待变长；本题没有给头长，不计算额外百分比。

## Q2. Three store-and-forward links | 每一跳的发送与传播都要数

**条件 / Conditions:** 长L bit的包经三条链路，两台存储转发交换机；第i链路长 $d_i\text{ m}$ 、传播速度 $s_i\text{ m/s}$ 、发送速率 $R_i\text{ bit/s}$ ，每台交换机处理时延 $d_{\text{proc}}$ ，无排队。 / An L-bit packet crosses three store-and-forward links and two switches. Link i has length  $d_i$ , propagation speed  $s_i$  and rate  $R_i$ ; each switch adds  $d_{\text{proc}}$  and there is no queueing.

a 把事件从左到右排：链路1发送/传播 → 交换机1处理 → 链路2发送/传播 → 交换机2处理 → 链路3发送/传播。

$$d_{\text{end}} = \sum_{i=1}^3 \frac{L}{R_i} + \sum_{i=1}^3 \frac{d_i}{s_i} + 2d_{\text{proc}} \quad (\text{选B}) . \quad (\text{T2.1})$$

b 给定 $L=1500\text{byte}=12000\text{bit}$ ，各 $R=2\text{Mbps}$ ，各 $s=2.5 \times 10^8\text{m/s}$ ，距离5000/4000/1000km，每交换机3ms：

| 部分   | 计算                         | 结果              |
|------|----------------------------|-----------------|
| 三段发送 | 每段 $12000/(2 \times 10^6)$ | 6+6+6ms         |
| 三段传播 | 每段先 $km \rightarrow m$ 再除s | 20+16+4ms       |
| 两次处理 | $2 \times 3ms$             | 6ms             |
| 总计   | $18+40+6$                  | <b>64ms, 选B</b> |

**English answer:** Sum the three serialization times, three propagation times and two processing times. The numerical total is 64 ms.

这里包长L在公式中必须是bit；若直接使用1500需在分子明确乘8。处理是两台交换机，不是三条链路所以乘3。

### Q3. Immediate bit forwarding | 不用每个中间节点再等整包

Tutorial 2, CS5222, Sem A, 2026-27

3. In the above problem, suppose  $R_1 = R_2 = R_3 = R$  and  $d_{\text{proc}} = 0$ . Further suppose the packet switch does not store-and-forward packets but instead immediately transmits each bit it receives before waiting for the entire packet to arrive. What is the end-to-end delay?

- A) 16 msec  
B) 26 msec  
C) 36 msec  
D) 46 msec

4. A packet switch receives a packet and determines the outbound link to which the packet should be forwarded. When the packet arrives, one other packet is halfway done being transmitted on this outbound link and four other packets are waiting to be transmitted. Packets are transmitted in order of arrival. Suppose all packets are 1,500 bytes and the link rate is 2 Mbps. What is the queuing delay for the packet? (Note that  $M = 10^6$ .)

- A) 27 msec  
B) 37 msec  
C) 47 msec  
D) 57 msec

More generally, what is the queuing delay when all packets have length  $L$ , the transmission rate is  $R$ ,  $x$  bits of the currently-being-transmitted packet have been transmitted, and  $n$  packets are already in the queue?

**条件 / Conditions:** 沿用Q2,  $R_1=R_2=R_3=R$ 、 $d_{\text{proc}}=0$ , 无排队, 中间节点每收到一bit立即转发, 不等整包。 / Use the Q2 path, equal rates and zero processing; each bit is forwarded immediately with no whole-packet waiting or queueing.

首位经过各传播段, 末位相对首位只落后源端一次 $L/R$ 。故总为  $L/R + \sum_i d_i/s_i$ 。带Q2数字为  $6+20+16+4=46\text{ms}$ , **选D**, Tutorial 2教师解答, 第8张幻灯片相同。

这不表示物理链路“传输时间消失”, 而是各段的发送过程可以流水重叠, 末位到达总时间只额外保留一次包序列长度。不同速率、实际cut-through启动/头部检查有其他限制; 不要把这个理想模型直接当所有交换机行为。

**English answer:** Equal-rate immediate bit forwarding gives  $L/R$  plus all propagation delays, hence 46 ms. Serialization on successive links overlaps.

**变式 / Transfer:** 四条相同速率链路，每条传播2ms，源包发送3ms，即时转发和整包存储转发分别多久？ / Four equal-rate links each propagate for 2 ms; serialization takes 3 ms. Compare immediate bit forwarding with store-and-forward for one packet.

**答案 / Answer**

即时 $3+4\times2=11\text{ms}$ ；存储转发 $4\times3+4\times2=20\text{ms}$ ，忽略排队/处理。 / 11 ms versus 20 ms, with no queueing or processing.

## Q4. Queueing | 只算轮到你之前的等待

**条件 / Conditions:** 新包到达时，出口上一个1500byte包已发一半，还有4包等待，均1500byte, FIFO,  $R=2\text{Mbps}$ 。求新包排队时延，再推广为包长 $L$  bit、当前包已发送 $x$  bit、另有 $n$ 包排队。 / A new packet finds one equal packet half transmitted and four equal packets queued, FIFO,  $L=1500$  bytes and  $R=2$  Mbps. Find queueing delay, then generalize to  $x$  bits already transmitted and  $n$  waiting packets.

前面剩4.5包， $4.5 \times 12000 / (2 \times 10^6) = 0.027\text{s} = 27\text{ms}$ ，选A。一般式为

$$d_{\text{queue}} = \frac{(L - x) + nL}{R}. \quad (\text{T2.2})$$

**English answer:** Queueing delay is 27 ms, or  $((L-x)+nL)/R$ . It excludes transmission of the arriving packet itself.

若问它完成发送的时间，则另加 $L/R$ ；若问到目的地还要加传播及后续节点条件。若出口本来空闲，就不是这条“有一个正在发送包”的场景，等待为0。

**变式 / Transfer:** 同速同包长，当前包已发75%，队列前面还有2包，新包排队多久？ / The current packet is 75% transmitted and two packets are queued ahead. Find the new packet's queueing delay.

**答案 / Answer**

$2.25 \times 6 = 13.5\text{ms}$ 。 / 13.5 ms. 不是加上新包自己的6ms变成19.5ms。

## Q5. Traceroute | 把真实测量和推断分开写

**原任务 / Task:** 对cityu.edu.hk运行traceroute，解释每列、星号和后跳RTT较小的原因。 / Run traceroute to cityu.edu.hk and explain its columns, asterisks, and non-monotonic RTTs.

macOS可运行原命令 `traceroute cityu.edu.hk`。下方2026-09-23的观测使用：

```
traceroute -n -m 12 -q 3 -w 1 cityu.edu.hk
```

-n保留数值IP, -m 12最多12跳, -q 3每跳3次探测, -w 1每次最多等1秒。这些参数用于限制本次探测的长度与等待时间, 并非题目指定值。输出与时间见2026-09-23观察, 采集脚本为 observe\_network.py。

每行第一项为TTL/hop编号, 随后为响应地址和三次RTT (ms); 某次无及时响应显示\*。地址识别的是响应接口, 复杂网络里不保证完整暴露每一台中间设备。macOS常用UDP探测和ICMP响应; Windows tracert的实现细节不同, 不应机械照搬命令选项。

星号可以来自过滤、应答限速、探测/响应丢失、超时等。Tutorial 2教师解答, 第12张幻灯片强调防火墙是常见原因, 但仅凭星号不能确定具体根因。后跳RTT小于前跳, 可能因为不同时刻队列变化、回程路由不同、路由器处理探测优先级不同; RTT也不是沿途正向传播的逐段累加测量。

**English answer:** Columns identify probe hop, responding address and round-trip times. An asterisk means no response within the timeout. Different probes and return paths can produce non-monotonic RTTs.

**已保存的实际观察 (2026-09-23 01:19, UTC+8):** 域名解析有两个地址, traceroute选用 45.60.199.218; 第1跳三个RTT约4.352/3.957/4.301ms, 第7跳有星号, 第11跳第三次升到 192.365ms。在最多12跳内没有看到目的地址响应, 故这不是完整路径。进程正常结束也不表示探测已到终点。同一跳有不同地址响应, 不足以断言一条固定唯一线路。

你自己的观测应另记时间、探测参数和实际输出; 换时间或网络, 路径和RTT可能变化。

## 回查与巩固

---

原题p1=Q1/Q2, p2=Q3/Q4, p3=Q5。Tutorial 2教师解答, 第1-4张幻灯片为四类时延/车队背景 (见Chapter1), 5=Q1, 6-7=Q2, 8=Q3, 9=Q4, 10-11=traceroute课堂示例, 12-13=Q5解释。全部已对应, 原教师结论与本文补充条件分开。

计算复核: network-example-results.json。复习可用现有时延卡N025与原题卡N080, 根据本题错误位置, 复习单位、时序、转发或排队的相关概念。

## CS5222–Tutorial03

## Tutorial 3 · HTTP and DNS | 从报文证据走到时间计算

Chapter 2 [本册 p.19] · 课程目录 · Assignment 1 [本册 p.74]

依据当前 原题PDF (3页), 按Problem1–4展开。教师解答共13页: Tutorial 3教师解答, 第3–4张幻灯片对应Q1, Tutorial 3教师解答, 第8张幻灯片对应Q2, Tutorial 3教师解答, 第11–13张幻灯片对应Q3; Q4未给当前测量答案。以下保留教师结论与整理补充的区别。Q1/Q2/Q3为核心必会 (中等难度, 卡在字段证据与RTT条件), Q4为常规掌握的实际操作任务 (需区别观察和推断)。

可选先修: ASCII、CRLF与报文 [基础册 p.39]、时间轴 [基础册 p.40]。

### Problem 1. Read an HTTP request | 不知道的就明确说不知道

Tutorial 3, CS5222, Sem A, 2026-27

Problem 1. Consider the following string of ASCII characters that were captured by Wireshark when the browser sent an **HTTP GET** message (i.e., this is the actual content of an **HTTP GET** message). The characters `<cr><lf>` are carriage return and line-feed characters (that is, the italicized character string `<cr>` in the text below represents the single carriage-return character that was contained at that point in the HTTP header). Answer the following questions, indicating where in the **HTTP GET** message below you find the answer.

```
GET /cs453/index.html HTTP/1.1<cr><lf>Host:gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 ( Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax) <cr><lf> Accept:ext/xml,
application/xml, application/xhtml+xml, text /html;q=0.9,
text/plain;q=0.8,image/png, */*;q=0.5
<cr><lf>Accept-Language: en-us,en;q=0.5<cr><lf>Accept-Encoding:
zip,deflate<cr><lf>Accept-Charset: ISO-8859-
1,utf8;q=0.7,*;q=0.7<cr><lf>Keep-Alive: 300<cr><lf>
Connection:keepalive<cr><lf><cr><lf>
```

- What is the URL of the document requested by the browser?
- What version of HTTP is the browser running?
- Does the browser request a non-persistent or a persistent connection?
- What is the IP address of the host on which the browser is running?
- What type of browser initiates this message? Why is the browser type needed in an HTTP request message?

**任务 / Task:** 原题给出Wireshark看到的一段HTTP GET的ASCII内容, <cr><lf>代表实际单字节CR、LF。指出URL、HTTP版本、是否持久、浏览器主机IP、浏览器类型及该字段的用途。 / From the supplied HTTP GET bytes, identify the URL, version, connection preference, client IP, and user-agent type and purpose, citing the relevant fields.

为便于定位, 下面仅摘录与答案直接相关的原字段; 完整报文 (包括Accept各项及原拼写) 见上图:

```
GET /cs453/index.html HTTP/1.1<cr><lf>
Host:gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 ( Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax) <cr><lf>
Keep-Alive: 300<cr><lf>
Connection:keepalive<cr><lf><cr><lf>
```

- a. **URL / 请求地址:** 按题目传统明文HTTP情境, `http://gaia.cs.umass.edu/cs453/index.html`; Host给主机, GET行给路径。若孤立看这些应用字节, 并不能独立证明外层是否经过TLS, 方案http来自题目情境。 / Under the exercise's plain-HTTP context, combine Host and the request target to obtain this URL.
- b. **Version / 版本:** 请求行明确HTTP/1.1。 / The request line states HTTP/1.1.
- c. **Connection / 连接:** 题意为持久连接。原keepalive不是规范常见的keep-alive拼写, 但HTTP/1.1默认可持久, 且这里未见Connection: close; 不要把拼写错误当新标准token。头表达客户端意图, 不单独证明服务器实际维持了多久。RFC9112 §9.3 / The intended answer is persistent HTTP, consistent with HTTP/1.1 defaults; the header alone does not prove the connection's observed lifetime.
- d. **Client IP / 浏览器主机IP:** 给出的HTTP头没有源IP, 无法仅据此确定。gaia.cs.umass.edu是请求目标, 不能当客户端。需原IP包头/捕获上下文。 / The client IP cannot be determined from these HTTP fields; inspect the enclosing IP header or capture metadata.
- e. **Browser / 浏览器:** Tutorial 3教师解答, 第4张幻灯片简答写Mozilla/5.0; 完整User-Agent还自称Netscape/7.2, 并带Mozilla/Gecko兼容信息。课堂作答可以指出教师采用的token, 同时说明完整字段不只一个产品名。服务器可能据此作兼容处理、统计或内容选择; 不要求每种浏览器都必须得到不同页面, 也不是身份认证证据。 / The field claims Netscape 7.2 with Mozilla/Gecko compatibility tokens. It can support compatibility handling or analytics, but is self-reported and spoofable.

**独立变式 / Transfer:** GET /a.png HTTP/1.1、Host: example.com、Connection: close, 能判断路径、连接意图及客户端IP吗? / Identify the path and connection preference, and determine whether these fields reveal the client IP.

#### 答案 / Answer

路径/a.png; 请求关闭连接; 不能确定客户端IP。 / Path /a.png, requested connection closure, client IP unavailable from these fields.

## Problem 2. DNS resolution | 一对箭头才是一次往返

Tutorial 3, CS5222, Sem A, 2026-27

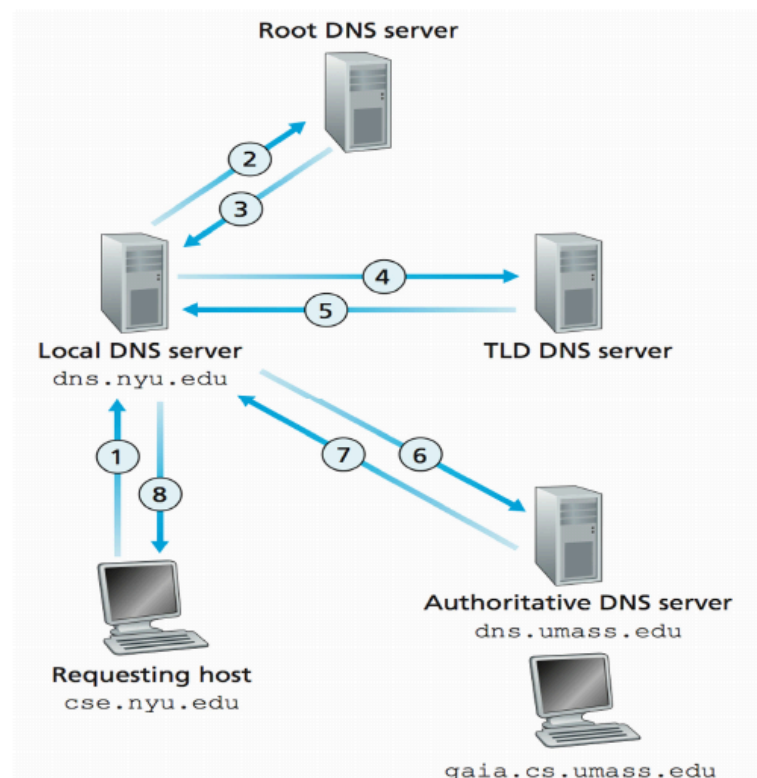
Problem 2. Assume that the RTT between a client and the local DNS server is  $RTT_L$ , while the RTT between the local DNS server and other DNS servers is  $RTT_r$ . Assume that no DNS server performs caching.

a. What is the total response time for the scenario illustrated in the following Figure?

- A)  $RTT_L$   
 B)  $RTT_L + RTT_r$   
 C)  $RTT_L + 2RTT_r$   
 D)  $RTT_L + 3RTT_r$

b. Assume now that the DNS record for the requested name is cached at the local DNS server. What is the total response time?

- A)  $RTT_L$   
 B)  $RTT_L + RTT_r$   
 C)  $RTT_L + 2RTT_r$   
 D)  $RTT_L + 3RTT_r$



**条件 / Conditions:** client到local DNS的RTT为 $RTT_L$ , local到其他DNS每次RTT为 $RTT_r$ 。a无任何缓存, 按图依次访问root、TLD、authoritative; b所需记录已在local缓存。忽略额外处理与传输大小。 / The client-local RTT is  $RTT_L$ ; each local-remote DNS RTT is  $RTT_r$ . Follow the figure's sequential root, TLD and authoritative queries without caches, then consider a local cache hit.

a 1/8合成一次client-local往返, 2/3、4/5、6/7分别三次local-remote往返。总  $RTT_L + 3RTT_r$ , **选D**。不是8RTT, 因为图上8条单向消息不能各算一次来回。

b local直接回答, 有效缓存无需再查询上级, 所以  $RTT_L$ , **选A**。本题缓存只在local; 若题设改为客户端本身已有可用缓存, 网络DNS往返又不同。

**English answer:** (a) $RTT_L + 3RTT_r$ ; (b) $RTT_L$ . Count round trips, not individual arrows, and locate the cache precisely.

**变式 / Transfer:**  $RTT_L = 4ms$ 、 $RTT_r = 20ms$ , 无缓存与local命中分别多久? / Compute both cases for these RTTs.

**答案 / Answer**

64ms与4ms。 / 64 ms and 4 ms. 缓存减少的是后续查询, 不让client到local的往返也凭空消失。

## Problem 3. A page with eight objects | 先取目录，再知道要取哪八件东西

Tutorial 3, CS5222, Sem A, 2026-27

Problem 3. Suppose within your browser you type in a URL to obtain a webpage. The IP address for the associated URL is not cached in your local host, so as DNS lookup is necessary to obtain the IP address. Suppose that  $n$  DNS servers are visited before your host receives the IP address from DNS; Visiting the  $i$ -th of the them incurs an  $RTT_i$  per DNS. Further suppose the HTML file references eight very small objects on the same server. Let  $RTT_0$  denote the RTT between the local host and the server containing an object. Neglecting transmission times, how much time elapses with

a. Non-persistent HTTP with no parallel TCP connections?

- A)  $RTT_1 + \dots + RTT_n + 8RTT_0$
- B)  $RTT_1 + \dots + RTT_n + 9RTT_0$
- C)  $RTT_1 + \dots + RTT_n + 16RTT_0$
- D)  $RTT_1 + \dots + RTT_n + 18RTT_0$

b. Non-persistent HTTP with the browser configured for 5 parallel connections?

- A)  $RTT_1 + \dots + RTT_n + 4RTT_0$
- B)  $RTT_1 + \dots + RTT_n + 5RTT_0$
- C)  $RTT_1 + \dots + RTT_n + 6RTT_0$
- D)  $RTT_1 + \dots + RTT_n + 7RTT_0$

c. Persistent HTTP?

- A)  $RTT_1 + \dots + RTT_n + 8RTT_0$
- B)  $RTT_1 + \dots + RTT_n + 10RTT_0$
- C)  $RTT_1 + \dots + RTT_n + 12RTT_0$
- D)  $RTT_1 + \dots + RTT_n + 14RTT_0$

Problem 4. (Looking up an IP address using nslookup) You can type nslookup followed by the hostname you want to look up (e.g., type “nslookup cityu.edu.hk” in the command prompt), and nslookup will issues a DNS query to find out the IP address.

- a. What is the IP address of the hostname?
- b. Can you type the IP address in your browser’s address bar and get to the webpage?

条件 / Conditions: 域名未缓存，顺序DNS查找代价  $D = \sum_{i=1}^n RTT_i$ 。base HTML引用同一服务器的8个很小对象；client-server RTT为 $RTT_0$ ，忽略发送时间。先得到HTML才能知道对象地址。 / Sequential DNS costs  $D = \sum RTT_i$ . One base HTML file references eight small objects on the

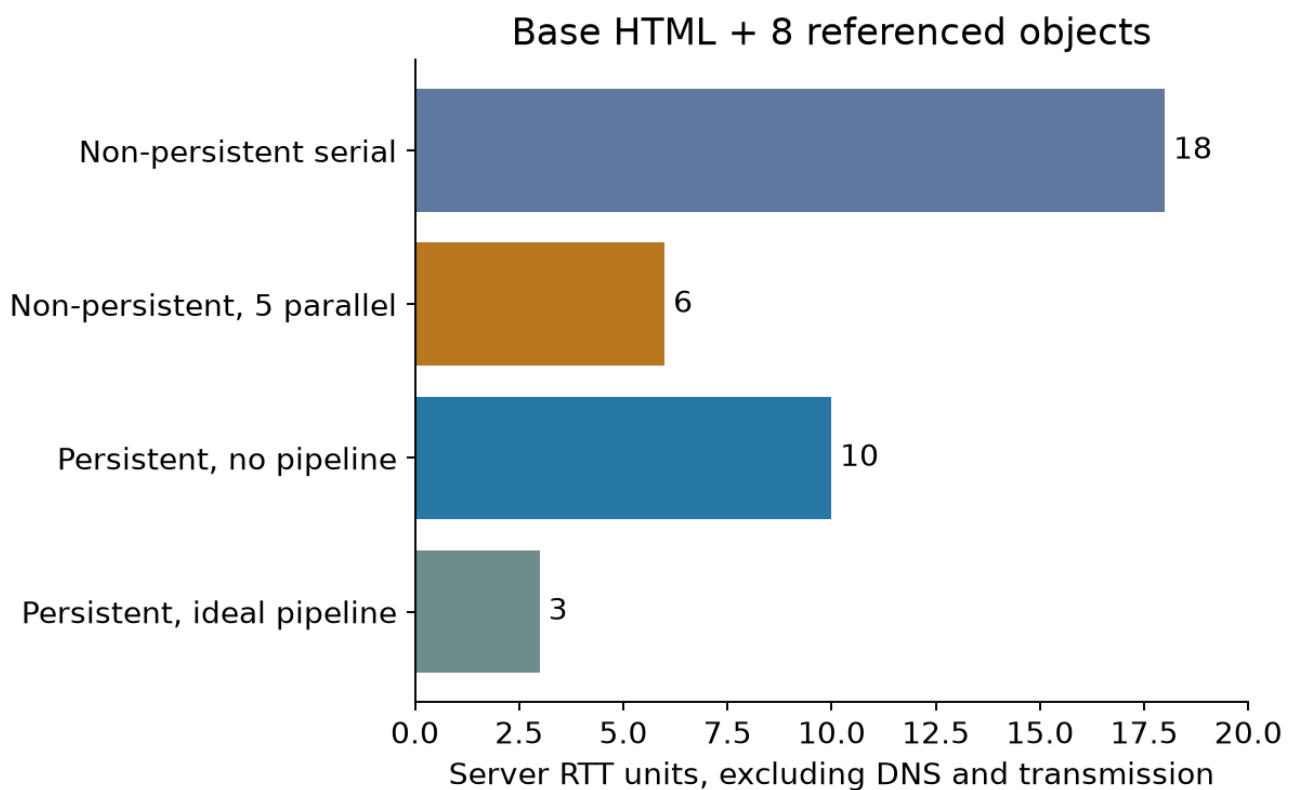
same server; server RTT is  $RTT_0$  and transmission times are negligible. Object requests start after the base HTML is received.

a **Non-persistent, serial** / 非持久且无并行：共9个对象，每个1RTT握手+1RTT请求返回，所以  $D+18RTT_0$ ，选D。

b **Non-persistent, five parallel connections** / 非持久最多5并行：HTML先用2RTT；8对象分两批（5+3），每批各2RTT，合计  $D+6RTT_0$ ，选C。不能把HTML也塞入尚未解析出来的8对象同一批。

c **Persistent** / 持久：原题未明说是否pipelining；新到Tutorial 3教师解答，第13张幻灯片明确按逐个请求计算，确认采用单连接**无流水线**。结合给出的选项，一次握手+HTML一次往返+8对象各一次，共  $D+10RTT_0$ ，选B。若另加“拿到HTML后流水线一次发完8请求”的理想条件，则  $D+3RTT_0$ ；这不在原选项中。因此，答案需注明采用单连接、无流水线的约定。

**English answer:** (a) $D+18RTT_0$ ; (b) $D+6RTT_0$ ; (c) $D+10RTT_0$  for non-pipelined persistent HTTP. Pipelining would change part c to  $D+3RTT_0$  under the idealized model.



图按握手/请求轮次计算，仅表示题目简化模型；不包括TLS、服务端处理时间、连接拥塞控制与大文件传输。

**变式 / Transfer:** 改为base HTML+6对象，非持久且最多4并行、其他条件不变，结果？ / Use six referenced objects and at most four parallel non-persistent connections.

**答案 / Answer**

$D+2RTT_0+2 \times \text{ceil}(6/4)RTT_0 = D+6RTT_0$ 。 /  $D+6RTT_0$ 。物件少两个仍需两批，所以本模型总时间没有减少。

## Problem 4. Nslookup and direct-IP browsing | 解析成功，不保证IP网址等于原站

---

**原任务 / Task:** 用nslookup cityu.edu.hk找地址，再判断把IP直接输入浏览器能否得到网页。 / Resolve cityu.edu.hk and investigate whether navigating directly to its IP retrieves the site.

本机macOS于 2026-09-23 01:19 (UTC+8) 实际查询，解析器223.5.5.5返回non-authoritative answer，两个A地址为 45.60.199.218、45.60.197.218。“非权威”表示答案来自递归/缓存服务器，不等于错误；多地址也不是异常。地点、时间、DNS策略变化可返回不同集合，不能把这两个IP当永久常量背下来。

直接输入IP可能打开默认页面、错误页、重定向，也可能某些服务可用；**不保证与域名访问相同**。HTTP虚拟主机依赖Host，HTTPS还涉及TLS服务器名SNI和证书名称，CDN/WAF可能按域名路由。不能从“DNS返回IP”推断“浏览器写这个IP必定显示该站”。这份2026-09-23的记录只包含DNS查询，未包含直接IP访问的结果。

**English answer:** The dated lookup returned two A records. Direct-IP navigation is not guaranteed to reproduce the hostname-based website because virtual hosting, TLS name validation and routing can depend on the hostname.

## 来源与复习

---

原题p1=Problem1全部五问，p2=Problem2原图及两问，p3=Problem3三种连接模型与Problem4实际查询。理论对应Chapter2 p19-31 (HTTP)、p53-67 (DNS)；教师解答与本稿DNS/HTTP时间计算一致；未给出的实际网络测量与协议边界仍标为整理补充。HTTP补充标准查阅于2026-09-23。

复算脚本见network\_examples.py。关联现有原题卡N231、N232、N233时，先能解释字段证据和计时条件，再用Markji巩固。

## CS5222–Tutorial04

# Tutorial 4 · File Distribution, BitTorrent and DASH

## 先找谁供不动，再算怎样分得完

Chapter 2 [本册 p.19] · 课程目录 · 下一份：Tutorial 5 [本册 p.69]

依据原题2页和教师解答v2共15页，按Problem1–4。本Tutorial配套Chapter2应用层。题目单位用十进制Gbit/Mbit/kbit。

**按需基础：** 单位换算 [基础册 p.37]、速率与完成时间 [基础册 p.40]。先会用“任务量÷每秒处理量”算时间。

| 题目             | 优先级  | 难度与卡点          | 掌握要求 / 依据                                            |
|----------------|------|----------------|------------------------------------------------------|
| Q1 分配方案        | 核心必会 | 中，下界还需构造达到它的方案 | 按两种瓶颈给实际速率；Tutorial 4教师解答 v2，第1–5张幻灯片                |
| Q2 时间计算        | 核心必会 | 中，总供给与最慢下载     | 统一单位、比较每个约束；Tutorial 4教师解答 v2，第6–8张幻灯片               |
| Q3 free-riding | 常规掌握 | 中，可能与保证不同      | 解释optimistic unchoking条件；Tutorial 4教师解答 v2，第9–11张幻灯片 |
| Q4 DASH        | 核心必会 | 中，配对假设         | 区分 $N$ 、 $N^2$ 与 $2N$ ；Tutorial 4教师解答 v2，第12–15张幻灯片  |

优先级依据当前原题；额外练习标为变式。

## Problem 1. Construct a distribution scheme

**原条件 / Conditions：** 服务器初始持有 $F$  bit文件，分给 $N$ 个客户端，服务器总上传率 $u_s$ ，客户端下载率 $d_i$ ， $d_{\min} = \min_i d_i$ 。流体模型允许同时以不同速率发送，总发送率不超过 $u_s$ ，忽略其他瓶颈。 / The server simultaneously allocates divisible traffic to  $N$  clients, subject to its total upload and each client's download limit.

服务器必须上传 $NF$  bit，所以时间至少 $NF/u_s$ ；最慢客户端必须收 $F$  bit，所以至少 $F/d_{\min}$ 。但“至少这么久”还不是“确实做得到”，题目要求给方案。

**a.  $u_s/N < d_{\min}$ ：** 平均给每个客户端 $u_s/N$ 。总速率正好 $u_s$ ，每人都不超过其下载上限；每人同时在 $F/(u_s/N) = NF/u_s$ 完成。瓶颈是服务器供给。

**b.  $u_s/N > d_{\min}$ ：** 每人按 $d_{\min}$ 发送。总量 $Nd_{\min} < u_s$ ，所有客户端都收得下；同在 $F/d_{\min}$ 完成。快客户端没有跑满也不妨碍整体最优，因为慢客户端仍决定结束时刻。

合起来，每人分配 $\min(u_s/N, d_{\min})$ ，达到两项下界的最大值。等号边界两种表达相同。Tutorial 4教师解答 v2，第2张幻灯片/Tutorial 4教师解答 v2，第4张幻灯片的反证是在同一流体约束下说明方案，不是现实TCP性能保证。

**English answer:** Allocate  $\min(u_s/N, d_{\min})$  to every client. The aggregate rate is feasible and every client completes in  $\max(NF/u_s, F/d_{\min})$ .

**独立题 / Transfer:**  $F=100\text{Mbit}$ 、 $N=4$ 、 $u_s=20\text{Mbps}$ 、每人下载8Mbps，多久？ / For  $F=100\text{ Mbit}$ ,  $N=4$ , server upload  $20\text{ Mbps}$  and each client download  $8\text{ Mbps}$ , give a feasible allocation and completion time.

**答案 / Answer**

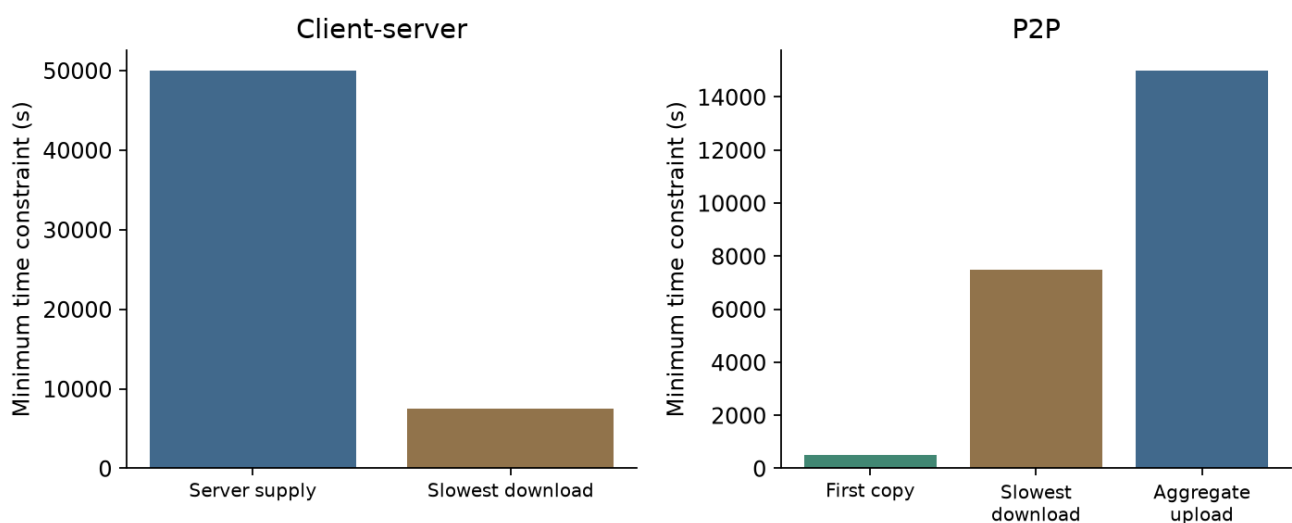
每人5Mbps，20秒。最慢下载约束12.5秒不是最终答案，要与总上传约束20秒取最大。 /  $5\text{ Mbps}$  each, finishing in 20 seconds.

## Problem 2. 15 Gbit分给100人

**完整条件 / Conditions:**  $F=15\text{Gbit}$ ， $N=100$ ，服务器30Mbps，每人下载2Mbps、上传700kbps=0.7Mbps。问client-server与P2P理想最短分发时间。 / A 15-Gbit file is distributed to 100 peers. Server upload is 30 Mbps; each peer downloads at 2 Mbps and uploads at 700 kbps. Using decimal units and the ideal model, compare client-server and P2P completion times.

先把文件写成15000Mbit。Client-server两项为 $100 \times 15000 / 30 = 50000$ 秒，以及 $15000 / 2 = 7500$ 秒，因此选**50000秒，C**。

P2P仍需要服务器发出至少一份文件，至少500秒；每人下载至少7500秒；全体上传总能力 $30 + 100 \times 0.7 = 100\text{Mbps}$ ，供给NF需要15000秒。三项取最大，得**15000秒，B**。



图中每根柱都是一个必要时间约束，最大者决定该理想模型下的瓶颈。不是把三根柱相加，因为不同资源可同时工作。原教师页的下界符号视觉上写成“>”，最短时间公式应理解为“至少”，流体理想条件下可以达到；严格大于会排除等号而与Q1的构造冲突。

**English answer:** Client-server:  $\max(50000, 7500) = 50000$  s. P2P:  $\max(500, 7500, 15000) = 15000$  s. Upload assistance reduces the aggregate bottleneck but does not remove the other constraints.

**变式 / Transfer:** N改为10, 其他条件不变, P2P会比client-server快吗? / Recompute both bounds for ten clients.

**答案 / Answer**

client-server= $\max(5000, 7500) = 7500$ 秒; P2P= $\max(500, 7500, 150000/37) = 7500$ 秒。此时每人的2Mbps下载是瓶颈, 增加上传互助不会突破它。 / Both are 7500 s, limited by the slowest download rate.

### Problem 3. 不上传的人, 是否仍可能下载完成?

**原题 / Task:** Bob加入BitTorrent但不上传; a能否完整下载? b多台不同IP电脑是否可能让这种free-riding更高效? / Can a non-uploading participant finish, and can multiple hosts improve that possibility?

Tutorial 4教师解答 v2, 第9—11张幻灯片答两个问题均**可能, B**。除了优先服务回馈较好的peer, 课堂BitTorrent模型还有optimistic unchoking: 周期性试着给某个新peer机会。如果有足够peer停留足够久, Bob可能逐渐收齐; 参与者过早离开则未必能完成。

多主机可各自获得机会并收集不同片段后合并, 教师称为一种Sybil式行为。这是机制分析; 实际速率还受peer数量、停留时间、网络资源等影响, 不能把主机数量乘上去就断言下载速度同比增加。

**English answer:** Both claims are possible under the stated classroom assumptions. Optimistic unchoking can supply a free-rider; multiple identities may collect complementary chunks. Neither completion nor proportional speedup is guaranteed.

**自测 / Check:** 如果所有peer在Bob收齐之前离开, 前面的结论是否矛盾? / Does early peer departure contradict the possibility claim?

**答案 / Answer**

不矛盾, 原答案有停留时间和供给条件。“可能”不等于“必然”。 / No; the sufficient opportunity may not exist in that swarm.

### Problem 4. DASH到底存N份, 还是N<sup>2</sup>份?

原题有N个视频质量版本和N个音频质量版本。开头说任意选择音视频, a的括号又明确要求按质量一一配对; 教师解答v2第14张幻灯片按一一**配对**作答, 因此本题a为**N, A**。b分离保存音频和视频, 需**2N, B**, 客户端负责同步。

以 $N=3$ 看清：固定高配高、中配中、低配低，预合并文件只有3份。若真正允许任意组合，要把 $3 \times 3$ 种配对都提前合并，则是9份。若分轨，保存3视频+3音频共6份，播放时组合即可。这三种情境不是互相推翻，差别在配对约束。

**English answer:** Under part a's explicit one-to-one matching assumption,  $N$  combined files are needed. Separate audio and video require  $2N$  files.  $N^2$  applies only when every pair is precombined. These counts concern representations, ignoring segment multiplication.

**独立题 / Transfer:** 视频4种、音频2种，全部预组合与分轨分别几份？ / Four video versions and two audio versions: all pairs versus separate tracks.

答案 / Answer

8与6。不是 $4^2$ ，因为两边版本数量不同。 / Eight combined files or six separate tracks.

## 复习与来源索引

---

原题p1含Q1—3，p2含Q4。Tutorial 4教师解答 v2，第1—5张幻灯片对应Q1，Tutorial 4教师解答 v2，第6—8张幻灯片对应Q2，Tutorial 4教师解答 v2，第9—11张幻灯片对应Q3，Tutorial 4教师解答 v2，第12—15张幻灯片对应Q4。原题数值、单位及括号条件均保留；图为按原数值生成的教学图。

复习沿用N234、N235、N236。Q3可结合optimistic unchoking，复述为什么答案是“可能”。

# Tutorial 5 · Demultiplexing, Checksums and Reliable Transfer

## 写清端点，算清位，再用反例检查协议假设

Chapter 3（已下载部分）〔本册 p.35〕 · 课程目录 · 上一份：Tutorial 4〔本册 p.65〕

依据当前原题2页，共四题；当前未下载教师解答。以下答案根据Chapter3推导，并对照历史问题核查。

基础薄弱时先补二进制与字节〔基础册 p.38〕、时间轴〔基础册 p.40〕。Q1–3为核心必会、中等难度，检查端点方向与反码；Q4也是核心必会，但难度较高，需要构造“旧包被当新包”的具体事件顺序。优先级直接来自当前原题。

### Q1. How is a socket identified?

**题目 / Question:** UDP socket怎样完整标识？TCP呢？区别是什么？ / Compare identification in the instructor's UDP and established–TCP demultiplexing models.

按Chapter 3 part 1，第11、13张幻灯片的普通模型：UDP接收socket看目的IP、目的端口；已建立TCP连接看源IP、源端口、目的IP、目的端口。同一个UDP接收socket可以接收多个来源的数据报，TCP按不同四元组区分连接。

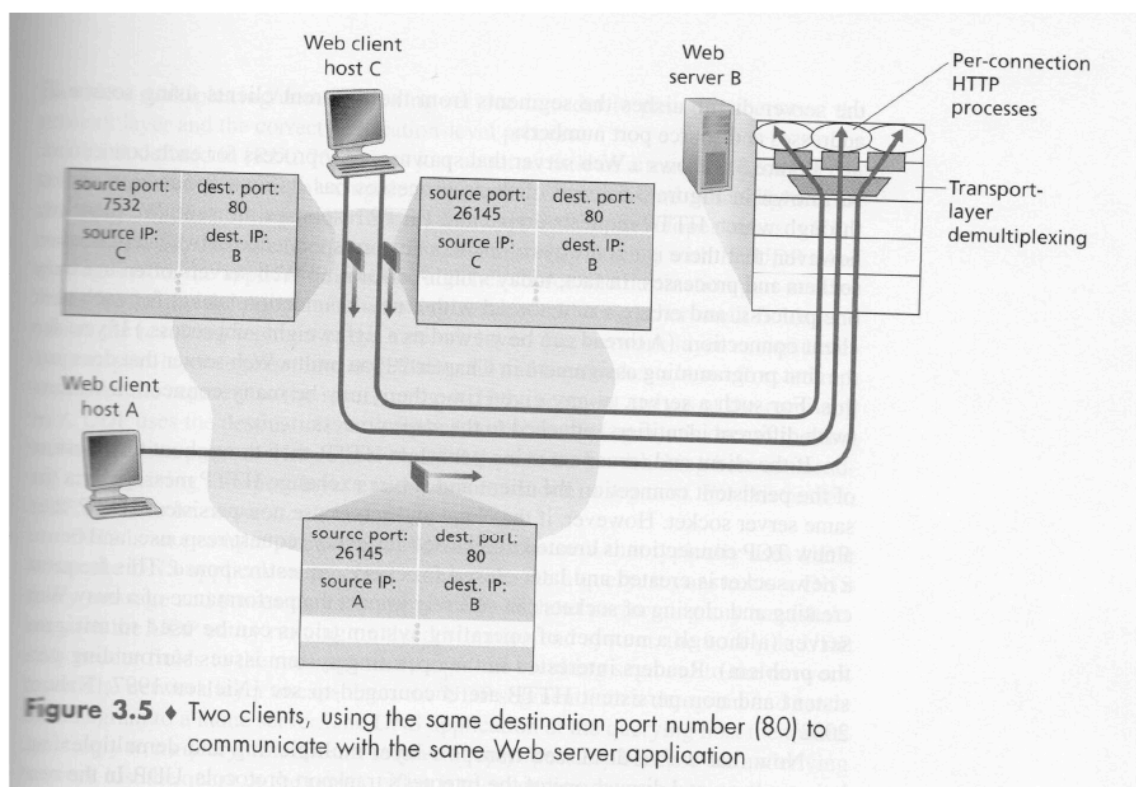
术语“完整标识”这里按课件抽象回答；监听socket和已建立连接不同，真实系统也允许connected UDP。本题按上述课堂模型作答。

**English answer:** The classroom UDP model uses the destination IP and destination port. An established TCP connection is identified by a four–tuple containing both endpoints. Different TCP clients can therefore share the same server port without sharing a connection.

## Q2. Server replies | 回程要把两端一起交换

Tutorial 5, CS5222, Sem A, 2026-27

1. How is a UDP socket fully identified? What about a TCP socket? What is the difference between the full identification of both sockets?
2. Consider the figure below. What are the source and destination port values in the segments flowing from the server back to the clients' processes? What are the IP addresses in the network-layer datagrams carrying the transport-layer segments?



3. UDP and TCP use 1s complement for their checksums. Suppose you have the following three 8-bit bytes: 00100011, 01001110, 01010100. What is the 1s complement of the sum of these 8-bit bytes? (Note that although UDP and TCP use 16-bit words in computing the checksum, for this problem you are being asked to consider 8-bit sums.) Show all work. With the 1s complement scheme, how does the receiver detect errors? Is it possible that a 1-bit error will go undetected? How about a 2-bit error?

**完整条件 / Conditions:** 原图有服务器B:80; 客户端C两条连接分别从7532和26145发起, 客户端A一条连接从26145发起。问服务器回程segment的源/目的端口及外层IP源/目的地址。 / Read all three return paths from the server to the client processes.

| 对应请求         | 回程源端口 | 回程目的端口 | IP源→目的 |
|--------------|-------|--------|--------|
| C:7532→B:80  | 80    | 7532   | B→C    |
| C:26145→B:80 | 80    | 26145  | B→C    |
| A:26145→B:80 | 80    | 26145  | B→A    |

端口写在传输层首部，IP地址写在网络层首部。图中的A/B/C代表主机IP，答案沿用这些符号。

**English answer:** The server's source port is 80 on all three replies. Destination ports are 7532, 26145 and 26145, with IP destinations C, C and A. The last two connections differ in client IP.

**变式 / Transfer:** 服务器改为8443，客户端C的7532仍不变，回程端口怎样写？ / Change only the server port to 8443.

**答案 / Answer**

8443→7532；IP仍B→C。 / Source 8443, destination 7532; IP endpoints remain B and C.

### Q3. One's-complement checksum | 八位是题目的简化

原题给三个8bit字节：00100011、01001110、01010100，要求逐步求和、取反，说明接收端检查及1bit/2bit错误检测。真实UDP/TCP使用16bit字；此处按8bit演算。

```

  00100011  = 35
+ 01001110  = 78
-----
  01110001  = 113
+ 01010100  = 84
-----
  11000101  = 197

```

没有最高位进位，按位取反得到checksum **00111010** (58)。接收端对数据与checksum采用同样反码加法：197+58=255，即11111111。若有超出8bit的进位，必须先卷回低位，再继续检查；本题恰好没有，不代表一般情况下可以省略。

一个bit翻转会改变加权和而被检测。两个bit可能相消：把第一个字的最低位从1改0（35变34），第二个字的最低位从0改1（78变79），总和仍197，checksum不变，错误可能漏检。这是具体反例，不是“两个bit一定漏检”。

**English answer:** Sum 35+78+84=197 (11000101), then complement to 00111010. The receiver expects an all-one one's-complement sum including the checksum. Single-bit corruption is detected; two compensating changes can be missed.

**独立题 / Transfer:** 数据只有11111111和00000010, 8bit校验和是什么? / Compute the toy 8-bit checksum for 11111111 and 00000010, including end-around carry.

**答案 / Answer**

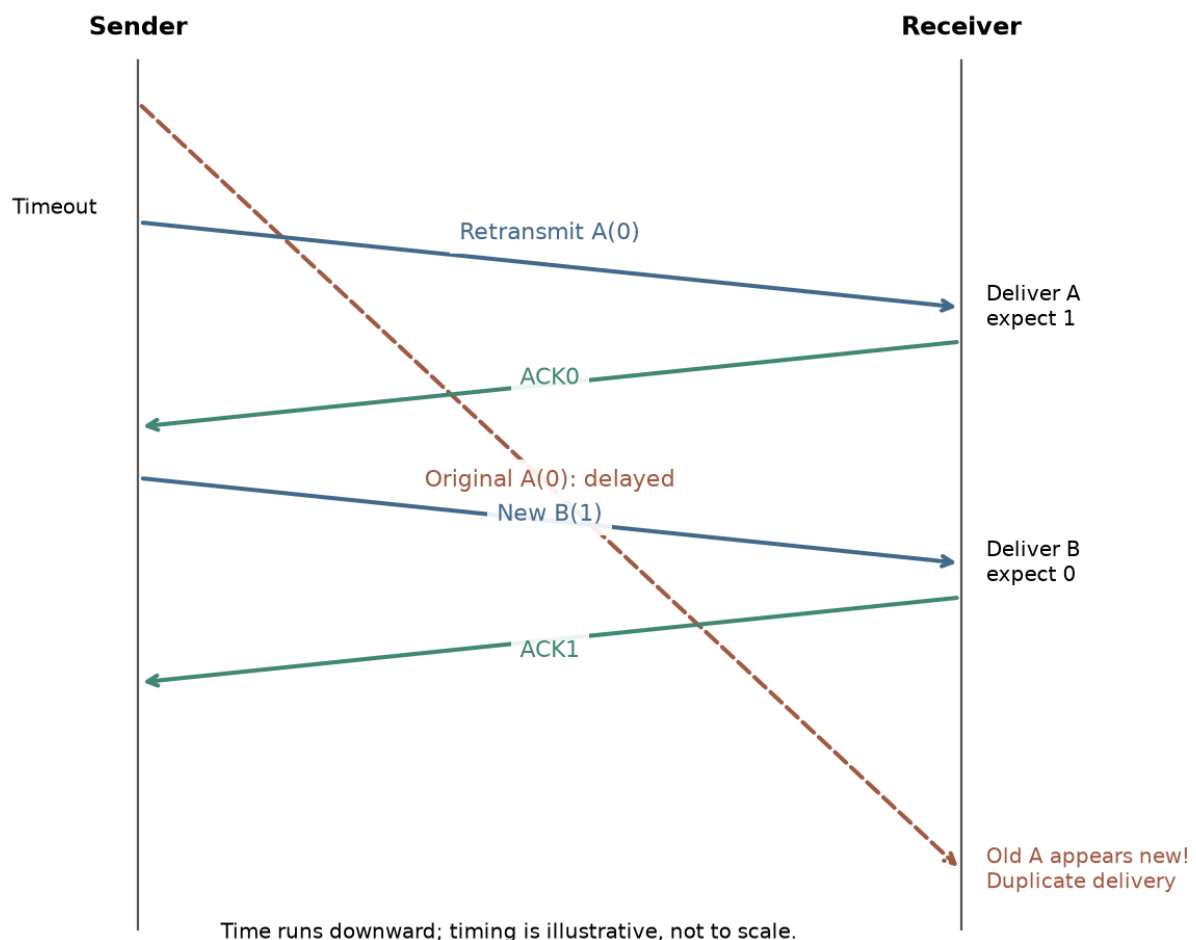
普通和257=1 00000001, 卷回后00000010, 再取反11111101。 / Fold the carry to get 2, then complement to 253.

## Q4. Alternating bit | 编号又回到0, 旧包也可能叫0

**原任务 / Task:** rdt3.0的信道允许消息重排时, 只有0/1序号的交替比特协议是否总能正确? 若不能, 画发送者左、接收者右、时间向下的数据/ACK交换, 标明序号。 / Give a reordering counterexample with explicit packet and ACK numbers.

只说“乱序会出错”不够, 需要让接收端在错误时刻恰好期待旧包的编号。构造如下:

1. 发送数据A, 编号0; 它在网络中很慢, 暂未到。
2. 发送端超时, 重传A(0)。重传先到, 接收端交付A并回ACK0。
3. 发送端收到ACK0, 发送B(1)。接收端交付B、回ACK1, 重新等待0。
4. 最早那份A(0)此时才到。接收方期望0且校验通过, 于是误把A当下一条新数据, 再次交付。



此例只用超时造成的重传和允许的重排，不要求把随机损坏伪装成有效包。收到第三个数据时接收方没法仅凭“0”知道它是老A还是新C。说明可靠性结论依赖对信道和旧副本寿命的假设。

**English answer:** A delayed original packet 0 can arrive after its retransmission and packet 1 have already been accepted. The receiver expects 0 again and can deliver the stale data as new. Thus alternating-bit correctness does not extend to arbitrary reordering and delayed duplicates without additional assumptions.

**迁移 / Transfer:** 只把序号从2种改为4种，是否能无条件解决“任意长延迟”？ / Does a larger finite sequence space eliminate arbitrarily delayed old duplicates?

#### 答案 / Answer

不能无条件保证；编号最终仍会重用，还需要寿命、窗口或其他协议约束。更多序号可以扩大安全范围，但不替代假设。 / No finite sequence space alone prevents a sufficiently old duplicate from surviving until number reuse.

## 来源与复习

---

原题p1为Q1–3及完整端点图，p2为Q4。Chapter 3 part 1，第9–15张幻灯片对应端点，Chapter 3 part 1，第19–20张幻灯片对应校验，Chapter 3 part 1，第35–44张幻灯片对应编号、超时与重复处理；Q4反例为按原条件整理的推导。

沿用N150、N237、N238。Q2可在表格里换一组端口，自行写出回程。

## CS5222–Assignment01

# Assignment 1 · Chapters 1 and 2 | 计算、事务与技术证据

课程目录 · Chapter1〔本册 p.2〕 · Chapter2〔本册 p.19〕 · 原任务（4页）

本讲义提供P6、P31、P17的纸笔推导及Zoom资料核查方法。当前未提供教师解答，以下推导与资料分析供学习参考。截止日期、提交格式和手续回查原任务及课程信息栏。

基础可选：单位与时间轴〔基础册 p.37〕、报文字段〔基础册 p.39〕。当前四题都属于核心任务；P6难点是首/末位事件，P31是跨链路重叠，P17是跨会话状态，Zoom是产品情境与来源边界。理解难度均为中等。

## 本页目录 / Contents

- Assignment 1 · Chapters 1 and 2 | 计算、事务与技术证据〔本册 p.74〕
  - P6. Transmission and propagation | 最后一位刚出发，第一位可能已经到达〔本册 p.75〕
  - P31. Message segmentation | 像流水线一样，让不同链路同时工作〔本册 p.77〕
  - P17. POP3 | 完成整次事务，不只补两行〔本册 p.80〕
    - a. Download and delete / 下载并删除〔本册 p.81〕
    - b. Download and keep / 下载并保留〔本册 p.81〕
    - c. Second keep-mode session / 五分钟后的第二次会话〔本册 p.82〕
  - Zoom. Apply protocol ideas with sources | 先说研究的是哪一种Zoom连接〔本册 p.83〕
    - a. 应用协议：产品生态的三项，与原生媒体路径分开〔本册 p.83〕
    - b. 架构 / Architecture〔本册 p.84〕
    - c. 传输 / Transport〔本册 p.84〕
    - d. 老师电脑的IP / Lecturer's host IP〔本册 p.84〕
    - 怎样整理成自己的作答〔本册 p.84〕
  - 覆盖、复算与验收〔本册 p.84〕

## P6. Transmission and propagation | 最后一位刚出发，第一位可能已经到达

### CS5222 Assignment 1 (on Chapter 1 and 2)

#### Chapter 1

- P6. This elementary problem begins to explore propagation delay and transmission delay, two central concepts in data networking. Consider two hosts, A and B, connected by a single link of rate  $R$  bps. Suppose that the two hosts are separated by  $m$  meters, and suppose the propagation speed along the link is  $s$  meters/sec. Host A is to send a packet of size  $L$  bits to Host B.
- Express the propagation delay,  $d_{\text{prop}}$ , in terms of  $m$  and  $s$ .
  - Determine the transmission time of the packet,  $d_{\text{trans}}$ , in terms of  $L$  and  $R$ .
  - Ignoring processing and queuing delays, obtain an expression for the end-to-end delay.
  - Suppose Host A begins to transmit the packet at time  $t = 0$ . At time  $t = d_{\text{trans}}$ , where is the last bit of the packet?
  - Suppose  $d_{\text{prop}}$  is greater than  $d_{\text{trans}}$ . At time  $t = d_{\text{trans}}$ , where is the first bit of the packet?
  - Suppose  $d_{\text{prop}}$  is less than  $d_{\text{trans}}$ . At time  $t = d_{\text{trans}}$ , where is the first bit of the packet?
  - Suppose  $s = 2.5 \cdot 10^8$ ,  $L = 120$  bits, and  $R = 56$  kbps. Find the distance  $m$  so that  $d_{\text{prop}}$  equals  $d_{\text{trans}}$ .

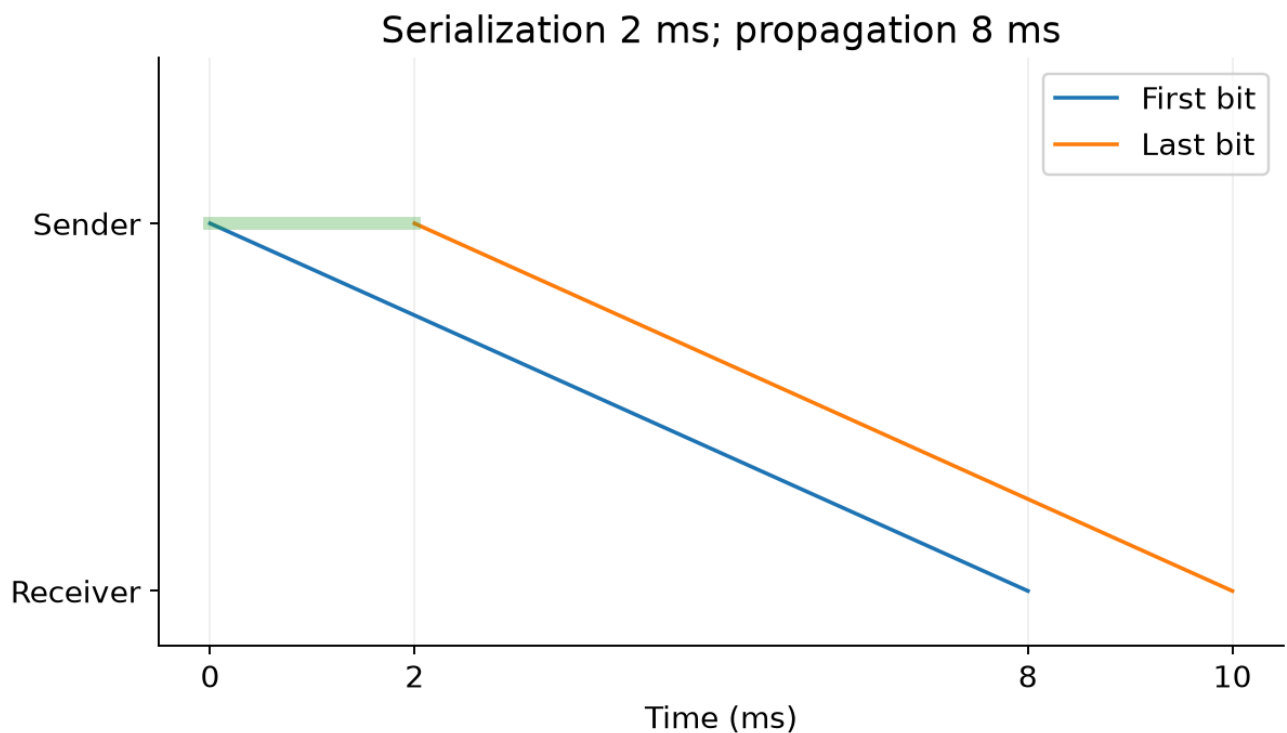
**条件 / Conditions:** 主机A、B相距 $m$ 米，链路发送速率 $R$  bit/s，传播速度 $s$  m/s，包长 $L$  bit。忽略处理、排队，从A开始发送首位计时 $t=0$ 。 / Hosts A and B are  $m$  metres apart; link rate is  $R$  bit/s, propagation speed  $s$  m/s, and packet length  $L$  bits. Ignore processing and queueing and start timing with the first bit.

| 原小问                          | 中文推导与答案                          | English answer                               |
|------------------------------|----------------------------------|----------------------------------------------|
| a 传播                         | $d_{\text{prop}} = m/s$ ，距离除速度   | Propagation delay is $m/s$ .                 |
| b 发送                         | $d_{\text{trans}} = L/R$ ，数据量除速率 | Serialization time is $L/R$ .                |
| c 整包端到端                      | $L/R + m/s$ ，末位先被送入链路再传播         | Complete-packet delay is $L/R + m/s$ .       |
| d $t=d_{\text{trans}}$ ，末位位置 | 在连续比特模型的边界时刻刚离开A进入链路，还没有走完整条路    | The last bit is just entering the link at A. |

| 原小问                             | 中文推导与答案                    | English answer                                           |
|---------------------------------|----------------------------|----------------------------------------------------------|
| e $d_{prop} > d_{trans}$ , 首位位置 | 仍在线路内, 离A的距离为 $sL/R < m$   | The first bit remains on the link, $sL/R$ metres from A. |
| f $d_{prop} < d_{trans}$ , 首位位置 | 已到达B; 不能继续说还在链路上           | The first bit has already reached B.                     |
| g 两时延相等                         | $m/s = L/R$ , 故 $m = sL/R$ | Set $m = sL/R$ .                                         |

g 给  $s = 2.5 \times 10^8 \text{ m/s}$ ,  $L = 120 \text{ bit}$ ,  $R = 56 \text{ kbps} = 56000 \text{ bit/s}$ :

$$d_{trans} = 120/56000 \approx 0.002142857 \text{ s}, \quad m \approx 535714.29 \text{ m} = 535.714 \text{ km}. \quad (\text{A1.1})$$



图是帮助读事件的教学示意 (参数2ms/8ms), 不是g题的参数图。g的数值单独复算, 避免把示意图标注抄进答案。

**变式 / Transfer:**  $L = 240 \text{ bit}$ ,  $R$  不变, 要保持两时延相等,  $m$  如何变化? / Double packet length while keeping  $R$  and  $s$  fixed; how must  $m$  change?

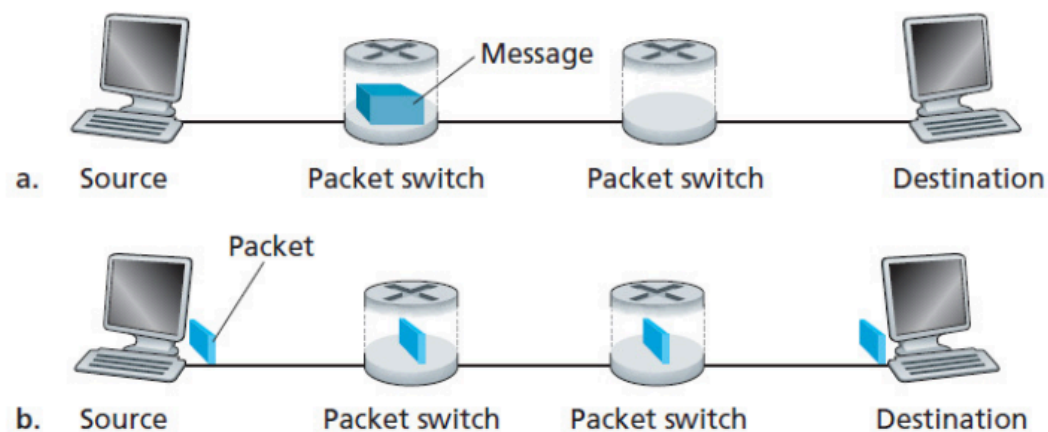
**答案 / Answer**

$m$  翻倍, 约1071.429km。 /  $m$  doubles to about 1071.429 km. 改变的是等时延所需距离, 不是信号速度。

## P31. Message segmentation | 像流水线一样，让不同链路同时工作

P31. In modern packet-switched networks, including the Internet, the source host segments long, application-layer messages (for example, an image or a music file) into smaller packets and sends the packets into the network. The receiver then reassembles the packets back into the original message. We refer to this process as *message segmentation*. Figure 1.27 illustrates the end-to-end transport of a message with and without message segmentation. Consider a message that is  $8 \cdot 10^6$  bits long that is to be sent from source to destination in Figure 1.27. Suppose each link in the figure is 2 Mbps. Ignore propagation, queuing, and processing delays.

- Consider sending the message from source to destination *without* message segmentation. How long does it take to move the message from the source host to the first packet switch? Keeping in mind that each switch uses store-and-forward packet switching, what is the total time to move the message from source host to destination host?
- Now suppose that the message is segmented into 800 packets, with each packet being 10,000 bits long. How long does it take to move the first packet from source host to the first switch? When the first packet is being sent from the first switch to the second switch, the second packet is being sent from the source host to the first switch. At what time will the second packet be fully received at the first switch?
- How long does it take to move the file from source host to destination host when message segmentation is used? Compare this result with your answer in part (a) and comment.
- In addition to reducing delay, what are reasons to use message segmentation?
- Discuss the drawbacks of message segmentation.



**Figure 1.27** ♦ End-to-end message transport: (a) without message segmentation; (b) with message segmentation

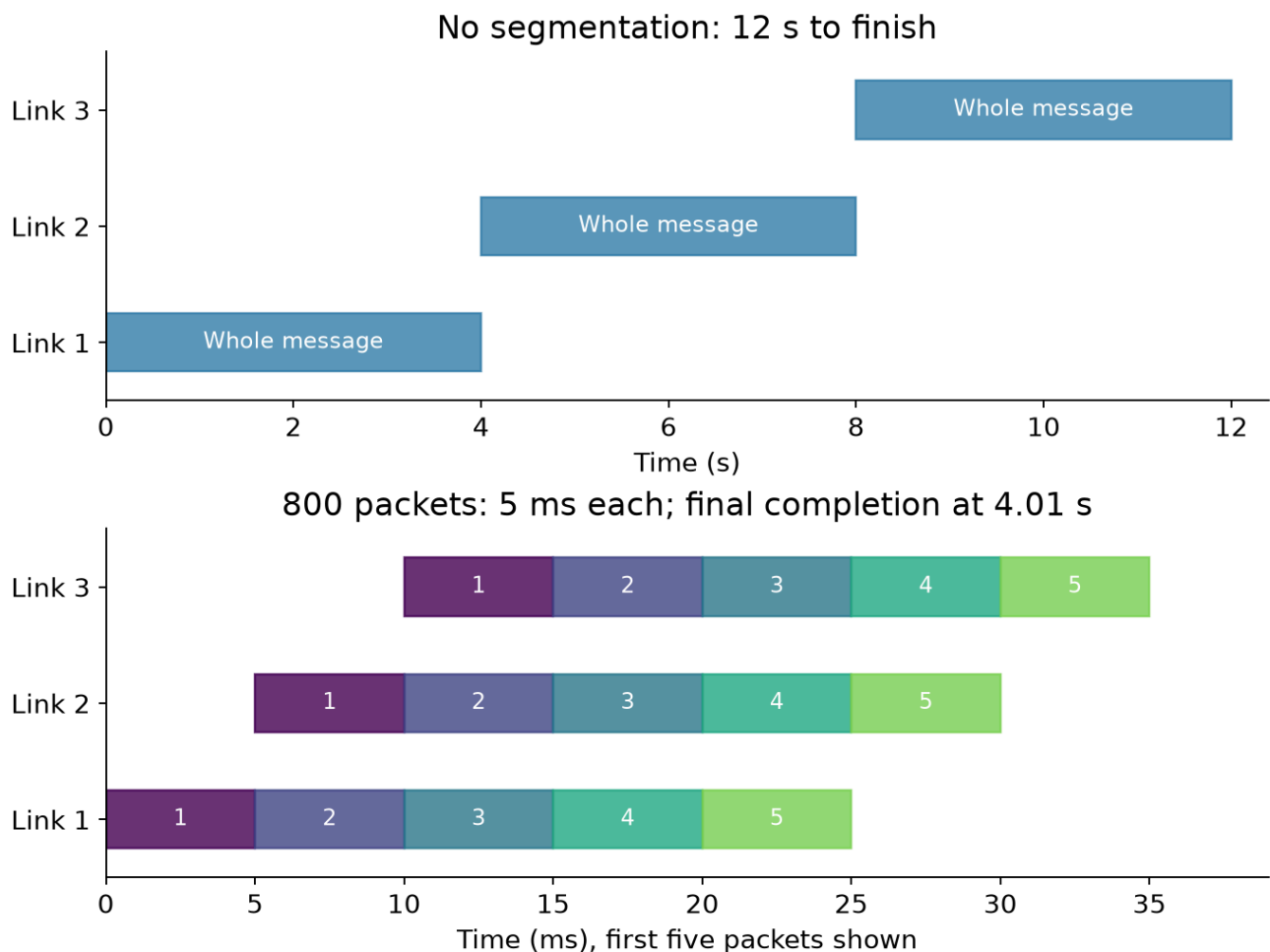
条件 / Conditions: 原图源→交换机1→交换机2→目的，共3条2Mbps链路；消息 $8 \times 10^6$ bit，存储转发，忽略传播、排队和处理。分段时为800个10000bit包，题目未另加包头。 / An 8-million-bit message crosses three 2-Mbps store-and-forward links. Ignore propagation, queueing and

processing. Segmentation creates 800 packets of 10000 bits each, with no additional header overhead specified.

a 不分段，第一链路  $8 \times 10^6 / (2 \times 10^6) = 4\text{s}$ ；交换机必须等整条消息，3跳总**12s**。 / Without segmentation: 4 s to the first switch and 12 s to the destination.

b 单包发送  $10000 / (2 \times 10^6) = 0.005\text{s} = 5\text{ms}$ 。第一包到第一交换机  $t = 5\text{ms}$ ；它继续去第二交换机时，源发送第二包，第二包在  $t = 10\text{ms}$  完全到第一交换机。 / First packet reaches switch1 at 5 ms; packet2 is completely received there at 10 ms.

c 第一包到目的需  $3 \times 5\text{ms}$ ，后续799包每隔5ms到，总  $(800 + 3 - 1) \times 0.005 = 4.01\text{s}$ 。 / Completion time is 4.01 s. 链路本身没变快，减少的是“大消息在每一跳全等完”的串行等待。



图中展开前几个包，末尾标出全部800包的完成时刻。

d 其他好处：较短包给其他流量穿插机会；出现丢失时可只重传受影响部分（若上层协议支持）；较小存储/处理单位便于逐包转发。 / Segmentation enables interleaving, selective retransmission when supported, and manageable forwarding units.

e 代价：每包头部和处理开销，更多调度；可能乱序，需要识别与重组；重传/超时机制增加复杂度。 / Costs include headers, per-packet processing, possible reordering, reassembly and recovery complexity. 不把“越小永远越快”当结论；本题忽略头部是有意简化。

**变式 / Transfer:** 同消息改成400包, 每包20000bit, 其他不变, 总多久? / Use 400 packets of 20000 bits with the same path.

**答案 / Answer**

每包10ms, 总 $(400+3-1) \times 0.01 = 4.02\text{s}$ 。 / 4.02 s. 仍近似受源端总上传4s约束, 只是流水线填充稍慢。

## P17. POP3 | 完成整次事务，不只补两行

### Chapter 2

P17. Consider accessing your e-mail with POP3.

- a. Suppose you have configured your POP mail client to operate in the download-and-delete mode. Complete the following transaction:

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .....blah
S: .
?
?
```

- b. Suppose you have configured your POP mail client to operate in the download-and-keep mode. Complete the following transaction:

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .....blah
S: .
?
?
```

(Note: For parts a and b above, you should continue writing the transaction until the service is finished. It is not limited to only the next two lines.)

- c. Suppose you have configured your POP mail client to operate in the download-and-keep mode. Using your transcript in part (b), suppose you retrieve messages 1 and 2, exit POP, and then five minutes later you again access POP to retrieve new e-mail. Suppose that in the five-minute interval no new messages have been sent to you. Provide a transcript of this second POP session.

**完整条件 / Conditions:** 已认证进入transaction, LIST给两封邮件, 编号1/2、大小498/912byte; 原片段已经RETR1. a为download-and-delete, b为download-and-keep, 均继续到服务完成。c在b后退出, 5分钟后再次登录, 无新邮件, 给出第二会话。 / Two messages of 498 and 912 octets

are present. Finish the full download-and-delete and download-and-keep transactions, then show a second keep-mode session five minutes later with no new mail.

原题略去若干+OK确认；下面补齐规范响应。[message ...]只是未知邮件内容的说明占位，不是实际发送的协议文本。每行以CRLF结束，单独.结束多行结果。命令字大小写不影响本题含义。

### a. Download and delete / 下载并删除

```
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: DELE 1
S: +OK message 1 marked for deletion
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
C: DELE 2
S: +OK message 2 marked for deletion
C: QUIT
S: +OK signing off
```

DELE先标记，成功QUIT使会话进入UPDATE并提交删除；本例假设操作成功且没有异常退出。只写DELE1然后QUIT会漏掉第二封的下载与删除。 / Retrieve both messages, mark both for deletion, then successfully quit to commit the update.

### b. Download and keep / 下载并保留

```
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
C: QUIT
S: +OK signing off
```

没有DELE，两封保留。RETR不会自动删除；“keep”不是另加一个名叫KEEP的POP3命令。 / No messages are marked for deletion, so both remain on the server.

### c. Second keep-mode session / 五分钟后的第二次会话

假设没有额外客户端UIDL去重、服务器未改变这两封邮件的编号，重新认证后仍LIST两封；可以再次RETR1/2。原问题仅说“无新邮件”，没有说服务器将已读邮件隐藏。

```
S: +OK POP3 server ready
C: USER <username>
S: +OK
C: PASS <password>
S: +OK authenticated
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
C: QUIT
S: +OK signing off
```

尖括号内为用户名和密码的占位。实际客户端可保存UIDL跳过已下载邮件；若采用这种扩展约定，可能LIST/UIDL核对后不RETR旧邮件。应明确这是客户端保存状态的行为，而不是“POP3服务器因为上次读过就删掉”。RFC1939（查阅于2026-09-23）

**English answer:** With no deletion and no client-side duplicate suppression, both messages are still available and can be downloaded again. State any UIDL-based client behavior explicitly.

**变式 / Transfer:** b中仅DELE1并成功QUIT，之后无新邮件，剩多少总字节？ / Delete only message1 in part b and successfully quit. How many message octets remain?

#### 答案 / Answer

只剩原邮件2，912byte；新会话编号不应依赖旧编号恒定。 / 912 octets remain; message numbers need not remain identical across sessions.

## Zoom. Apply protocol ideas with sources | 先说研究的是哪一种Zoom连接

**Question.** Zoom (<https://zoom.us/>) is a videoconferencing application. After doing some research about its communication protocols, answer the following questions:

- What kind of application-layer protocols do Zoom use? Please state three of them and briefly discuss their functions.
- Is Zoom primarily based on a client-server or P2P architecture? Why?
- Which transport layer protocol is Zoom primarily using? Why?
- During our CS5222 lecture using Zoom, does a student's computer need to obtain the IP address of the lecturer's computer? If so, how? If not, why?

**原问题 / Questions:** a列举三种应用层协议并解释作用；b主要client-server还是P2P；c主要传输协议及原因；d学生是否需要老师电脑IP及原因。 / Identify three application-layer protocols and their roles, explain the main architecture and transport choice, and determine whether a student needs the lecturer's host IP.

**核查情境：2026-09-23，普通多人Zoom Meetings、原生Zoom Workplace客户端、经Zoom云入会的默认讨论范围。** 没有实际抓取本校课堂流量，也未确认本校启用哪些企业功能。因此下述为官方资料支撑的典型架构分析，不是对某一次课堂连接的实测报告。

### a. 应用协议：产品生态的三项，与原生媒体路径分开

题目未限定客户端类型，可用下表列出Zoom产品生态明确支持的三项，并把各自情境写进答案：

| 协议             | 功能（中文）                     | English explanation                                                      | 证据/适用情境                                   |
|----------------|----------------------------|--------------------------------------------------------------------------|-------------------------------------------|
| HTTPS          | 网页/客户端与Zoom Web服务的请求、资源获取等 | Secure HTTP access to web services and content                           | 官方网络配置明确HTTP/HTTPS访问与CDN；不据端口猜全部音视频都经HTTP |
| SIP            | 让标准会议室端点进行会话建立/互通          | Session signaling for supported standards-based room endpoints           | Conference Room Connector；不是所有原生客户端必经路径   |
| H.323<br>(协议族) | 另一类标准音视频会议端点经连接器接入         | A conferencing protocol suite for supported room-system interoperability | 同一官方连接器资料；需注明它是协议族而非单一报文协议                |

表中HTTPS用于Web服务，SIP/H.323用于会议室端点互通，作答时分别注明情境。若教师将题意限定为**原生客户端自身的媒体/控制三种协议**，应按该范围补充特定版本的技术资料，而不能把SIP/H.323互通项代替原生客户端路径的说明。公开端口表只能证明传输端口用途，不能单独证明RTP/SRTP等内部封装；作答时应注明所选客户端与部署情境。

## b. 架构 / Architecture

普通多人云会议主要由云服务协调与转发，按课程分类主要是client-server：各客户端向服务基础设施连接，云处理参与者发现、会话与媒体分发。**这是基于官方云连接/回退说明的架构归纳**，具体部署也可能包含下面介绍的P2P或Mesh连接。Zoom Mesh官方说明

**English:** Ordinary multi-participant cloud meetings are primarily client-server in the course's sense; clients rely on Zoom infrastructure for coordination and media distribution.

明确例外：官方允许同一局域网的两人会议在符合设置条件时用peer-to-peer；加入第三人等情况会使用云。两人P2P说明 企业启用Zoom Mesh时，局域网内可由parent客户端给child分发视频；这是有云协调的混合机制，不能拿来证明普通多人课堂始终直接连接老师。

## c. 传输 / Transport

实时媒体通常优先UDP，官方DSCP文档说明默认媒体在UDP8801等端口，特定管理设置可拆分音频/视频/共享端口。官方媒体端口说明 控制/Web访问及媒体回退可以使用TCP；“Zoom只用UDP”错误。课程层面的原因是实时音视频更在意及时性，可由应用处理丢失与适应码率，而TCP有序交付可能让后续数据等重传；这解释选择动机，不意味UDP本身自动更可靠或保证低延迟。

**English:** Real-time media commonly prefers UDP, while control/web traffic and fallback paths may use TCP. Low delay and application-level adaptation matter more than waiting indefinitely for every media byte.

## d. 老师电脑的IP / Lecturer's host IP

在普通云课堂情境，学生连接的是Zoom提供的服务端点，不需要自己先解析或输入老师个人电脑的IP。老师和学生各自向基础设施建立通信，服务负责转发；知道meeting ID也不是把它翻译成老师IP。

**English:** In an ordinary cloud meeting, the student does not need to obtain the lecturer's host IP; Zoom infrastructure routes the session. Peer/mesh configurations may use other local endpoints and require separate analysis.

## 怎样整理成自己的作答

每个判断写成“结论 → 明确情境 → 机制 → 官方来源及查阅日期”。区分产品支持、当前默认与本机实测三种证据。原题是应用课程知识的调查题，不应把一段不知年份的博客当软件永久架构。

## 覆盖、复算与验收

原PDF p1的P6(a–g)、p2的P31(a–e)及原拓扑、p3的P17(a–c)及“写到事务结束”要求、p4的Zoom(a–d)全部对应。数值和分包图由 network\_examples.py 复算；原题未给邮件正文，事务中使用占位说明。

完成前检查：每个速率/长度有单位，每段延迟不多算漏算；POP3包含两封邮件与成功结束；Zoom区分客户端/部署并注明来源时间。再回到已有N260–N263复习对应知识。

# Research Report Guide | 把一个真实问题解释清楚

课程目录 · Chapter1〔本册 p.2〕 · Chapter2〔本册 p.19〕 · 课程信息栏

这份指南帮助你选题、解释机制、核查资料和整理反思。原任务要求研究你在日常生活、新闻或工作中遇到的新网络技术，联系教材Chapter1–7，排除Chapter8安全主题；用自己的解释和图说明机制，并反思学习过程。格式、页数、评分与提交要求以Canvas中的当前原任务与Rubric为准。

按选题补基础：涉及码率/吞吐就回网络单位〔基础册 p.37〕，涉及状态/时间就回报文与时间轴〔基础册 p.39〕。没有实际卡点便直接开始。

| 能力        | 优先级  | 难度与卡点       | 达标表现；依据              |
|-----------|------|-------------|----------------------|
| 明确问题、解释机制 | 核心必会 | 中至高；不能只有名词  | 不看原文也能画出数据/控制流；原任务A  |
| 来源核查与边界   | 核心必会 | 中；资料版本/适用场景 | 每个关键结论有对应证据；原任务p2    |
| 图文与英语表达   | 核心必会 | 中；精简与可读     | 图能帮助解释一个具体难点；原Rubric |
| 真实反思      | 核心必会 | 中；具体变化而非套话  | 记录自己确实做过的核查与困难；原任务B  |
| 复杂数学/未用机制 | 按需拓展 | 可能高         | 只有支撑论点时补；整理者建议，不暗示不考 |

## 1. 选题从一件事开始，不从一串热门缩写开始

先记录你**确实遇到**的情况：某个应用在带宽变化时切画质、同一内容从不同地点下载速度不同、多人共享网络时服务突然卡顿。题目应围绕一个能解释的机制问题，例如“客户端怎样在吞吐波动时选择下一视频块的编码版本”。

用三个问题缩小范围：这个问题属于网络吗？能对应教材哪一层/哪章？在本次篇幅内能用一幅图、一个小场景说明主要机制吗？“所有5G技术”往往太宽；“某种特定机制在什么条件下改善什么问题”更容易论证。选题的新意应落在具体变化、部署或问题上。若选DASH等课内基础，需说明你研究的具体新变化、部署或问题，不能只复述课件。

**English framing (整理表达):** *This report examines how [a specific mechanism] addresses [a concrete networking problem] under [a defined setting].* 方括号中填写你的具体问题、机制和调查情境。

## 2. 把问题、机制、效果连成一条可核查的解释

先写旧方法遇到什么限制，再写新机制改了哪一步，最后讨论收益与代价。不要“协议很先进，所以性能很好”这样跳过中间因果链。

以分块视频为**教学例**：播放每秒消耗编码数据 → 网络吞吐会变动 → 缓冲可能耗尽 → 客户端按估计吞吐/缓冲选择下一块 → 可能减少停顿，但画质也会变化。这里每条箭头都能被课程概念支持，而不是把“自适应”三个字当解释。

**完整补算 / Worked example:** 4秒片段编码2Mbps，数据量8Mbit；若稳定吞吐4Mbps，下载2秒。在缓冲足够、同时播放、忽略请求与解码开销时，缓冲净增加4−2=2秒。 / A four-second segment encoded at 2 Mbps contains 8 Mbit. At 4 Mbps throughput it downloads in two seconds, increasing the playable buffer by two seconds under the stated assumptions.

**变式 / Transfer:** 网络降至1Mbps，同一块需几秒？若开始时只缓冲3秒会发生什么？ / At 1 Mbps, how long does the same chunk take, and what if only three seconds are buffered initially?

答案 / Answer

下载8秒，已有缓冲只能撑3秒，理想模型中会停顿约5秒，等块到后才续播。 / Download takes eight seconds; under the simplified model, playback stalls for about five seconds. 缓冲余量和吞吐估计误差都会影响是否停顿。

上面是理想模型的教学算例；实际停顿还取决于播放器策略和网络变化。

3. 找来源时，为每一句重要话准备证据

优先找协议标准、技术提供方文档、原始研究论文或可复核的测量报告。二手讲解和AI适合帮助形成问题，但需要回到原资料核对。阅读来源时，找到直接支持该结论的段落或数据。

| 写作中的主张   | 应找的证据          | 必须检查            |
|----------|----------------|-----------------|
| 协议怎样工作   | 规范/原设计文档       | 版本、消息顺序、可选项、条件  |
| 某产品使用此机制 | 官方当前产品文档       | 原生/浏览器/企业部署是否相同 |
| 性能提高多少   | 原实验/可重复测量      | 对比方法、数据、指标、环境   |
| 有哪些限制    | 规范限制、实验失败或设计分析 | 不把推断写成已测事实      |
| 为什么和课程有关 | 教材/当前课件定位      | 是当前已学，还是教材后续章节  |

可维护一张很小的工作表，放在你自己的研究草稿里：Claim | Source/section | Version/date | Supports what | Does not support what。用它检查每条主张与来源是否对应。

如引用AI输出，原任务要求说明使用并自行负责核查；图、引用也要查，不能只给一个模型名字就当技术来源。对论文核对题名、作者、年份、DOI/官方页面，打开原文确认所引结论和实验设置。查不到原文的来源标为待核查，确认后再引用。

4. 图要解释困难，不是给文字化妆

选择一种与你的核心问题匹配的图：

- 交互问题画时序图，标清参与者、请求、响应、时间方向；
- 架构问题画节点与数据/控制流，说明图中箭头是什么；
- 权衡问题画有单位的曲线，说明数据来自实测、论文还是教学计算。

画完问自己：读者通过这张图能更容易答出哪个问题？节点之间的箭头应说明数据从哪里来、到哪里去，以及每一步做什么。引用原图标来源，改编图标“Adapted from...”，自己计算的图保存脚本和输入。AI图同样要署明并逐条检查技术结构，不能因为好看就默认网络方向正确。

## 5. 英文报告怎样组织才连贯

建议采用内容驱动的小标题，而非每段中英对照：英文完成正式论证，中文草稿帮助自己先讲懂。术语第一次出现给全称与缩写，随后保持一致。可以按以下结构组织：

1. **Problem and context**：你实际怎样接触主题？问题是什么、研究范围到哪里？
2. **Background from the course**：挑真正用到的1–3个概念，解释它们为何必要。
3. **How the mechanism works**：沿一个具体请求/数据流走完步骤，配关键图。
4. **Benefits, limits and evidence**：哪项收益有证据，代价是什么，什么条件下不适用？
5. **Discussion**：你基于证据形成的判断、未解决问题与合理后续方向。

每段先给主张，再解释机制，再放证据与条件。比如 *The mechanism reduces repeated origin fetches when reusable content is available in cache* 比 *It greatly improves everything* 更可核查。不要把“可能”藏起来，也不用给每句话堆三个形容词。

**英文自测 / English self-check:** *What changes, why does it help, and under what conditions might it not help?* / 改变了什么，为何有效，何时可能无效？若不能脱稿回答，正文还需要补机制，而非继续增加术语。

## 6. Reflection log | 写你真的经历过的理解变化

原任务的五个问题，可以分别对应一条具体记录：

| 原问题                                              | 真实记录的抓手              |
|--------------------------------------------------|----------------------|
| What did you learn that you did not know before? | 原来怎么理解，哪个具体事实改变了它    |
| What was the most challenging part?              | 一个真正卡住的机制/推导/资料冲突    |
| How did GenAI help or hinder?                    | 它帮你拆了哪一步，或给过什么误导     |
| How did you verify AI-generated information?     | 打开了哪个原来源、比对哪句、做了什么检查 |
| What would you do differently next time?         | 一个具体可实行的改进           |

可从现在开始记两三条带日期的研究记录，报告结束时据实整理。写清你做过什么、哪里卡住、怎样查证，以及理解发生了什么变化；工具使用情况按实际记录。

## 7. 交付前的自查与来源覆盖

读者应能复述你的问题与机制，认出它对应课程哪部分；一张图能独立读懂；数字都有单位、来源和条件；观点与已验证事实分开。英语表达、引用、AI使用说明、格式与评分要求逐项回查原任务及课程信息。

| 原任务位置                       | 本指南对应              |
|-----------------------------|--------------------|
| p1 Goal、题目范围                | 开篇、§1              |
| p1 Format、提交构成              | 原任务/信息栏链接；不生成正式提交件 |
| p1 A：接触途径、机制、联系、动机/创新/挑战/观点 | §1—2、§5            |
| p1 自己的表达、图与清晰度              | §4—5               |
| p2 GenAI引用、图/参考核查责任         | §3—4               |
| p2 B：五个Reflection问题         | §6逐项对应             |
| p2 Evaluation、p3 Rubric     | 开篇能力表、§7；具体权重以原页为准 |

确定主题后，再围绕核心问题补充必要基础、原始来源和图示。